

Form Neutrino Masses: From Neutral Depth to a Planck-Ratio Normal Hierarchy

Johannes Michael Wielsch

Literate Agda paper in the *Void and Form* programme

May 2026

Abstract

This paper gives a machine-checked rational skeleton for a normal-hierarchy neutrino mass readout inside the *Void and Form* invariant ledger. A finite K_4 candidate space is reduced to the neutral-depth survivor $d + \chi = 5$; an electron-anchored reading is retained as a comparison layer; and a Planck-anchored mass chain is classified through route and exponent filters, a running correction, a sign-ordered correction tower, charged-sector correction data, normal-hierarchy observables, and the neutral residual carrier. The residual carrier is the coupling complement $295/306$, so the final neutral separator is $m_3/m_2 = 6 \cdot 295/306 = 1770/306$. With the displayed Planck input this gives $m_1 = 0$, $m_2 \approx 0.008589080150 \text{ eV}$, $m_3 \approx 0.049681934200 \text{ eV}$, and $\sum_i m_i \approx 0.058271014350 \text{ eV}$. The Agda content is the finite integer and rational chain. The assertion that this chain is the neutrino mass spectrum of nature remains an empirical identification.

Introduction

Oscillation experiments measure neutrino mass splittings, but the absolute mass scale remains outside the Standard Model. A serious prediction of that scale must do more than write down small numbers. It must specify the finite candidate space, show which candidates are eliminated, state every external physical input, and separate the checked arithmetic from the empirical claim that the resulting chain describes the world.

The construction below follows that discipline. The first formal output is the neutral-depth square $5^2 = 25$. The electron-anchored reading is retained as a low-energy comparison layer. The Planck-anchored layer then classifies the route to the electron scale, the suppression exponents, the running factor, the correction tower action, the charged-sector correction data, and the normal-hierarchy observables. None of those choices is selected by decimal agreement with measured neutrino masses; the decimals are read only after the rational chain has been fixed.

The charged and neutral residual layers are treated differently. The charged mass ratios still carry structured ppm-level residuals whose sign pattern belongs to the physical comparison with hadronic and leptonic sectors. The neutral splitting residual is classified at the carrier level. Matching weights are blocked by $\text{Aut}(K_4)$ -uniformity, so the residual acts one layer above the matchings. The surviving carrier is the coupling complement $q - p$ over q , with $q = 306$, $p = 11$, and $q - p = 295$.

Scientific claim and validation

The formal claim of this paper is narrow: within the displayed finite candidate spaces, the rational chain from neutral depth to final Planck ratios is mechanically checked in Agda under `--safe` and

--without-K. The physical claim is stronger and remains exposed: the exact rational readout is proposed as the normal neutrino mass hierarchy of nature.

For the paper to count as a scientific contribution rather than a formal curiosity, six obligations must remain visible.

1. The Agda file must type-check from a clean checkout, without postulates or unsafe flags.
2. The candidate spaces must be explicit enough that the survivor is not a fitted choice hidden behind notation.
3. The Planck input, unit conventions, and low-energy comparison constants must be named rather than absorbed into the derivation.
4. The exact rational outputs must be stated before the electronvolt decimals are compared with data.
5. The comparison with oscillation data, cosmological mass bounds, beta decay, and neutrinoless-double-beta searches must be reproducible from the displayed numbers.
6. The failure conditions must be explicit: an incompatible hierarchy, nonzero lightest mass beyond the stated boundary, a conflicting mass sum, or squared splittings outside the proposed readout would refute the physical identification, not the formal arithmetic.

Structure of the paper

The large programme is a dependency chain. It starts at the boundary argument for distinction, crosses the formal threshold only after the binary datum has been supplied, and ends only when a dimensionful mass number can be compared with experiment. Every link must preserve the status of the previous link.

The intended route is:

distinction $\rightarrow$ binary normal form $\rightarrow K_4$ closure $\rightarrow$ invariant ledger
 $\rightarrow$ number tower $\rightarrow$ spectral/operator layer $\rightarrow$ sector assignment
 $\rightarrow$ mass ratios $\rightarrow$ correction/running $\rightarrow$ absolute scale $\rightarrow$ particle masses.

The paper follows that route in the following order.

1. Use the distinction-to- K_4 kernel supplied by *Void* without reopening its proof.
2. Import only the small invariant ledger needed for the neutral problem: V, E, d, χ, κ , the Fermat slots, loop numerator, and loop volumes.
3. Classify neutral-depth candidates and prove the finite survivor statement for depth 5 and denominator 25.
4. Record the electron-anchored neutrino prediction as a physical comparison candidate, because the charged-mass ledger is expressed in ratios to m_e .
5. Classify the Planck-to-low-energy bridge. The bridge must explain the large suppression from the Planck unit to the electron and neutral scale without inserting a tunable parameter.
6. State the resulting Planck-ratio particle masses and the neutrino observables that follow from them.

7. Compare the resulting masses with experiment, including charged leptons, quarks where the scheme applies, neutrino splittings, total neutrino mass, beta-decay bounds, and neutrinoless-double-beta limits. This comparison is not an Agda theorem; it is the empirical boundary.

```
-- $ Options, module, and imports.
{-# OPTIONS --safe --without-K #-}

module FormNeutrinoMasses where

open import Data.Empty using (⊥)
open import Data.Unit using (τ; tt)
open import Data.Product using (×; _,_; Σ)
open import Data.Nat using (N; +; *; _-; ^)
open import Relation.Binary.PropositionalEquality using (≡; refl; sym; trans; cong)
```

1 Formal and empirical claim ledger

The following record does not reprove upstream theorems. It records which parts are imported from the checked kernel, which parts are proved in this paper, and which parts remain physical tests rather than formal theorems.

```
-- $ ProgrammeStatus.
data ProgrammeStatus : Set where
  checked-existing : ProgrammeStatus
  checked-here : ProgrammeStatus
  physical-test : ProgrammeStatus
```

The programme record is a table of accountability. Each field names one link in the chain and immediately records whether that link is imported, proved here, or left as physical comparison. This keeps the empirical mass claim visible without letting it masquerade as a theorem.

```
-- $ DistinctionToParticleMassProgramme.
record DistinctionToParticleMassProgramme : Set where
  field
    distinction-threshold      : ProgrammeStatus
    binary-normal-form         : ProgrammeStatus
    k4-closure                  : ProgrammeStatus
    invariant-ledger           : ProgrammeStatus
    number-tower                : ProgrammeStatus
    spectral-operator-layer    : ProgrammeStatus
    sector-assignment          : ProgrammeStatus
    neutral-depth-classification : ProgrammeStatus
    electron-anchored-ratios   : ProgrammeStatus
    planck-suppression-classification : ProgrammeStatus
    running-correction-classification : ProgrammeStatus
    correction-tower-interface : ProgrammeStatus
    tower-action-classification : ProgrammeStatus
    planck-low-energy-bridge    : ProgrammeStatus
    charged-sector-corrections : ProgrammeStatus
    charged-sector-residual-corrections : ProgrammeStatus
    neutrino-observable-ratios : ProgrammeStatus
    neutrino-residual-corrections : ProgrammeStatus
    absolute-particle-masses   : ProgrammeStatus
    experimental-comparison     : ProgrammeStatus
    distinction-stage-checked   : distinction-threshold ≡ checked-existing
    binary-stage-checked        : binary-normal-form ≡ checked-existing
    k4-stage-checked            : k4-closure ≡ checked-existing
    invariant-stage-checked     : invariant-ledger ≡ checked-existing
    number-stage-checked        : number-tower ≡ checked-existing
    spectral-stage-checked      : spectral-operator-layer ≡ checked-existing
    sector-stage-physical       : sector-assignment ≡ physical-test
    neutral-stage-checked-here  : neutral-depth-classification ≡ checked-here
```

electron-stage-checked-here : electron-anchored-ratios = checked-here
 planck-suppression-checked-here :
 planck-suppression-classification = checked-here
 running-correction-checked-here :
 running-correction-classification = checked-here
 correction-tower-checked-here : correction-tower-interface = checked-here
 tower-action-checked-here : tower-action-classification = checked-here
 planck-bridge-checked-here : planck-low-energy-bridge = checked-here
 charged-corrections-checked-here :
 charged-sector-corrections = checked-here
 charged-residuals-physical :
 charged-sector-residual-corrections = physical-test
 charged-residual-kind-classification : ProgrammeStatus
 charged-residual-kind-checked-here :
 charged-residual-kind-classification = checked-here
 neutrino-observables-checked-here :
 neutrino-observable-ratios = checked-here
 neutrino-residuals-checked-here :
 neutrino-residual-corrections = checked-here
 neutrino-residual-kind-classification : ProgrammeStatus
 neutrino-residual-kind-checked-here :
 neutrino-residual-kind-classification = checked-here
 carrier-closure-required-input-classification : ProgrammeStatus
 carrier-closure-required-input-checked-here :
 carrier-closure-required-input-classification = checked-here
 ledger-extension-for-carrier : ProgrammeStatus
 ledger-extension-checked-here :
 ledger-extension-for-carrier = checked-here
 sign-side-assignment-status : ProgrammeStatus
 generation-matching-assignment-status : ProgrammeStatus
 sign-side-assignment-physical :
 sign-side-assignment-status = physical-test
 generation-matching-assignment-physical :
 generation-matching-assignment-status = physical-test
 cell-structure-classification : ProgrammeStatus
 cell-structure-checked-here :
 cell-structure-classification = checked-here
 hadronic-extra-generations : ProgrammeStatus
 hadronic-extra-generations-physical :
 hadronic-extra-generations = physical-test
 neutral-splitting-redundancy : ProgrammeStatus
 neutral-splitting-redundancy-checked-here :
 neutral-splitting-redundancy = checked-here
 charge-duality-classification : ProgrammeStatus
 charge-duality-checked-here :
 charge-duality-classification = checked-here
 elementary-slot-count-status : ProgrammeStatus
 elementary-slot-count-checked-here :
 elementary-slot-count-status = checked-here
 slot-content-assignment-status : ProgrammeStatus
 slot-content-assignment-physical :
 slot-content-assignment-status = physical-test
 slot-fill-completeness-status : ProgrammeStatus
 slot-fill-completeness-checked-here :
 slot-fill-completeness-status = checked-here
 spectral-signature-status : ProgrammeStatus
 spectral-signature-checked-here :
 spectral-signature-status = checked-here
 matching-spectral-anchor-status : ProgrammeStatus
 matching-spectral-anchor-checked-here :
 matching-spectral-anchor-status = checked-here
 duality-spectral-anchor-status : ProgrammeStatus
 duality-spectral-anchor-checked-here :
 duality-spectral-anchor-status = checked-here
 twelve-spectral-factorisation-status : ProgrammeStatus
 twelve-spectral-factorisation-checked-here :
 twelve-spectral-factorisation-status = checked-here
 one-and-three-physical-status : ProgrammeStatus
 one-and-three-physical-locked :
 one-and-three-physical-status = physical-test
 formal-layer-closure-status : ProgrammeStatus
 formal-layer-closure-checked-here :

```

formal-layer-closure-status = checked-here
neutral-pair-separator-status : ProgrammeStatus
neutral-pair-separator-checked-here :
  neutral-pair-separator-status = checked-here
neutral-pair-uniformity-status : ProgrammeStatus
neutral-pair-uniformity-checked-here :
  neutral-pair-uniformity-status = checked-here
matching-weight-aut-invariance-status : ProgrammeStatus
matching-weight-aut-invariance-checked-here :
  matching-weight-aut-invariance-status = checked-here
intrinsic-neutral-pair-lock-status : ProgrammeStatus
intrinsic-neutral-pair-lock-checked-here :
  intrinsic-neutral-pair-lock-status = checked-here
matching-weight-aut-classification-status : ProgrammeStatus
matching-weight-aut-classification-checked-here :
  matching-weight-aut-classification-status = checked-here
vertex-aut-matching-classification-status : ProgrammeStatus
vertex-aut-matching-classification-checked-here :
  vertex-aut-matching-classification-status = checked-here
k4-graph-automorphism-status : ProgrammeStatus
k4-graph-automorphism-checked-here :
  k4-graph-automorphism-status = checked-here
neutral-pair-separator-essentiality-status : ProgrammeStatus
neutral-pair-separator-essentiality-checked-here :
  neutral-pair-separator-essentiality-status = checked-here
neutral-residual-carrier-status : ProgrammeStatus
neutral-residual-carrier-checked-here :
  neutral-residual-carrier-status = checked-here
neutral-residual-action-status : ProgrammeStatus
neutral-residual-action-checked-here :
  neutral-residual-action-status = checked-here
neutral-residual-physical-reading-status : ProgrammeStatus
neutral-residual-physical-reading-test :
  neutral-residual-physical-reading-status = physical-test
absolute-masses-physical : absolute-particle-masses = physical-test
experiment-is-test : experimental-comparison = physical-test

```

The canonical programme instance fills the ledger. Every proof field is ‘refl’, so the object is inspectable as a closed table: no later prose can quietly upgrade a physical-test field into a checked theorem.

```

-- $ distinction-to-particle-mass-programme.
distinction-to-particle-mass-programme : DistinctionToParticleMassProgramme
distinction-to-particle-mass-programme = record
{ distinction-threshold      = checked-existing
; binary-normal-form        = checked-existing
; k4-closure                 = checked-existing
; invariant-ledger           = checked-existing
; number-tower               = checked-existing
; spectral-operator-layer    = checked-existing
; sector-assignment          = physical-test
; neutral-depth-classification = checked-here
; electron-anchored-ratios   = checked-here
; planck-suppression-classification = checked-here
; running-correction-classification = checked-here
; correction-tower-interface = checked-here
; tower-action-classification = checked-here
; planck-low-energy-bridge    = checked-here
; charged-sector-corrections = checked-here
; charged-sector-residual-corrections = physical-test
; neutrino-observable-ratios = checked-here
; neutrino-residual-corrections = checked-here
; absolute-particle-masses    = physical-test
; experimental-comparison     = physical-test
; distinction-stage-checked    = refl
; binary-stage-checked        = refl
; k4-stage-checked            = refl
; invariant-stage-checked     = refl
; number-stage-checked        = refl
; spectral-stage-checked      = refl

```

```

; sector-stage-physical          = refl
; neutral-stage-checked-here     = refl
; electron-stage-checked-here    = refl
; planck-suppression-checked-here = refl
; running-correction-checked-here = refl
; correction-tower-checked-here  = refl
; tower-action-checked-here      = refl
; planck-bridge-checked-here     = refl
; charged-corrections-checked-here = refl
; charged-residuals-physical     = refl
; charged-residual-kind-classification = checked-here
; charged-residual-kind-checked-here = refl
; neutrino-observables-checked-here = refl
; neutrino-residuals-checked-here = refl
; neutrino-residual-kind-classification = checked-here
; neutrino-residual-kind-checked-here = refl
; carrier-closure-required-input-classification = checked-here
; carrier-closure-required-input-checked-here = refl
; ledger-extension-for-carrier = checked-here
; ledger-extension-checked-here = refl
; sign-side-assignment-status = physical-test
; generation-matching-assignment-status = physical-test
; sign-side-assignment-physical = refl
; generation-matching-assignment-physical = refl
; cell-structure-classification = checked-here
; cell-structure-checked-here = refl
; hadronic-extra-generations = physical-test
; hadronic-extra-generations-physical = refl
; neutral-splitting-redundancy = checked-here
; neutral-splitting-redundancy-checked-here = refl
; charge-duality-classification = checked-here
; charge-duality-checked-here = refl
; elementary-slot-count-status = checked-here
; elementary-slot-count-checked-here = refl
; slot-content-assignment-status = physical-test
; slot-content-assignment-physical = refl
; slot-fill-completeness-status = checked-here
; slot-fill-completeness-checked-here = refl
; spectral-signature-status = checked-here
; spectral-signature-checked-here = refl
; matching-spectral-anchor-status = checked-here
; matching-spectral-anchor-checked-here = refl
; duality-spectral-anchor-status = checked-here
; duality-spectral-anchor-checked-here = refl
; twelve-spectral-factorisation-status = checked-here
; twelve-spectral-factorisation-checked-here = refl
; one-and-three-physical-status = physical-test
; one-and-three-physical-locked = refl
; formal-layer-closure-status = checked-here
; formal-layer-closure-checked-here = refl
; neutral-pair-separator-status = checked-here
; neutral-pair-separator-checked-here = refl
; neutral-pair-uniformity-status = checked-here
; neutral-pair-uniformity-checked-here = refl
; matching-weight-aut-invariance-status = checked-here
; matching-weight-aut-invariance-checked-here = refl
; intrinsic-neutral-pair-lock-status = checked-here
; intrinsic-neutral-pair-lock-checked-here = refl
; matching-weight-aut-classification-status = checked-here
; matching-weight-aut-classification-checked-here = refl
; vertex-aut-matching-classification-status = checked-here
; vertex-aut-matching-classification-checked-here = refl
; k4-graph-automorphism-status = checked-here
; k4-graph-automorphism-checked-here = refl
; neutral-pair-separator-essentiality-status = checked-here
; neutral-pair-separator-essentiality-checked-here = refl
; neutral-residual-carrier-status = checked-here
; neutral-residual-carrier-checked-here = refl
; neutral-residual-action-status = checked-here
; neutral-residual-action-checked-here = refl
; neutral-residual-physical-reading-status = physical-test
; neutral-residual-physical-reading-test = refl

```

```

; absolute-masses-physical = refl
; experiment-is-test      = refl
}

```

2 The K_4 ledger used by the schema

The record below is deliberately small. It names only the invariant slots needed by the neutral-depth and Planck-suppression filters: vertices, edges, degree, Euler boundary, the boundary unit, the Fermat-prime slot, the discrete κ scale, the closure integer, the universal loop numerator, and the two loop volumes already used elsewhere in the project. These are formal ledger entries, not physical names.

```

-- $ K4Ledger.
record K4Ledger : Set where
field
  vertices      : ℕ
  edges         : ℕ
  degree        : ℕ
  euler         : ℕ
  boundaryUnit  : ℕ
  fermatPrime   : ℕ
  kappaScale    : ℕ
  closureInteger : ℕ
  rgDenominator : ℕ
  loopNumerator : ℕ
  bulkVolume    : ℕ
  extendedVolume : ℕ

canonicalK4Ledger : K4Ledger
canonicalK4Ledger = record
{ vertices      = 4
; edges         = 6
; degree        = 3
; euler         = 2
; boundaryUnit  = 4 ÷ 3
; fermatPrime   = 17
; kappaScale    = 4 * 2
; closureInteger = (4 * 4 * 4) * 2 + (3 * 3)
; rgDenominator = 2 * ((4 * 4 * 4) * 2 + (3 * 3))
; loopNumerator = 6 + 3 + 2
; bulkVolume    = 4 * 3 * 6
; extendedVolume = 8 * (4 * 3 * 6)
}

open K4Ledger

ledger-boundary-unit : boundaryUnit canonicalK4Ledger ≡ 1
ledger-boundary-unit = refl

ledger-fermat-prime : fermatPrime canonicalK4Ledger ≡ 17
ledger-fermat-prime = refl

ledger-kappa-scale : kappaScale canonicalK4Ledger ≡ 8
ledger-kappa-scale = refl

ledger-closure-integer : closureInteger canonicalK4Ledger ≡ 137
ledger-closure-integer = refl

ledger-rg-denominator : rgDenominator canonicalK4Ledger ≡ 274
ledger-rg-denominator = refl

ledger-loop-numerator : loopNumerator canonicalK4Ledger ≡ 11
ledger-loop-numerator = refl

ledger-bulk-volume : bulkVolume canonicalK4Ledger ≡ 72
ledger-bulk-volume = refl

ledger-extended-volume : extendedVolume canonicalK4Ledger ≡ 576
ledger-extended-volume = refl

```

3 Finite neutral-depth candidates

The schema does not start with a mass value. It starts with four local depth candidates already visible in the ledger:

- the generation depth, $d = 3$;
- the closure-carrier depth, $V = 4$;
- the neutral depth, $d + \chi = 5$;
- the edge-sector depth, $E = 6$.

The neutral slot is asked to lie beyond the generation count, not to be identical with the closure carrier itself, and not to be the full edge sector. Those three filters are the formal admission data of this schema. They are intentionally weaker than a physical claim: they only say which candidate survives the displayed classification.

```
-- $ DepthCandidate.
data DepthCandidate : Set where
  generation-depth : DepthCandidate
  closure-depth    : DepthCandidate
  neutral-depth    : DepthCandidate
  edge-depth       : DepthCandidate

depth-value : DepthCandidate → ℕ
depth-value generation-depth = degree canonicalK4Ledge
depth-value closure-depth    = vertices canonicalK4Ledge
depth-value neutral-depth    = degree canonicalK4Ledge + euler canonicalK4Ledge
depth-value edge-depth       = edges canonicalK4Ledge

generation-depth-is-three : depth-value generation-depth ≡ 3
generation-depth-is-three = refl

closure-depth-is-four : depth-value closure-depth ≡ 4
closure-depth-is-four = refl

neutral-depth-is-five : depth-value neutral-depth ≡ 5
neutral-depth-is-five = refl

edge-depth-is-six : depth-value edge-depth ≡ 6
edge-depth-is-six = refl
```

4 The neutral-depth filter

The elimination is finite. The generation slot fails because it has not moved beyond the charged generation count. The closure slot fails because it is the already-used four-case carrier. The edge slot fails because it belongs to the full edge sector. Only the neutral depth survives all three filters.

```
-- $ BeyondGeneration.
BeyondGeneration : DepthCandidate → Set
BeyondGeneration generation-depth = ⊥
BeyondGeneration closure-depth    = ⊤
BeyondGeneration neutral-depth    = ⊤
BeyondGeneration edge-depth       = ⊤

NotClosureCarrier : DepthCandidate → Set
NotClosureCarrier generation-depth = ⊤
NotClosureCarrier closure-depth    = ⊥
NotClosureCarrier neutral-depth    = ⊤
```



```

NotClosureCarrier edge-depth =  $\top$ 

BeforeEdgeSector : DepthCandidate → Set
BeforeEdgeSector generation-depth =  $\top$ 
BeforeEdgeSector closure-depth =  $\top$ 
BeforeEdgeSector neutral-depth =  $\top$ 
BeforeEdgeSector edge-depth =  $\perp$ 

NeutralDepthAdmissible : DepthCandidate → Set
NeutralDepthAdmissible candidate =
  BeyondGeneration candidate ×
  (NotClosureCarrier candidate × BeforeEdgeSector candidate)

neutral-depth-admissible : NeutralDepthAdmissible neutral-depth
neutral-depth-admissible = tt, (tt, tt)

admissible-depth-unique :
  (candidate : DepthCandidate) →
  NeutralDepthAdmissible candidate →
  candidate ≡ neutral-depth
admissible-depth-unique generation-depth ((), _)
admissible-depth-unique closure-depth (_, ((), _))
admissible-depth-unique neutral-depth _ = refl
admissible-depth-unique edge-depth (_, (_, ()))

```

5 Complete schema interface

The complete schema packages the survivor, the fact that it survives, its value, and its uniqueness against every other candidate in the finite window. The depth square is the formal output of this section; the later Planck and residual layers use it as input rather than reopening the neutral-depth classification.

```

-- $ NeutralCompleteSchema.
record NeutralCompleteSchema : Set where
  field
    survivor          : DepthCandidate
    survivor-admissible : NeutralDepthAdmissible survivor
    survivor-value     : depth-value survivor ≡ 5
    survivor-unique    :
      (candidate : DepthCandidate) →
      NeutralDepthAdmissible candidate →
      candidate ≡ survivor

neutral-complete-schema : NeutralCompleteSchema
neutral-complete-schema = record
{ survivor          = neutral-depth
; survivor-admissible = neutral-depth-admissible
; survivor-value     = refl
; survivor-unique    = admissible-depth-unique
}

neutral-depth-value :  $\mathbb{N}$ 
neutral-depth-value =
  depth-value (NeutralCompleteSchema.survivor neutral-complete-schema)

neutral-depth-value-is-five : neutral-depth-value ≡ 5
neutral-depth-value-is-five = refl

neutral-depth-square :  $\mathbb{N}$ 
neutral-depth-square = neutral-depth-value * neutral-depth-value

neutral-depth-square-is-25 : neutral-depth-square ≡ 25
neutral-depth-square-is-25 = refl

record NeutralMassInterface : Set where

```

```

field
  complete-schema : NeutralCompleteSchema
  depth-denominator : ℕ
  denominator-locked : depth-denominator = 25

neutral-mass-interface : NeutralMassInterface
neutral-mass-interface = record
{ complete-schema = neutral-complete-schema
; depth-denominator = neutral-depth-square
; denominator-locked = refl
}

```

6 Planck-to-low-energy bridge classification

The bridge problem can now be stated without drama. The Planck anchor is available as the natural ultraviolet unit, and the neutral denominator is available from the finite schema. The first predicate records why the uncorrected routes do not suffice: direct Planck division stays in the ultraviolet sector; the Higgs route names an intermediate scale without deriving the running; the bare electron route carries only the structured suppression denominator; and the electron-only route uses a low-energy physical anchor.

The corrected predicate then adds the missing structure. It requires the Planck-suppression survivor, the running survivor, the complete correction-tower interface, the canonical tower action, and the absence of an empirical patch. In this finite candidate space, only the sign-ordered tower reading survives. This is the formal bridge used in the mass-chain section below.

```

-- $ ScaleAnchor.
data ScaleAnchor : Set where
  planck-scale : ScaleAnchor
  higgs-scale : ScaleAnchor
  electron-scale : ScaleAnchor
  neutrino-scale : ScaleAnchor

data BridgeLayer : Set where
  initial-search : BridgeLayer
  structured-candidate : BridgeLayer
  closed-formal-bridge : BridgeLayer
  physical-input-stage : BridgeLayer

data ScaleBridgeCandidate : Set where
  direct-planck-neutral : ScaleBridgeCandidate
  planck-higgs-neutral : ScaleBridgeCandidate
  planck-electron-neutral : ScaleBridgeCandidate
  electron-neutral-only : ScaleBridgeCandidate

candidate-start : ScaleBridgeCandidate → ScaleAnchor
candidate-start direct-planck-neutral = planck-scale
candidate-start planck-higgs-neutral = planck-scale
candidate-start planck-electron-neutral = planck-scale
candidate-start electron-neutral-only = electron-scale

candidate-end : ScaleBridgeCandidate → ScaleAnchor
candidate-end direct-planck-neutral = neutrino-scale
candidate-end planck-higgs-neutral = higgs-scale
candidate-end planck-electron-neutral = electron-scale
candidate-end electron-neutral-only = neutrino-scale

PlanckOrigin : ScaleBridgeCandidate → Set
PlanckOrigin direct-planck-neutral = ⊤
PlanckOrigin planck-higgs-neutral = ⊤
PlanckOrigin planck-electron-neutral = ⊤
PlanckOrigin electron-neutral-only = ⊥

```

```

NotDirectNeutrinoShortcut : ScaleBridgeCandidate → Set
NotDirectNeutrinoShortcut direct-planck-neutral = ⊥
NotDirectNeutrinoShortcut planck-higgs-neutral = ⊤
NotDirectNeutrinoShortcut planck-electron-neutral = ⊤
NotDirectNeutrinoShortcut electron-neutral-only = ⊤

NotHiggsPlaceholder : ScaleBridgeCandidate → Set
NotHiggsPlaceholder direct-planck-neutral = ⊤
NotHiggsPlaceholder planck-higgs-neutral = ⊥
NotHiggsPlaceholder planck-electron-neutral = ⊤
NotHiggsPlaceholder electron-neutral-only = ⊤

ElectronAnchorTarget : ScaleBridgeCandidate → Set
ElectronAnchorTarget direct-planck-neutral = ⊥
ElectronAnchorTarget planck-higgs-neutral = ⊥
ElectronAnchorTarget planck-electron-neutral = ⊤
ElectronAnchorTarget electron-neutral-only = ⊥

StructuredBridgeRoute : ScaleBridgeCandidate → Set
StructuredBridgeRoute candidate =
  PlanckOrigin candidate ×
  (NotDirectNeutrinoShortcut candidate ×
   (NotHiggsPlaceholder candidate × ElectronAnchorTarget candidate))

structured-bridge-route : StructuredBridgeRoute planck-electron-neutral
structured-bridge-route = tt , (tt , (tt , tt))

structured-bridge-route-unique :
  (candidate : ScaleBridgeCandidate) →
  StructuredBridgeRoute candidate →
  candidate ≡ planck-electron-neutral
structured-bridge-route-unique direct-planck-neutral ( , (( , _))
structured-bridge-route-unique planck-higgs-neutral ( , ( , (( , _)))
structured-bridge-route-unique planck-electron-neutral _ = refl
structured-bridge-route-unique electron-neutral-only (( , _))

```

The route fixes only the scale path. The suppression still needs exponents. The first exponent is the κ slot: among the boundary, Euler, degree, and carrier candidates, only the Euler exponent survives.

```

-- $ KappaExponentCandidate.
data KappaExponentCandidate : Set where
  kappa-boundary-slot : KappaExponentCandidate
  kappa-euler-slot : KappaExponentCandidate
  kappa-degree-slot : KappaExponentCandidate
  kappa-carrier-slot : KappaExponentCandidate

kappa-exponent-value : KappaExponentCandidate → ℕ
kappa-exponent-value kappa-boundary-slot = boundaryUnit canonicalK4Ledge
kappa-exponent-value kappa-euler-slot = euler canonicalK4Ledge
kappa-exponent-value kappa-degree-slot = degree canonicalK4Ledge
kappa-exponent-value kappa-carrier-slot = vertices canonicalK4Ledge

KappaExponentAdmissible : KappaExponentCandidate → Set
KappaExponentAdmissible kappa-boundary-slot = ⊥
KappaExponentAdmissible kappa-euler-slot = ⊤
KappaExponentAdmissible kappa-degree-slot = ⊥
KappaExponentAdmissible kappa-carrier-slot = ⊥

kappa-exponent-admissible : KappaExponentAdmissible kappa-euler-slot
kappa-exponent-admissible = tt

kappa-exponent-unique :
  (candidate : KappaExponentCandidate) →
  KappaExponentAdmissible candidate →
  candidate ≡ kappa-euler-slot
kappa-exponent-unique kappa-boundary-slot ()
kappa-exponent-unique kappa-euler-slot _ = refl

```

```
kappa-exponent-unique kappa-degree-slot ()
kappa-exponent-unique kappa-carrier-slot ()
```

The Fermat part uses the carrier count. This selects 17^4 from the finite slot list and rules out Euler, degree, and neutral-depth reuse.

```
-- $ FermatExponentCandidate.
data FermatExponentCandidate : Set where
  fermat-euler-slot   : FermatExponentCandidate
  fermat-degree-slot  : FermatExponentCandidate
  fermat-carrier-slot : FermatExponentCandidate
  fermat-neutral-slot : FermatExponentCandidate

fermat-exponent-value : FermatExponentCandidate → ℕ
fermat-exponent-value fermat-euler-slot   = euler canonicalK4Ledge
fermat-exponent-value fermat-degree-slot  = degree canonicalK4Ledge
fermat-exponent-value fermat-carrier-slot = vertices canonicalK4Ledge
fermat-exponent-value fermat-neutral-slot = neutral-depth-value

FermatExponentAdmissible : FermatExponentCandidate → Set
FermatExponentAdmissible fermat-euler-slot   = ⊥
FermatExponentAdmissible fermat-degree-slot  = ⊥
FermatExponentAdmissible fermat-carrier-slot = ⊤
FermatExponentAdmissible fermat-neutral-slot = ⊥

fermat-exponent-admissible :
  FermatExponentAdmissible fermat-carrier-slot
fermat-exponent-admissible = tt

fermat-exponent-unique :
  (candidate : FermatExponentCandidate) →
  FermatExponentAdmissible candidate →
  candidate ≡ fermat-carrier-slot
fermat-exponent-unique fermat-euler-slot ()
fermat-exponent-unique fermat-degree-slot ()
fermat-exponent-unique fermat-carrier-slot _ = refl
fermat-exponent-unique fermat-neutral-slot ()
```

The closure exponent must contain both carrier and generation information while avoiding the neutral and edge-sector slots. The surviving exponent is therefore $4 + 3 = 7$.

```
-- $ ClosureExponentCandidate.
data ClosureExponentCandidate : Set where
  closure-degree-slot : ClosureExponentCandidate
  closure-carrier-slot : ClosureExponentCandidate
  closure-neutral-slot : ClosureExponentCandidate
  closure-edge-slot   : ClosureExponentCandidate
  closure-carrier-generation-slot : ClosureExponentCandidate

closure-exponent-value : ClosureExponentCandidate → ℕ
closure-exponent-value closure-degree-slot = degree canonicalK4Ledge
closure-exponent-value closure-carrier-slot = vertices canonicalK4Ledge
closure-exponent-value closure-neutral-slot = neutral-depth-value
closure-exponent-value closure-edge-slot   = edges canonicalK4Ledge
closure-exponent-value closure-carrier-generation-slot =
  vertices canonicalK4Ledge + degree canonicalK4Ledge

ContainsCarrier : ClosureExponentCandidate → Set
ContainsCarrier closure-degree-slot = ⊥
ContainsCarrier closure-carrier-slot = ⊤
ContainsCarrier closure-neutral-slot = ⊥
ContainsCarrier closure-edge-slot   = ⊥
ContainsCarrier closure-carrier-generation-slot = ⊤

ContainsGeneration : ClosureExponentCandidate → Set
ContainsGeneration closure-degree-slot = ⊤
ContainsGeneration closure-carrier-slot = ⊥
ContainsGeneration closure-neutral-slot = ⊤
```

```

ContainsGeneration closure-edge-slot = ⊥
ContainsGeneration closure-carrier-generation-slot = ⊤

NotNeutralReuse : ClosureExponentCandidate → Set
NotNeutralReuse closure-degree-slot = ⊤
NotNeutralReuse closure-carrier-slot = ⊤
NotNeutralReuse closure-neutral-slot = ⊥
NotNeutralReuse closure-edge-slot = ⊤
NotNeutralReuse closure-carrier-generation-slot = ⊤

NotEdgeSectorReuse : ClosureExponentCandidate → Set
NotEdgeSectorReuse closure-degree-slot = ⊤
NotEdgeSectorReuse closure-carrier-slot = ⊤
NotEdgeSectorReuse closure-neutral-slot = ⊤
NotEdgeSectorReuse closure-edge-slot = ⊥
NotEdgeSectorReuse closure-carrier-generation-slot = ⊤

ClosureExponentAdmissible : ClosureExponentCandidate → Set
ClosureExponentAdmissible candidate =
  ContainsCarrier candidate ×
  (ContainsGeneration candidate ×
   (NotNeutralReuse candidate × NotEdgeSectorReuse candidate))

closure-exponent-admissible :
  ClosureExponentAdmissible closure-carrier-generation-slot
closure-exponent-admissible = tt , (tt , (tt , tt))

closure-exponent-unique :
  (candidate : ClosureExponentCandidate) →
  ClosureExponentAdmissible candidate →
  candidate ≡ closure-carrier-generation-slot
closure-exponent-unique closure-degree-slot ((), _)
closure-exponent-unique closure-carrier-slot (_, ((), _))
closure-exponent-unique closure-neutral-slot ((), _)
closure-exponent-unique closure-edge-slot ((), _)
closure-exponent-unique closure-carrier-generation-slot _ = refl

```

The three exponent filters are then packaged as one suppression schema. This record is the formal reason the denominator below is a product of the neutral slot, 8^2 , 17^4 , and 137^7 .

```

-- $ CanonicalPlanckSuppressionSchema.
record CanonicalPlanckSuppressionSchema : Set where
  field
    route-candidate      : ScaleBridgeCandidate
    route-admissible     : StructuredBridgeRoute route-candidate
    route-unique         :
      (candidate : ScaleBridgeCandidate) →
      StructuredBridgeRoute candidate →
      candidate ≡ route-candidate
    neutral-classification : NeutralCompleteSchema
    kappa-base-locked    : kappaScale canonicalK4Ledger ≡ 8
    fermat-base-locked   : fermatPrime canonicalK4Ledger ≡ 17
    closure-base-locked  : closureInteger canonicalK4Ledger ≡ 137
    kappa-exponent       : KappaExponentCandidate
    kappa-admissible     : KappaExponentAdmissible kappa-exponent
    kappa-value-locked   : kappa-exponent-value kappa-exponent ≡ 2
    kappa-unique         :
      (candidate : KappaExponentCandidate) →
      KappaExponentAdmissible candidate →
      candidate ≡ kappa-exponent
    fermat-exponent      : FermatExponentCandidate
    fermat-admissible    : FermatExponentAdmissible fermat-exponent
    fermat-value-locked  : fermat-exponent-value fermat-exponent ≡ 4
    fermat-unique       :
      (candidate : FermatExponentCandidate) →
      FermatExponentAdmissible candidate →
      candidate ≡ fermat-exponent
    closure-exponent     : ClosureExponentCandidate
    closure-admissible   : ClosureExponentAdmissible closure-exponent
    closure-value-locked : closure-exponent-value closure-exponent ≡ 7

```

```

closure-unique      :
(candidate : ClosureExponentCandidate) →
ClosureExponentAdmissible candidate →
candidate ≡ closure-exponent

canonical-planck-suppression-schema : CanonicalPlanckSuppressionSchema
canonical-planck-suppression-schema = record
{ route-candidate      = planck-electron-neutral
; route-admissible     = structured-bridge-route
; route-unique         = structured-bridge-route-unique
; neutral-classification = neutral-complete-schema
; kappa-base-locked    = refl
; fermat-base-locked   = refl
; closure-base-locked  = refl
; kappa-exponent       = kappa-euler-slot
; kappa-admissible     = kappa-exponent-admissible
; kappa-value-locked   = refl
; kappa-unique         = kappa-exponent-unique
; fermat-exponent      = fermat-carrier-slot
; fermat-admissible    = fermat-exponent-admissible
; fermat-value-locked  = refl
; fermat-unique        = fermat-exponent-unique
; closure-exponent     = closure-carrier-generation-slot
; closure-admissible   = closure-exponent-admissible
; closure-value-locked = refl
; closure-unique       = closure-exponent-unique
}

```

The next short block reads out the locked data as explicit integers. It is included because a reader should see the denominator before any decimal mass comparison appears.

```

-- $ canonical-neutral-survivor.
canonical-neutral-survivor : DepthCandidate
canonical-neutral-survivor =
NeutralCompleteSchema.survivor
(CanonicalPlanckSuppressionSchema.neutral-classification
canonical-planck-suppression-schema)

canonical-neutral-survivor-is-neutral :
canonical-neutral-survivor ≡ neutral-depth
canonical-neutral-survivor-is-neutral = refl

canonical-bridge-route : ScaleBridgeCandidate
canonical-bridge-route =
CanonicalPlanckSuppressionSchema.route-candidate
canonical-planck-suppression-schema

canonical-bridge-route-is-planck-electron :
canonical-bridge-route ≡ planck-electron-neutral
canonical-bridge-route-is-planck-electron = refl

canonical-kappa-exponent : KappaExponentCandidate
canonical-kappa-exponent =
CanonicalPlanckSuppressionSchema.kappa-exponent
canonical-planck-suppression-schema

canonical-kappa-exponent-is-euler :
canonical-kappa-exponent ≡ kappa-euler-slot
canonical-kappa-exponent-is-euler = refl

canonical-fermat-exponent : FermatExponentCandidate
canonical-fermat-exponent =
CanonicalPlanckSuppressionSchema.fermat-exponent
canonical-planck-suppression-schema

canonical-fermat-exponent-is-carrier :
canonical-fermat-exponent ≡ fermat-carrier-slot
canonical-fermat-exponent-is-carrier = refl

```

canonical-closure-exponent : ClosureExponentCandidate
 canonical-closure-exponent =
 CanonicalPlanckSuppressionSchema.closure-exponent
 canonical-planck-suppression-schema

canonical-closure-exponent-is-carrier-generation :
 canonical-closure-exponent $\equiv$ closure-carrier-generation-slot
 canonical-closure-exponent-is-carrier-generation = refl

bridge-neutral-slot : $\mathbb{N}$
 bridge-neutral-slot = depth-value canonical-neutral-survivor

bridge-neutral-slot-is-5 : bridge-neutral-slot $\equiv$ 5
 bridge-neutral-slot-is-5 = refl

bridge-kappa-slot-count : $\mathbb{N}$
 bridge-kappa-slot-count = kappa-exponent-value canonical-kappa-exponent

bridge-kappa-slot-count-is-2 : bridge-kappa-slot-count $\equiv$ 2
 bridge-kappa-slot-count-is-2 = refl

bridge-fermat-slot-count : $\mathbb{N}$
 bridge-fermat-slot-count = fermat-exponent-value canonical-fermat-exponent

bridge-fermat-slot-count-is-4 : bridge-fermat-slot-count $\equiv$ 4
 bridge-fermat-slot-count-is-4 = refl

bridge-closure-slot-count : $\mathbb{N}$
 bridge-closure-slot-count =
 closure-exponent-value canonical-closure-exponent

bridge-closure-slot-count-is-7 : bridge-closure-slot-count $\equiv$ 7
 bridge-closure-slot-count-is-7 = refl

bridge-kappa : $\mathbb{N}$
 bridge-kappa = kappaScale canonicalK4Ledger

bridge-kappa-square : $\mathbb{N}$
 bridge-kappa-square = bridge-kappa ^ bridge-kappa-slot-count

bridge-kappa-square-is-64 : bridge-kappa-square $\equiv$ 64
 bridge-kappa-square-is-64 = refl

bridge-fermat-fourth : $\mathbb{N}$
 bridge-fermat-fourth =
 fermatPrime canonicalK4Ledger ^ bridge-fermat-slot-count

bridge-fermat-fourth-is-83521 : bridge-fermat-fourth $\equiv$ 83521
 bridge-fermat-fourth-is-83521 = refl

bridge-closure-constant : $\mathbb{N}$
 bridge-closure-constant = closureInteger canonicalK4Ledger

bridge-closure-seventh : $\mathbb{N}$
 bridge-closure-seventh =
 bridge-closure-constant ^ bridge-closure-slot-count

bridge-closure-seventh-is-905824306333433 :
 bridge-closure-seventh $\equiv$ 905824306333433
 bridge-closure-seventh-is-905824306333433 = refl

planck-electron-suppression-denominator : $\mathbb{N}$
 planck-electron-suppression-denominator =
 bridge-neutral-slot * bridge-kappa-square *
 bridge-fermat-fourth * bridge-closure-seventh

planck-electron-denominator-is-24209712604567890429760 :
 planck-electron-suppression-denominator $\equiv$ 24209712604567890429760
 planck-electron-denominator-is-24209712604567890429760 = refl

The uncorrected suppression is not yet the final bridge. The running filter requires the loop remainder, the RG scale, and the generation depth, while blocking volume substitution. This leaves the factor 833/822.

```
-- $ RunningCorrectionCandidate.
data RunningCorrectionCandidate : Set where
  no-running-correction : RunningCorrectionCandidate
  loop-only-correction  : RunningCorrectionCandidate
  rg-only-correction    : RunningCorrectionCandidate
  degree-rg-loop        : RunningCorrectionCandidate
  volume-loop-correction : RunningCorrectionCandidate

UsesLoopRemainder : RunningCorrectionCandidate → Set
UsesLoopRemainder no-running-correction = ⊥
UsesLoopRemainder loop-only-correction  = ⊤
UsesLoopRemainder rg-only-correction    = ⊥
UsesLoopRemainder degree-rg-loop        = ⊤
UsesLoopRemainder volume-loop-correction = ⊤

UsesRGScale : RunningCorrectionCandidate → Set
UsesRGScale no-running-correction = ⊥
UsesRGScale loop-only-correction  = ⊥
UsesRGScale rg-only-correction    = ⊤
UsesRGScale degree-rg-loop        = ⊤
UsesRGScale volume-loop-correction = ⊤

CarriesGenerationDepth : RunningCorrectionCandidate → Set
CarriesGenerationDepth no-running-correction = ⊥
CarriesGenerationDepth loop-only-correction  = ⊥
CarriesGenerationDepth rg-only-correction    = ⊥
CarriesGenerationDepth degree-rg-loop        = ⊤
CarriesGenerationDepth volume-loop-correction = ⊤

NotVolumeSubstitution : RunningCorrectionCandidate → Set
NotVolumeSubstitution no-running-correction = ⊤
NotVolumeSubstitution loop-only-correction  = ⊤
NotVolumeSubstitution rg-only-correction    = ⊤
NotVolumeSubstitution degree-rg-loop        = ⊤
NotVolumeSubstitution volume-loop-correction = ⊥

RunningCorrectionAdmissible : RunningCorrectionCandidate → Set
RunningCorrectionAdmissible candidate =
  UsesLoopRemainder candidate ×
  (UsesRGScale candidate ×
   (CarriesGenerationDepth candidate × NotVolumeSubstitution candidate))

running-correction-admissible :
  RunningCorrectionAdmissible degree-rg-loop
running-correction-admissible = tt , (tt , (tt , tt))

running-correction-unique :
  (candidate : RunningCorrectionCandidate) →
  RunningCorrectionAdmissible candidate →
  candidate ≡ degree-rg-loop
running-correction-unique no-running-correction ((), _)
running-correction-unique loop-only-correction (_, ((), _))
running-correction-unique rg-only-correction ((), _)
running-correction-unique degree-rg-loop _ = refl
running-correction-unique volume-loop-correction (_, (_, (_, ())))

running-correction-denominator : ℕ
running-correction-denominator =
  degree canonicalK4Ledge * rgDenominator canonicalK4Ledge

running-correction-denominator-is-822 :
  running-correction-denominator ≡ 822
running-correction-denominator-is-822 = refl

running-correction-numerator : ℕ
```



```

running-correction-numerator =
  running-correction-denominator + loopNumerator canonicalK4Ledger

running-correction-numerator-is-833 :
  running-correction-numerator ≡ 833
running-correction-numerator-is-833 = refl

record CanonicalRunningCorrectionSchema : Set where
  field
    correction-candidate      : RunningCorrectionCandidate
    correction-admissible     : RunningCorrectionAdmissible correction-candidate
    correction-unique         :
      (candidate : RunningCorrectionCandidate) →
      RunningCorrectionAdmissible candidate →
      candidate ≡ correction-candidate
    correction-denominator    : ℕ
    correction-numerator      : ℕ
    correction-denom-locked   : correction-denominator ≡ 822
    correction-num-locked     : correction-numerator ≡ 833

canonical-running-correction-schema : CanonicalRunningCorrectionSchema
canonical-running-correction-schema = record
{ correction-candidate      = degree-rg-loop
; correction-admissible     = running-correction-admissible
; correction-unique         = running-correction-unique
; correction-denominator    = running-correction-denominator
; correction-numerator      = running-correction-numerator
; correction-denom-locked   = refl
; correction-num-locked     = refl
}

```

The correction tower contributes four signed slots. The first two raise the bridge, the last two lower it; the point is not a fitted decimal but a fixed signed tower action.

```

-- $ CorrectionSign.
data CorrectionSign : Set where
  positive-correction : CorrectionSign
  negative-correction : CorrectionSign

data CorrectionTowerSlot : Set where
  boundary-correction : CorrectionTowerSlot
  mass-correction     : CorrectionTowerSlot
  degree-correction   : CorrectionTowerSlot
  coupling-correction : CorrectionTowerSlot

correction-tower-numerator : CorrectionTowerSlot → ℕ
correction-tower-numerator boundary-correction = euler canonicalK4Ledger
correction-tower-numerator mass-correction     = boundaryUnit canonicalK4Ledger
correction-tower-numerator degree-correction   =
  degree canonicalK4Ledger * degree canonicalK4Ledger
correction-tower-numerator coupling-correction =
  (loopNumerator canonicalK4Ledger + euler canonicalK4Ledger) ^ 2

correction-tower-denominator : CorrectionTowerSlot → ℕ
correction-tower-denominator boundary-correction =
  closureInteger canonicalK4Ledger ^ 2
correction-tower-denominator mass-correction =
  kappaScale canonicalK4Ledger * degree canonicalK4Ledger *
  (closureInteger canonicalK4Ledger ^ 2)
correction-tower-denominator degree-correction =
  kappaScale canonicalK4Ledger * (closureInteger canonicalK4Ledger ^ 2)
correction-tower-denominator coupling-correction =
  (kappaScale canonicalK4Ledger ^ 2) *
  (closureInteger canonicalK4Ledger ^ 2)

correction-tower-sign : CorrectionTowerSlot → CorrectionSign
correction-tower-sign boundary-correction = positive-correction
correction-tower-sign mass-correction     = positive-correction
correction-tower-sign degree-correction   = negative-correction
correction-tower-sign coupling-correction = negative-correction

```

```

correction-boundary-num-is-2 :
  correction-tower-numerator boundary-correction ≡ 2
correction-boundary-num-is-2 = refl

correction-mass-num-is-1 :
  correction-tower-numerator mass-correction ≡ 1
correction-mass-num-is-1 = refl

correction-degree-num-is-9 :
  correction-tower-numerator degree-correction ≡ 9
correction-degree-num-is-9 = refl

correction-coupling-num-is-169 :
  correction-tower-numerator coupling-correction ≡ 169
correction-coupling-num-is-169 = refl

correction-boundary-den-is-18769 :
  correction-tower-denominator boundary-correction ≡ 18769
correction-boundary-den-is-18769 = refl

correction-mass-den-is-450456 :
  correction-tower-denominator mass-correction ≡ 450456
correction-mass-den-is-450456 = refl

correction-degree-den-is-150152 :
  correction-tower-denominator degree-correction ≡ 150152
correction-degree-den-is-150152 = refl

correction-coupling-den-is-1201216 :
  correction-tower-denominator coupling-correction ≡ 1201216
correction-coupling-den-is-1201216 = refl

record CanonicalCorrectionTowerInterface : Set where
  field
    boundary-num-locked : correction-tower-numerator boundary-correction ≡ 2
    mass-num-locked      : correction-tower-numerator mass-correction ≡ 1
    degree-num-locked    : correction-tower-numerator degree-correction ≡ 9
    coupling-num-locked  : correction-tower-numerator coupling-correction ≡ 169
    boundary-den-locked  : correction-tower-denominator boundary-correction ≡ 18769
    mass-den-locked      : correction-tower-denominator mass-correction ≡ 450456
    degree-den-locked    : correction-tower-denominator degree-correction ≡ 150152
    coupling-den-locked  : correction-tower-denominator coupling-correction ≡ 1201216
    boundary-sign        : correction-tower-sign boundary-correction ≡ positive-correction
    mass-sign            : correction-tower-sign mass-correction ≡ positive-correction
    degree-sign          : correction-tower-sign degree-correction ≡ negative-correction
    coupling-sign        : correction-tower-sign coupling-correction ≡ negative-correction

canonical-correction-tower-interface : CanonicalCorrectionTowerInterface
canonical-correction-tower-interface = record
{ boundary-num-locked = refl
; mass-num-locked     = refl
; degree-num-locked   = refl
; coupling-num-locked = refl
; boundary-den-locked = refl
; mass-den-locked     = refl
; degree-den-locked   = refl
; coupling-den-locked = refl
; boundary-sign       = refl
; mass-sign           = refl
; degree-sign         = refl
; coupling-sign       = refl
}

```

The tower interface still leaves possible actions. The finite action filter requires both signs, preserves their order, and refuses sector naming as a choice. Only the sign-ordered full action survives.

```

-- $ TowerActionCandidate.
data TowerActionCandidate : Set where

```

```

no-tower-action      : TowerActionCandidate
positive-face-only-action : TowerActionCandidate
negative-face-only-action : TowerActionCandidate
unordered-full-action  : TowerActionCandidate
sign-ordered-full-action : TowerActionCandidate
sector-named-action    : TowerActionCandidate

UsesPositiveFace : TowerActionCandidate → Set
UsesPositiveFace no-tower-action      = ⊥
UsesPositiveFace positive-face-only-action = ⊤
UsesPositiveFace negative-face-only-action = ⊥
UsesPositiveFace unordered-full-action  = ⊤
UsesPositiveFace sign-ordered-full-action = ⊤
UsesPositiveFace sector-named-action    = ⊤

UsesNegativeFace : TowerActionCandidate → Set
UsesNegativeFace no-tower-action      = ⊥
UsesNegativeFace positive-face-only-action = ⊥
UsesNegativeFace negative-face-only-action = ⊤
UsesNegativeFace unordered-full-action  = ⊤
UsesNegativeFace sign-ordered-full-action = ⊤
UsesNegativeFace sector-named-action    = ⊤

RespectsForcedSignOrder : TowerActionCandidate → Set
RespectsForcedSignOrder no-tower-action      = ⊥
RespectsForcedSignOrder positive-face-only-action = ⊥
RespectsForcedSignOrder negative-face-only-action = ⊥
RespectsForcedSignOrder unordered-full-action  = ⊥
RespectsForcedSignOrder sign-ordered-full-action = ⊤
RespectsForcedSignOrder sector-named-action    = ⊥

NoSectorNameChoice : TowerActionCandidate → Set
NoSectorNameChoice no-tower-action      = ⊤
NoSectorNameChoice positive-face-only-action = ⊤
NoSectorNameChoice negative-face-only-action = ⊤
NoSectorNameChoice unordered-full-action  = ⊤
NoSectorNameChoice sign-ordered-full-action = ⊤
NoSectorNameChoice sector-named-action    = ⊥

TowerActionAdmissible : TowerActionCandidate → Set
TowerActionAdmissible candidate =
  UsesPositiveFace candidate ×
  (UsesNegativeFace candidate ×
  (RespectsForcedSignOrder candidate × NoSectorNameChoice candidate))

tower-action-admissible : TowerActionAdmissible sign-ordered-full-action
tower-action-admissible = tt , (tt , (tt , tt))

tower-action-unique :
  (candidate : TowerActionCandidate) →
  TowerActionAdmissible candidate → candidate ≡ sign-ordered-full-action
tower-action-unique no-tower-action ((), _)
tower-action-unique positive-face-only-action (_, ((), _))
tower-action-unique negative-face-only-action ((), _)
tower-action-unique unordered-full-action (_, (_, ((), _)))
tower-action-unique sign-ordered-full-action _ = refl
tower-action-unique sector-named-action (_, (_, (_, ())))

```

The surviving action has a single rational factor. The following block shows the positive face, the negative face, and the product which enters the final Planck-ratio masses.

```

-- $ tower-positive-action-numerator.
tower-positive-action-numerator : ℕ
tower-positive-action-numerator =
  (correction-tower-denominator boundary-correction +
  correction-tower-numerator boundary-correction) *
  (correction-tower-denominator mass-correction +
  correction-tower-numerator mass-correction)

```

```

tower-positive-action-denominator : ℕ
tower-positive-action-denominator =
  correction-tower-denominator boundary-correction *
  correction-tower-denominator mass-correction

tower-negative-action-numerator : ℕ
tower-negative-action-numerator =
  (correction-tower-denominator degree-correction ÷
   correction-tower-numerator degree-correction) *
  (correction-tower-denominator coupling-correction ÷
   correction-tower-numerator coupling-correction)

tower-negative-action-denominator : ℕ
tower-negative-action-denominator =
  correction-tower-denominator degree-correction *
  correction-tower-denominator coupling-correction

tower-action-numerator : ℕ
tower-action-numerator =
  tower-positive-action-numerator * tower-negative-action-numerator

tower-action-denominator : ℕ
tower-action-denominator =
  tower-positive-action-denominator * tower-negative-action-denominator

tower-positive-numerator-is-8455528347 :
tower-positive-action-numerator ≡ 8455528347
tower-positive-numerator-is-8455528347 = refl

tower-negative-numerator-is-180328799721 :
tower-negative-action-numerator ≡ 180328799721
tower-negative-numerator-is-180328799721 = refl

tower-positive-denominator-is-8454608664 :
tower-positive-action-denominator ≡ 8454608664
tower-positive-denominator-is-8454608664 = refl

tower-negative-denominator-is-180364984832 :
tower-negative-action-denominator ≡ 180364984832
tower-negative-denominator-is-180364984832 = refl

tower-action-numerator-is-1524775277821401191187 :
tower-action-numerator ≡ 1524775277821401191187
tower-action-numerator-is-1524775277821401191187 = refl

tower-action-denominator-is-1524915363442855784448 :
tower-action-denominator ≡ 1524915363442855784448
tower-action-denominator-is-1524915363442855784448 = refl

record CanonicalTowerActionSchema : Set where
  field
    tower-action-candidate : TowerActionCandidate
    tower-action-proof : TowerActionAdmissible tower-action-candidate
    tower-action-is-unique :
      (candidate : TowerActionCandidate) →
      TowerActionAdmissible candidate → candidate ≡ sign-ordered-full-action
    tower-action-num : ℕ
    tower-action-den : ℕ
    tower-action-num-locked : tower-action-num ≡ 1524775277821401191187
    tower-action-den-locked : tower-action-den ≡ 1524915363442855784448

canonical-tower-action-schema : CanonicalTowerActionSchema
canonical-tower-action-schema = record
{ tower-action-candidate = sign-ordered-full-action
; tower-action-proof = tower-action-admissible
; tower-action-is-unique = tower-action-unique
; tower-action-num = tower-action-numerator
; tower-action-den = tower-action-denominator
; tower-action-num-locked = refl

```

```
; tower-action-den-locked = refl
}
```

Before the corrected bridge is admitted, the paper records that no bare Planck bridge reaches the low-energy scale. This is the guardrail that keeps the correction tower from looking optional.

```
-- $ PreservesNeutralSchema.
PreservesNeutralSchema : ScaleBridgeCandidate → Set
PreservesNeutralSchema direct-planck-neutral = ⊤
PreservesNeutralSchema planck-higgs-neutral = ⊤
PreservesNeutralSchema planck-electron-neutral = ⊤
PreservesNeutralSchema electron-neutral-only = ⊤

HasStructuredSuppression : ScaleBridgeCandidate → Set
HasStructuredSuppression candidate = StructuredBridgeRoute candidate

MatchesLowEnergyScale : ScaleBridgeCandidate → Set
MatchesLowEnergyScale direct-planck-neutral = ⊥
MatchesLowEnergyScale planck-higgs-neutral = ⊥
MatchesLowEnergyScale planck-electron-neutral = ⊥
MatchesLowEnergyScale electron-neutral-only = ⊥

PlanckBridgeAdmissible : ScaleBridgeCandidate → Set
PlanckBridgeAdmissible candidate =
  HasStructuredSuppression candidate ×
  (MatchesLowEnergyScale candidate × PreservesNeutralSchema candidate)

NoUncorrectedPlanckBridge : Set
NoUncorrectedPlanckBridge =
  (candidate : ScaleBridgeCandidate) → PlanckBridgeAdmissible candidate → ⊥

no-uncorrected-planck-bridge : NoUncorrectedPlanckBridge
no-uncorrected-planck-bridge direct-planck-neutral (⊥, ((), ⊥))
no-uncorrected-planck-bridge planck-higgs-neutral (⊥, ((), ⊥))
no-uncorrected-planck-bridge planck-electron-neutral (⊥, ((), ⊥))
no-uncorrected-planck-bridge electron-neutral-only (⊥, ((), ⊥))

planck-electron-neutral-not-admissible-before-correction :
  PlanckBridgeAdmissible planck-electron-neutral → ⊥
planck-electron-neutral-not-admissible-before-correction (⊥, ((), ⊥))
```

The structured candidate records the denominator and every locked slot in the Planck-to-electron suppression. It is not yet the final low-energy bridge; it is the checked structured candidate before the final reading filter.

```
-- $ StructuredPlanckElectronCandidate.
record StructuredPlanckElectronCandidate : Set where
  field
    candidate-classification      : CanonicalPlanckSuppressionSchema
    candidate-route               : canonical-bridge-route ≡ planck-electron-neutral
    candidate-neutral-survivor    : canonical-neutral-survivor ≡ neutral-depth
    candidate-kappa-base         : bridge-kappa ≡ 8
    candidate-fermat-base         : fermatPrime canonicalK4Ledger ≡ 17
    candidate-closure-base       : bridge-closure-constant ≡ 137
    candidate-kappa-survivor      : canonical-kappa-exponent ≡ kappa-euler-slot
    candidate-fermat-survivor     :
      canonical-fermat-exponent ≡ fermat-carrier-slot
    candidate-closure-survivor    :
      canonical-closure-exponent ≡ closure-carrier-generation-slot
    candidate-neutral-interface   : NeutralMassInterface
    candidate-denominator         : ℕ
    candidate-denominator-locked :
      candidate-denominator ≡ 24209712604567890429760
    candidate-depth              : neutral-depth-value ≡ 5
    candidate-neutral-slot        : bridge-neutral-slot ≡ 5
    candidate-kappa-slot-count    : bridge-kappa-slot-count ≡ 2
    candidate-fermat-slot-count   : bridge-fermat-slot-count ≡ 4
    candidate-closure-slot-count  : bridge-closure-slot-count ≡ 7
```

```

candidate-kappa-square      : bridge-kappa-square ≡ 64
candidate-kappa-power       :
  bridge-kappa-square ≡ bridge-kappa ^ bridge-kappa-slot-count
candidate-fermat-fourth     : bridge-fermat-fourth ≡ 83521
candidate-fermat-power      :
  bridge-fermat-fourth ≡
    fermatPrime canonicalK4Ledger ^ bridge-fermat-slot-count
candidate-closure-seventh   : bridge-closure-seventh ≡ 905824306333433
candidate-closure-power     :
  bridge-closure-seventh ≡ bridge-closure-constant ^ bridge-closure-slot-count
candidate-layer             : BridgeLayer
candidate-layer-structured  : candidate-layer ≡ structured-candidate
candidate-uncorrected-eliminated :
  PlanckBridgeAdmissible planck-electron-neutral → ⊥

```

```

structured-planck-electron-candidate : StructuredPlanckElectronCandidate
structured-planck-electron-candidate = record
{ candidate-classification = canonical-planck-suppression-schema
; candidate-route          = refl
; candidate-neutral-survivor = refl
; candidate-kappa-base     = refl
; candidate-fermat-base    = refl
; candidate-closure-base   = refl
; candidate-kappa-survivor = refl
; candidate-fermat-survivor = refl
; candidate-closure-survivor = refl
; candidate-neutral-interface = neutral-mass-interface
; candidate-denominator    = planck-electron-suppression-denominator
; candidate-denominator-locked = refl
; candidate-depth          = refl
; candidate-neutral-slot   = refl
; candidate-kappa-slot-count = refl
; candidate-fermat-slot-count = refl
; candidate-closure-slot-count = refl
; candidate-kappa-square   = refl
; candidate-kappa-power    = refl
; candidate-fermat-fourth  = refl
; candidate-fermat-power   = refl
; candidate-closure-seventh = refl
; candidate-closure-power  = refl
; candidate-layer          = structured-candidate
; candidate-layer-structured = refl
; candidate-uncorrected-eliminated = planck-electron-neutral-not-admissible-before-correction
}

```

The search-state record preserves the negative information: before correction, the bridge search is still at the initial stage and all uncorrected candidates have been eliminated.

```

-- $ PlanckBridgeSearchState.
record PlanckBridgeSearchState : Set where
field
  search-neutral-interface : NeutralMassInterface
  search-current-anchor    : ScaleAnchor
  search-target-anchor     : ScaleAnchor
  search-current-is-electron : search-current-anchor ≡ electron-scale
  search-target-is-planck   : search-target-anchor ≡ planck-scale
  search-status            : BridgeLayer
  search-status-initial    : search-status ≡ initial-search
  uncorrected-candidates-eliminated : NoUncorrectedPlanckBridge

planck-bridge-search-state : PlanckBridgeSearchState
planck-bridge-search-state = record
{ search-neutral-interface = neutral-mass-interface
; search-current-anchor    = electron-scale
; search-target-anchor     = planck-scale
; search-current-is-electron = refl
; search-target-is-planck   = refl
; search-status            = initial-search
; search-status-initial    = refl
; uncorrected-candidates-eliminated = no-uncorrected-planck-bridge
}

```

The final low-energy bridge candidate must combine all preceding pieces: Planck suppression, running, complete tower interface, canonical tower action, and no empirical patch. This is the bridge used by the mass prediction.

```
-- $ LowEnergyReadingCandidate.
data LowEnergyReadingCandidate : Set where
bare-structured-reading : LowEnergyReadingCandidate
running-only-reading   : LowEnergyReadingCandidate
tower-interface-reading : LowEnergyReadingCandidate
sign-ordered-tower-reading : LowEnergyReadingCandidate
empirical-patch-reading : LowEnergyReadingCandidate

UsesPlanckSuppressionSurvivor : LowEnergyReadingCandidate → Set
UsesPlanckSuppressionSurvivor bare-structured-reading = ⊤
UsesPlanckSuppressionSurvivor running-only-reading   = ⊤
UsesPlanckSuppressionSurvivor tower-interface-reading = ⊤
UsesPlanckSuppressionSurvivor sign-ordered-tower-reading = ⊤
UsesPlanckSuppressionSurvivor empirical-patch-reading = ⊤

UsesRunningSurvivor : LowEnergyReadingCandidate → Set
UsesRunningSurvivor bare-structured-reading = ⊥
UsesRunningSurvivor running-only-reading   = ⊤
UsesRunningSurvivor tower-interface-reading = ⊤
UsesRunningSurvivor sign-ordered-tower-reading = ⊤
UsesRunningSurvivor empirical-patch-reading = ⊤

UsesCompleteTowerInterface : LowEnergyReadingCandidate → Set
UsesCompleteTowerInterface bare-structured-reading = ⊥
UsesCompleteTowerInterface running-only-reading   = ⊥
UsesCompleteTowerInterface tower-interface-reading = ⊤
UsesCompleteTowerInterface sign-ordered-tower-reading = ⊤
UsesCompleteTowerInterface empirical-patch-reading = ⊤

UsesCanonicalTowerAction : LowEnergyReadingCandidate → Set
UsesCanonicalTowerAction bare-structured-reading = ⊥
UsesCanonicalTowerAction running-only-reading   = ⊥
UsesCanonicalTowerAction tower-interface-reading = ⊥
UsesCanonicalTowerAction sign-ordered-tower-reading = ⊤
UsesCanonicalTowerAction empirical-patch-reading = ⊤

AvoidsEmpiricalPatch : LowEnergyReadingCandidate → Set
AvoidsEmpiricalPatch bare-structured-reading = ⊤
AvoidsEmpiricalPatch running-only-reading   = ⊤
AvoidsEmpiricalPatch tower-interface-reading = ⊤
AvoidsEmpiricalPatch sign-ordered-tower-reading = ⊤
AvoidsEmpiricalPatch empirical-patch-reading = ⊥

LowEnergyBridgeAdmissible : LowEnergyReadingCandidate → Set
LowEnergyBridgeAdmissible candidate =
  UsesPlanckSuppressionSurvivor candidate ×
  (UsesRunningSurvivor candidate ×
  (UsesCompleteTowerInterface candidate ×
  (UsesCanonicalTowerAction candidate × AvoidsEmpiricalPatch candidate)))

low-energy-bridge-admissible :
  LowEnergyBridgeAdmissible sign-ordered-tower-reading
low-energy-bridge-admissible = tt , (tt , (tt , (tt , tt)))

low-energy-bridge-unique :
  (candidate : LowEnergyReadingCandidate) →
  LowEnergyBridgeAdmissible candidate →
  candidate ≡ sign-ordered-tower-reading
low-energy-bridge-unique bare-structured-reading ( , ( , ))
low-energy-bridge-unique running-only-reading ( , ( , ( , )))
low-energy-bridge-unique tower-interface-reading ( , ( , ( , ( , ))))
low-energy-bridge-unique sign-ordered-tower-reading _ = refl
low-energy-bridge-unique empirical-patch-reading ( , ( , ( , ( , ))))

record CanonicalLowEnergyBridgeSchema : Set where
```



```

field
bridge-reading-candidate : LowEnergyReadingCandidate
bridge-reading-proof    : LowEnergyBridgeAdmissible bridge-reading-candidate
bridge-reading-unique   :
  (candidate : LowEnergyReadingCandidate) →
  LowEnergyBridgeAdmissible candidate →
  candidate = sign-ordered-tower-reading
bridge-start            : ScaleAnchor
bridge-end              : ScaleAnchor
bridge-layer            : BridgeLayer
bridge-start-is-planck  : bridge-start = planck-scale
bridge-end-is-electron  : bridge-end = electron-scale
bridge-layer-is-closed  : bridge-layer = closed-formal-bridge

canonical-low-energy-bridge-schema : CanonicalLowEnergyBridgeSchema
canonical-low-energy-bridge-schema = record
{ bridge-reading-candidate = sign-ordered-tower-reading
; bridge-reading-proof    = low-energy-bridge-admissible
; bridge-reading-unique   = low-energy-bridge-unique
; bridge-start            = planck-scale
; bridge-end              = electron-scale
; bridge-layer            = closed-formal-bridge
; bridge-start-is-planck  = refl
; bridge-end-is-electron  = refl
; bridge-layer-is-closed  = refl
}

```

The corrected Planck bridge also audits the correction chain. The running factor must use the loop remainder and the RG denominator without replacing the bridge by a volume correction. The tower must use the sign-ordered action without letting sector names choose the result. The final low-energy bridge must close without an empirical patch. This is where the QFT/QCD reading is kept in its proper place: it can motivate the physical language of loop correction, but the neutral mass chain inherits only the formal loop grammar.

```

-- $ Planck bridge correction audit.
running-volume-substitution-blocked :
  RunningCorrectionAdmissible volume-loop-correction → ⊥
running-volume-substitution-blocked (⊥, (⊥, (⊥, ())))

tower-sector-name-choice-blocked :
  TowerActionAdmissible sector-named-action → ⊥
tower-sector-name-choice-blocked (⊥, (⊥, (⊥, ())))

low-energy-empirical-patch-blocked :
  LowEnergyBridgeAdmissible empirical-patch-reading → ⊥
low-energy-empirical-patch-blocked (⊥, (⊥, (⊥, (⊥, ())))

record PlanckBridgeCorrectionAudit : Set where
field
audit-suppression-schema : CanonicalPlanckSuppressionSchema
audit-running-schema      : CanonicalRunningCorrectionSchema
audit-tower-interface     : CanonicalCorrectionTowerInterface
audit-tower-schema        : CanonicalTowerActionSchema
audit-low-energy-schema    : CanonicalLowEnergyBridgeSchema
audit-route-locked        : canonical-bridge-route = planck-electron-neutral
audit-running-locked      :
  CanonicalRunningCorrectionSchema.correction-candidate
  audit-running-schema = degree-rg-loop
audit-tower-action-locked :
  CanonicalTowerActionSchema.tower-action-candidate
  audit-tower-schema = sign-ordered-full-action
audit-low-energy-reading-locked :
  CanonicalLowEnergyBridgeSchema.bridge-reading-candidate
  audit-low-energy-schema = sign-ordered-tower-reading
audit-volume-substitution-blocked :
  RunningCorrectionAdmissible volume-loop-correction → ⊥
audit-sector-name-action-blocked :
  TowerActionAdmissible sector-named-action → ⊥

```



```

audit-empirical-patch-blocked :
  LowEnergyBridgeAdmissible empirical-patch-reading → ⊥
audit-status : ProgrammeStatus
audit-checked-here : audit-status ≡ checked-here

planck-bridge-correction-audit : PlanckBridgeCorrectionAudit
planck-bridge-correction-audit = record
{
  audit-suppression-schema = canonical-planck-suppression-schema
; audit-running-schema    = canonical-running-correction-schema
; audit-tower-interface   = canonical-correction-tower-interface
; audit-tower-schema      = canonical-tower-action-schema
; audit-low-energy-schema  = canonical-low-energy-bridge-schema
; audit-route-locked      = canonical-bridge-route-is-planck-electron
; audit-running-locked    = refl
; audit-tower-action-locked = refl
; audit-low-energy-reading-locked = refl
; audit-volume-substitution-blocked = running-volume-substitution-blocked
; audit-sector-name-action-blocked = tower-sector-name-choice-blocked
; audit-empirical-patch-blocked = low-energy-empirical-patch-blocked
; audit-status             = checked-here
; audit-checked-here       = refl
}

```

7 Exact electron-anchored prediction candidate

The Planck scale is the natural scale for the ultraviolet cutoff: in the discrete reading, one lattice edge may be read as one Planck length and the corresponding cutoff energy as one Planck mass. This fixes the unit in which a low-energy particle mass is measured; it does not select such a mass by itself. If the denominator below were applied directly to the Planck mass, the result would be a Planck-sector excitation, not a neutrino-scale mass: $M_P/(4 \cdot 137^3) \approx 1.19 \cdot 10^{21}$ eV. To obtain a 0.05 eV neutrino directly from M_P , the ledger would need an additional suppression of about $2.46 \cdot 10^{29}$. The Planck-to-low-energy bridge supplying that suppression is classified in the Planck-anchored layer; the electron-anchored layer is retained as a comparison reading.

The exact prediction needs one physical scale. The choice made here is the electron mass, because the mass ledger developed here expresses charged-particle masses as ratios to m_e . The neutral prediction is therefore not an electron-volt theorem. It is an exact dimensionless claim:

$$m_1 = 0, \quad \frac{m_2}{m_e} = \frac{1}{6 \cdot 4 \cdot 137^3}, \quad \frac{m_3}{m_e} = \frac{1}{4 \cdot 137^3}.$$

Here 137 is the closure integer, the exponent 3 is the generation depth, 4 is the K_4 carrier, and 6 is the edge-sector separation between the two nonzero neutral masses. The neutral-depth interface above remains present: it locks the admissible neutral slot at depth 5 with square 25. The scale formula below is the additional physical reading that turns the slot into an absolute mass proposal.

Using $m_e = 510998.950$ eV only as the external scale, this candidate gives

$$m_1 = 0, \quad m_2 = 0.008280318928 \text{ eV}, \quad m_3 = 0.049681913568 \text{ eV},$$

and

$$\sum_i m_i = 0.057962232497 \text{ eV}.$$

Those decimals are not hard-coded below; the checked content is the integer denominator pattern and the boundary level of the scale anchor.

```

-- $ PredictionLayer.
data PredictionLayer : Set where

```

```

formal-ledger      : PredictionLayer
physical-anchor    : PredictionLayer
experimental-comparison : PredictionLayer

closure-constant : ℕ
closure-constant = 137

closure-cubed : ℕ
closure-cubed = closure-constant * closure-constant * closure-constant

closure-cubed-is-2571353 : closure-cubed ≡ 2571353
closure-cubed-is-2571353 = refl

heaviest-neutral-denominator : ℕ
heaviest-neutral-denominator = vertices canonicalK4Ledge * closure-cubed

heaviest-neutral-denominator-is-10285412 :
heaviest-neutral-denominator ≡ 10285412
heaviest-neutral-denominator-is-10285412 = refl

middle-neutral-denominator : ℕ
middle-neutral-denominator = edges canonicalK4Ledge * heaviest-neutral-denominator

middle-neutral-denominator-is-61712472 :
middle-neutral-denominator ≡ 61712472
middle-neutral-denominator-is-61712472 = refl

```

The two nonzero neutral denominators differ by a multiplicative separator. The candidate space below prevents that separator from being smuggled in as the desired number: boundary, Euler, degree, vertices, and edges are all tested against the same four filters.

```

-- $ NeutralPairSeparator.
data NeutralPairSeparator : Set where
-- $ boundaryUnit = 1
separator-boundary : NeutralPairSeparator
-- $ euler = 2
separator-euler : NeutralPairSeparator
-- $ degree = 3
separator-degree : NeutralPairSeparator
-- $ vertices = 4
separator-vertices : NeutralPairSeparator
-- $ edges = 6
separator-edges : NeutralPairSeparator

separator-value : NeutralPairSeparator → ℕ
separator-value separator-boundary = boundaryUnit canonicalK4Ledge
separator-value separator-euler = euler canonicalK4Ledge
separator-value separator-degree = degree canonicalK4Ledge
separator-value separator-vertices = vertices canonicalK4Ledge
separator-value separator-edges = edges canonicalK4Ledge

-- $ SeparatesPair.
SeparatesPair : NeutralPairSeparator → Set
SeparatesPair separator-boundary = ⊥
SeparatesPair separator-euler = ⊤
SeparatesPair separator-degree = ⊤
SeparatesPair separator-vertices = ⊤
SeparatesPair separator-edges = ⊤

-- $ NotCarrierReuse.
NotCarrierReuse : NeutralPairSeparator → Set
NotCarrierReuse separator-boundary = ⊤
NotCarrierReuse separator-euler = ⊤
NotCarrierReuse separator-degree = ⊤
NotCarrierReuse separator-vertices = ⊥
NotCarrierReuse separator-edges = ⊤

-- $ NotExponentReuse.
NotExponentReuse : NeutralPairSeparator → Set

```

```

NotExponentReuse separator-boundary = T
NotExponentReuse separator-euler    = T
NotExponentReuse separator-degree   = ⊥
NotExponentReuse separator-vertices = T
NotExponentReuse separator-edges    = T

-- $ PrimitiveCellCount.
PrimitiveCellCount : NeutralPairSeparator → Set
PrimitiveCellCount separator-boundary = T
PrimitiveCellCount separator-euler    = ⊥
PrimitiveCellCount separator-degree   = T
PrimitiveCellCount separator-vertices = T
PrimitiveCellCount separator-edges    = T

NeutralPairSeparatorAdmissible : NeutralPairSeparator → Set
NeutralPairSeparatorAdmissible candidate =
  SeparatesPair candidate ×
  (NotCarrierReuse candidate ×
   (NotExponentReuse candidate × PrimitiveCellCount candidate))

neutral-pair-separator-admissible :
  NeutralPairSeparatorAdmissible separator-edges
neutral-pair-separator-admissible = tt , (tt , (tt , tt))

neutral-pair-separator-unique :
  (candidate : NeutralPairSeparator) →
  NeutralPairSeparatorAdmissible candidate →
  candidate ≡ separator-edges
neutral-pair-separator-unique separator-boundary ((), _)
neutral-pair-separator-unique separator-euler    (⊥ , (⊥ , (⊥ , ())))
neutral-pair-separator-unique separator-degree   (⊥ , (⊥ , ((), _)))
neutral-pair-separator-unique separator-vertices (⊥ , ((), _))
neutral-pair-separator-unique separator-edges    _ = refl

separator-value-of-edges-is-six :
  separator-value separator-edges ≡ 6
separator-value-of-edges-is-six = refl

```

Once the edge separator is forced, the neutral ratio locks follow directly. This short block records the factorisation and the bare squared ratio.

```

-- $ middle-denominator-factors-as-separator.
middle-denominator-factors-as-separator :
  middle-neutral-denominator ≡
  separator-value separator-edges * heaviest-neutral-denominator
middle-denominator-factors-as-separator = refl

bare-mass-ratio-square : ℕ
bare-mass-ratio-square =
  separator-value separator-edges * separator-value separator-edges

bare-mass-ratio-square-is-36 : bare-mass-ratio-square ≡ 36
bare-mass-ratio-square-is-36 = refl

record NeutralPairSeparatorSchema : Set where
  field
    separator-candidate : NeutralPairSeparator
    separator-admissible : NeutralPairSeparatorAdmissible separator-candidate
    separator-uniqueness :
      (candidate : NeutralPairSeparator) →
      NeutralPairSeparatorAdmissible candidate →
      candidate ≡ separator-edges
    separator-locks-edges : separator-candidate ≡ separator-edges
    separator-locks-six : separator-value separator-candidate ≡ 6
    middle-as-separator :
      middle-neutral-denominator ≡
      separator-value separator-edges * heaviest-neutral-denominator
    bare-square-locks-36 : bare-mass-ratio-square ≡ 36

```

```

neutral-pair-separator-schema : NeutralPairSeparatorSchema
neutral-pair-separator-schema = record
{ separator-candidate = separator-edges
; separator-admissible = neutral-pair-separator-admissible
; separator-uniqueness = neutral-pair-separator-unique
; separator-locks-edges = refl
; separator-locks-six = refl
; middle-as-separator = refl
; bare-square-locks-36 = refl
}

```

The four filters are load-bearing. The essentiality witnesses show what would go wrong if any one condition were dropped: boundary, vertices, degree, or Euler would re-enter as a rival.

```

-- $ AdmissibleWithoutSeparatesPair.
AdmissibleWithoutSeparatesPair : NeutralPairSeparator → Set
AdmissibleWithoutSeparatesPair candidate =
  NotCarrierReuse candidate ×
  (NotExponentReuse candidate × PrimitiveCellCount candidate)

AdmissibleWithoutNotCarrierReuse : NeutralPairSeparator → Set
AdmissibleWithoutNotCarrierReuse candidate =
  SeparatesPair candidate ×
  (NotExponentReuse candidate × PrimitiveCellCount candidate)

AdmissibleWithoutNotExponentReuse : NeutralPairSeparator → Set
AdmissibleWithoutNotExponentReuse candidate =
  SeparatesPair candidate ×
  (NotCarrierReuse candidate × PrimitiveCellCount candidate)

AdmissibleWithoutPrimitiveCellCount : NeutralPairSeparator → Set
AdmissibleWithoutPrimitiveCellCount candidate =
  SeparatesPair candidate ×
  (NotCarrierReuse candidate × NotExponentReuse candidate)

NotEdges : NeutralPairSeparator → Set
NotEdges separator-boundary = ⊤
NotEdges separator-euler = ⊤
NotEdges separator-degree = ⊤
NotEdges separator-vertices = ⊤
NotEdges separator-edges = ⊥

-- $ Drop SeparatesPair: separator-boundary survives.
separates-pair-essential :
  Σ NeutralPairSeparator
  (λ candidate →
    AdmissibleWithoutSeparatesPair candidate × NotEdges candidate)
separates-pair-essential =
  separator-boundary , ((tt , (tt , tt)) , tt)

-- $ Drop NotCarrierReuse: separator-vertices survives.
not-carrier-reuse-essential :
  Σ NeutralPairSeparator
  (λ candidate →
    AdmissibleWithoutNotCarrierReuse candidate × NotEdges candidate)
not-carrier-reuse-essential =
  separator-vertices , ((tt , (tt , tt)) , tt)

-- $ Drop NotExponentReuse: separator-degree survives.
not-exponent-reuse-essential :
  Σ NeutralPairSeparator
  (λ candidate →
    AdmissibleWithoutNotExponentReuse candidate × NotEdges candidate)
not-exponent-reuse-essential =
  separator-degree , ((tt , (tt , tt)) , tt)

-- $ Drop PrimitiveCellCount: separator-euler survives.
primitive-cell-count-essential :
  Σ NeutralPairSeparator

```

```

(λ candidate →
  AdmissibleWithoutPrimitiveCellCount candidate × NotEdges candidate)
primitive-cell-count-essential =
separator-euler , ((tt , (tt , tt)) , tt)

record NeutralPairSeparatorEssentiality : Set where
field
  separates-pair-witness :
    Σ NeutralPairSeparator
    (λ candidate →
      AdmissibleWithoutSeparatesPair candidate × NotEdges candidate)
  not-carrier-reuse-witness :
    Σ NeutralPairSeparator
    (λ candidate →
      AdmissibleWithoutNotCarrierReuse candidate × NotEdges candidate)
  not-exponent-reuse-witness :
    Σ NeutralPairSeparator
    (λ candidate →
      AdmissibleWithoutNotExponentReuse candidate × NotEdges candidate)
  primitive-cell-count-witness :
    Σ NeutralPairSeparator
    (λ candidate →
      AdmissibleWithoutPrimitiveCellCount candidate × NotEdges candidate)

neutral-pair-separator-essentiality : NeutralPairSeparatorEssentiality
neutral-pair-separator-essentiality = record
{ separates-pair-witness = separates-pair-essential
; not-carrier-reuse-witness = not-carrier-reuse-essential
; not-exponent-reuse-witness = not-exponent-reuse-essential
; primitive-cell-count-witness = primitive-cell-count-essential
}

```

With the denominator structure fixed, the rational mass records simply attach the external electron scale. The zero, middle, heaviest, and summed entries remain exact ratios.

```

-- $ electron-mass-microeV.
electron-mass-microeV : ℕ
electron-mass-microeV = 510998950000

record RationalMass : Set where
field
  numerator-microeV : ℕ
  denominator : ℕ

lightest-neutral-mass : RationalMass
lightest-neutral-mass = record
{ numerator-microeV = 0
; denominator = 1
}

middle-neutral-mass : RationalMass
middle-neutral-mass = record
{ numerator-microeV = electron-mass-microeV
; denominator = middle-neutral-denominator
}

heaviest-neutral-mass : RationalMass
heaviest-neutral-mass = record
{ numerator-microeV = electron-mass-microeV
; denominator = heaviest-neutral-denominator
}

normal-hierarchy-sum : RationalMass
normal-hierarchy-sum = record
{ numerator-microeV = 7 * electron-mass-microeV
; denominator = middle-neutral-denominator
}

record ElectronAnchoredNeutralPrediction : Set where

```

```

field
neutral-interface      : NeutralMassInterface
neutral-square-locked  : NeutralMassInterface.depth-denominator neutral-interface = 25
closure-tower          : closure-cubed = 2571353
heaviest-denominator   : heaviest-neutral-denominator = 10285412
middle-denominator     : middle-neutral-denominator = 61712472
lightest-ratio         : RationalMass
middle-ratio           : RationalMass
heaviest-ratio         : RationalMass
summed-ratio           : RationalMass
denominator-layer      : PredictionLayer
electron-scale-layer    : PredictionLayer
comparison-layer        : PredictionLayer
denominator-is-formal  : denominator-layer = formal-ledger
electron-scale-physical : electron-scale-layer = physical-anchor
comparison-is-empirical : comparison-layer = experimental-comparison

electron-anchored-neutral-prediction : ElectronAnchoredNeutralPrediction
electron-anchored-neutral-prediction = record
{ neutral-interface      = neutral-mass-interface
; neutral-square-locked = refl
; closure-tower         = refl
; heaviest-denominator  = refl
; middle-denominator    = refl
; lightest-ratio        = lightest-neutral-mass
; middle-ratio          = middle-neutral-mass
; heaviest-ratio        = heaviest-neutral-mass
; summed-ratio          = normal-hierarchy-sum
; denominator-layer     = formal-ledger
; electron-scale-layer   = physical-anchor
; comparison-layer       = experimental-comparison
; denominator-is-formal = refl
; electron-scale-physical = refl
; comparison-is-empirical = refl
}

```

8 Planck-anchored mass-chain

The Planck-anchored chain begins with the survivor of a finite route classification and three finite slot classifications. The route is forced to the Planck-to-electron bridge; the κ exponent is forced to the Euler slot; the Fermat exponent is forced to the carrier slot; and the closure exponent is forced to the carrier-plus-generation slot. Together with the already proved neutral survivor this gives the canonical denominator

$$5 \cdot 8^2 \cdot 17^4 \cdot 137^7$$

without using the decimal comparison as a filter.

Read by itself, this bare survivor already gives exact rational multiples of M_P . With $M_P \approx 1.22089 \cdot 10^{28}$ eV it yields, before the running and tower corrections, approximately

$$m_e = 504297.6015 \text{ eV}, \quad m_p = 925.8904 \text{ MeV}, \quad m_\mu = 104.3896 \text{ MeV}, \quad m_\tau = 1774.6233 \text{ MeV},$$

and

$$m_3 = 0.0490303744 \text{ eV}, \quad m_2 = 0.0081717291 \text{ eV}, \quad \sum_i m_i = 0.0572021035 \text{ eV}.$$

These numbers are consequences of the classified survivor. They are not the reason it was selected. They are also not the final reading of the chain: the bare bridge lacks the running and tower action that the next filters require.

The running gate is also classified rather than fitted. The survivor is the degree-weighted RG loop correction

$$\frac{d \cdot 274 + 11}{d \cdot 274} = \frac{833}{822}.$$

Applied to the bare Planck chain, it gives the running-corrected consequences

$$m_e = 511046.1096 \text{ eV}, \quad m_p = 938.2807 \text{ MeV}, \quad m_\mu = 105.7865 \text{ MeV}, \quad m_\tau = 1798.3713 \text{ MeV},$$

and

$$m_3 = 0.0496864987 \text{ eV}, \quad m_2 = 0.0082810831 \text{ eV}, \quad \sum_i m_i = 0.0579675818 \text{ eV}.$$

These corrected numbers are again consequences, not filters. The running-only stage is intermediate: the full correction tower follows.

The correction tower has a finite interface: two positive terms

$$\frac{2}{137^2}, \quad \frac{1}{8 \cdot 3 \cdot 137^2}$$

and two negative terms

$$-\frac{3^2}{8 \cdot 137^2}, \quad -\frac{(11+2)^2}{8^2 \cdot 137^2}.$$

The sign-forced action is classified next: the two positive terms act first, then the two negative terms. This gives the exact tower factor

$$\frac{1524775277821401191187}{1524915363442855784448}.$$

Composed with the running-corrected Planck chain, this yields

$$m_e = 510999.1626 \text{ eV}, \quad m_p = 938.1945 \text{ MeV}, \quad m_\mu = 105.7768 \text{ MeV}, \quad m_\tau = 1798.2061 \text{ MeV},$$

and

$$m_3 = 0.0496819342 \text{ eV}, \quad m_2 = 0.0082803224 \text{ eV}, \quad \sum_i m_i = 0.0579622566 \text{ eV}.$$

The tower-level normal-hierarchy observables are therefore fixed as exact squared Planck ratios before the neutral residual carrier is applied. In eV units, using the same Planck input, they read approximately

$$\Delta m_{21}^2 = 6.8564 \cdot 10^{-5} \text{ eV}^2, \quad \Delta m_{31}^2 = 2.4683 \cdot 10^{-3} \text{ eV}^2.$$

At this point the formal bridge closes at the level of the displayed filters. Among the finite low-energy readings, only the sign-ordered tower action uses the Planck-suppression survivor, the running survivor, the complete tower interface, and no empirical patch. The residual carrier classification below supplies the final neutral separator; the empirical boundary is the comparison of the resulting Planck ratios with measured masses and neutrino data.

The charged tiers then receive their sector corrections. The proton uses $1836 + 11/72 = 132203/72$, the muon uses $207 - 16/69 = 14267/69$, and the tau/muon tier uses $17 - 28/153 = 2573/153$. Applied after the formal Planck bridge, these give the charged-mass reading

$$m_p = 938.2725 \text{ MeV}, \quad m_\mu = 105.6583 \text{ MeV}, \quad m_\tau = 1776.8555 \text{ MeV}.$$

Again, the displayed decimals are consequences of the rational chain; the filters select the fractions before any measured mass is consulted.

```
-- $ PlanckRatio.
record PlanckRatio : Set where
field
numerator : N
```

```

denominator : ℕ

proton-electron-ratio : ℕ
proton-electron-ratio = 1836

muon-electron-ratio : ℕ
muon-electron-ratio = 207

tau-muon-ratio : ℕ
tau-muon-ratio = fermatPrime canonicalK4Ledge

tau-electron-ratio : ℕ
tau-electron-ratio = muon-electron-ratio * tau-muon-ratio

tau-electron-ratio-is-3519 : tau-electron-ratio = 3519
tau-electron-ratio-is-3519 = refl

data MuonBridgeCandidate : Set where
  muon-chi-bridge : MuonBridgeCandidate
  muon-vertex-bridge : MuonBridgeCandidate
  muon-kappa-bridge : MuonBridgeCandidate
  muon-spinor-bridge : MuonBridgeCandidate

MuonBridgeSaturatesSpinor : MuonBridgeCandidate → Set
MuonBridgeSaturatesSpinor muon-chi-bridge = ⊥
MuonBridgeSaturatesSpinor muon-vertex-bridge = ⊥
MuonBridgeSaturatesSpinor muon-kappa-bridge = ⊥
MuonBridgeSaturatesSpinor muon-spinor-bridge = ⊤

muon-bridge-admissible : MuonBridgeSaturatesSpinor muon-spinor-bridge
muon-bridge-admissible = tt

muon-bridge-unique :
  (candidate : MuonBridgeCandidate) →
  MuonBridgeSaturatesSpinor candidate → candidate = muon-spinor-bridge
muon-bridge-unique muon-chi-bridge ()
muon-bridge-unique muon-vertex-bridge ()
muon-bridge-unique muon-kappa-bridge ()
muon-bridge-unique muon-spinor-bridge _ = refl

data TauExtensionCandidate : Set where
  tau-vertex-extension : TauExtensionCandidate
  tau-edge-extension : TauExtensionCandidate
  tau-degree-extension : TauExtensionCandidate
  tau-euler-extension : TauExtensionCandidate
  tau-kappa-extension : TauExtensionCandidate

TauGeometricCoprime : TauExtensionCandidate → Set
TauGeometricCoprime tau-vertex-extension = ⊥
TauGeometricCoprime tau-edge-extension = ⊤
TauGeometricCoprime tau-degree-extension = ⊤
TauGeometricCoprime tau-euler-extension = ⊤
TauGeometricCoprime tau-kappa-extension = ⊤

TauCompositionalCoprime : TauExtensionCandidate → Set
TauCompositionalCoprime tau-vertex-extension = ⊤
TauCompositionalCoprime tau-edge-extension = ⊤
TauCompositionalCoprime tau-degree-extension = ⊤
TauCompositionalCoprime tau-euler-extension = ⊤
TauCompositionalCoprime tau-kappa-extension = ⊥

TauSubDegreeBound : TauExtensionCandidate → Set
TauSubDegreeBound tau-vertex-extension = ⊥
TauSubDegreeBound tau-edge-extension = ⊥
TauSubDegreeBound tau-degree-extension = ⊥
TauSubDegreeBound tau-euler-extension = ⊤
TauSubDegreeBound tau-kappa-extension = ⊥

TauExtensionAdmissible : TauExtensionCandidate → Set
TauExtensionAdmissible candidate =

```



```

TauGeometricCoprime candidate ×
(TauCompositionalCoprime candidate × TauSubDegreeBound candidate)

```

```

tau-extension-admissible : TauExtensionAdmissible tau-euler-extension
tau-extension-admissible = tt , (tt , tt)

```

```

tau-extension-unique :
(candidate : TauExtensionCandidate) →
TauExtensionAdmissible candidate → candidate ≡ tau-euler-extension
tau-extension-unique tau-vertex-extension (() , _)
tau-extension-unique tau-edge-extension (_, ( , ()))
tau-extension-unique tau-degree-extension (_, ( , ()))
tau-extension-unique tau-euler-extension _ = refl
tau-extension-unique tau-kappa-extension (_, (() , _))

```

The sector corrections are rational refinements of the charged ratios. The proton receives $+11/72$, the muon receives $-16/69$, and the tau/muon tier receives $-28/153$.

```

-- $ proton-sector-correction-numerator.
proton-sector-correction-numerator : ℕ
proton-sector-correction-numerator = loopNumerator canonicalK4Ledger

proton-sector-correction-denominator : ℕ
proton-sector-correction-denominator = bulkVolume canonicalK4Ledger

proton-sector-corrected-numerator : ℕ
proton-sector-corrected-numerator =
proton-electron-ratio * proton-sector-correction-denominator +
proton-sector-correction-numerator

proton-sector-corrected-denominator : ℕ
proton-sector-corrected-denominator = proton-sector-correction-denominator

muon-sector-correction-numerator : ℕ
muon-sector-correction-numerator =
kappaScale canonicalK4Ledger * euler canonicalK4Ledger

muon-sector-correction-denominator : ℕ
muon-sector-correction-denominator =
degree canonicalK4Ledger *
(edges canonicalK4Ledger + fermatPrime canonicalK4Ledger)

muon-sector-corrected-numerator : ℕ
muon-sector-corrected-numerator =
muon-electron-ratio * muon-sector-correction-denominator ÷
muon-sector-correction-numerator

muon-sector-corrected-denominator : ℕ
muon-sector-corrected-denominator = muon-sector-correction-denominator

tau-sector-correction-numerator : ℕ
tau-sector-correction-numerator =
degree canonicalK4Ledger * degree canonicalK4Ledger +
fermatPrime canonicalK4Ledger + euler canonicalK4Ledger

tau-sector-correction-denominator : ℕ
tau-sector-correction-denominator =
degree canonicalK4Ledger * degree canonicalK4Ledger *
fermatPrime canonicalK4Ledger

tau-muon-sector-corrected-numerator : ℕ
tau-muon-sector-corrected-numerator =
tau-muon-ratio * tau-sector-correction-denominator ÷
tau-sector-correction-numerator

tau-muon-sector-corrected-denominator : ℕ
tau-muon-sector-corrected-denominator = tau-sector-correction-denominator

tau-electron-sector-corrected-numerator : ℕ

```

```

tau-electron-sector-corrected-numerator =
  muon-sector-corrected-numerator * tau-muon-sector-corrected-numerator

tau-electron-sector-corrected-denominator : ℕ
tau-electron-sector-corrected-denominator =
  muon-sector-corrected-denominator * tau-muon-sector-corrected-denominator

proton-sector-correction-is-11-over-72 :
  proton-sector-correction-numerator ≡ 11 ×
  proton-sector-correction-denominator ≡ 72
proton-sector-correction-is-11-over-72 = refl , refl

proton-sector-corrected-numerator-is-132203 :
  proton-sector-corrected-numerator ≡ 132203
proton-sector-corrected-numerator-is-132203 = refl

muon-sector-correction-is-16-over-69 :
  muon-sector-correction-numerator ≡ 16 ×
  muon-sector-correction-denominator ≡ 69
muon-sector-correction-is-16-over-69 = refl , refl

muon-sector-corrected-numerator-is-14267 :
  muon-sector-corrected-numerator ≡ 14267
muon-sector-corrected-numerator-is-14267 = refl

tau-sector-correction-is-28-over-153 :
  tau-sector-correction-numerator ≡ 28 ×
  tau-sector-correction-denominator ≡ 153
tau-sector-correction-is-28-over-153 = refl , refl

tau-muon-sector-corrected-numerator-is-2573 :
  tau-muon-sector-corrected-numerator ≡ 2573
tau-muon-sector-corrected-numerator-is-2573 = refl

tau-electron-sector-corrected-numerator-is-36708991 :
  tau-electron-sector-corrected-numerator ≡ 36708991
tau-electron-sector-corrected-numerator-is-36708991 = refl

tau-electron-sector-corrected-denominator-is-10557 :
  tau-electron-sector-corrected-denominator ≡ 10557
tau-electron-sector-corrected-denominator-is-10557 = refl

```

The charged correction record packages the finite muon/tau filters with the three corrected charged ratios. This keeps the sector corrections as a checked object instead of scattered arithmetic.

```

-- $ Package the charged corrections and their uniqueness proofs.
record CanonicalChargedSectorCorrections : Set where
  field
    muon-bridge-survivor      : MuonBridgeCandidate
    muon-bridge-proof        : MuonBridgeSaturatesSpinor muon-bridge-survivor
    muon-bridge-is-unique     :
      (candidate : MuonBridgeCandidate) →
      MuonBridgeSaturatesSpinor candidate → candidate ≡ muon-spinor-bridge
    tau-extension-survivor    : TauExtensionCandidate
    tau-extension-proof       : TauExtensionAdmissible tau-extension-survivor
    tau-extension-is-unique   :
      (candidate : TauExtensionCandidate) →
      TauExtensionAdmissible candidate → candidate ≡ tau-euler-extension
    proton-ratio-numerator    : proton-sector-corrected-numerator ≡ 132203
    proton-ratio-denominator  : proton-sector-corrected-denominator ≡ 72
    muon-ratio-numerator      : muon-sector-corrected-numerator ≡ 14267
    muon-ratio-denominator    : muon-sector-corrected-denominator ≡ 69
    tau-muon-ratio-numerator   : tau-muon-sector-corrected-numerator ≡ 2573
    tau-muon-ratio-denominator : tau-muon-sector-corrected-denominator ≡ 153
    tau-electron-ratio-numerator :
      tau-electron-sector-corrected-numerator ≡ 36708991
    tau-electron-ratio-denominator :
      tau-electron-sector-corrected-denominator ≡ 10557

```

```

canonical-charged-sector-corrections : CanonicalChargedSectorCorrections
canonical-charged-sector-corrections = record
{
  muon-bridge-survivor      = muon-spinor-bridge
; muon-bridge-proof        = muon-bridge-admissible
; muon-bridge-is-unique    = muon-bridge-unique
; tau-extension-survivor   = tau-euler-extension
; tau-extension-proof      = tau-extension-admissible
; tau-extension-is-unique  = tau-extension-unique
; proton-ratio-numerator   = refl
; proton-ratio-denominator = refl
; muon-ratio-numerator     = refl
; muon-ratio-denominator  = refl
; tau-muon-ratio-numerator = refl
; tau-muon-ratio-denominator = refl
; tau-electron-ratio-numerator = refl
; tau-electron-ratio-denominator = refl
}

```

The next block moves from charged ratios to Planck ratios. First the bare Planck bridge is recorded, then the running correction is applied uniformly to electron, proton, muon, tau, and the two nonzero neutral masses.

```

-- $ electron-planck-ratio.
electron-planck-ratio : PlanckRatio
electron-planck-ratio = record
{
  numerator = 1
; denominator = planck-electron-suppression-denominator
}

running-corrected-electron-planck-denominator : ℕ
running-corrected-electron-planck-denominator =
  running-correction-denominator * planck-electron-suppression-denominator

running-corrected-electron-planck-denominator-is-19900383760954805933262720 :
  running-corrected-electron-planck-denominator ≡ 19900383760954805933262720
running-corrected-electron-planck-denominator-is-19900383760954805933262720 = refl

running-corrected-electron-planck-ratio : PlanckRatio
running-corrected-electron-planck-ratio = record
{
  numerator = running-correction-numerator
; denominator = running-corrected-electron-planck-denominator
}

proton-planck-ratio : PlanckRatio
proton-planck-ratio = record
{
  numerator = proton-electron-ratio
; denominator = planck-electron-suppression-denominator
}

muon-planck-ratio : PlanckRatio
muon-planck-ratio = record
{
  numerator = muon-electron-ratio
; denominator = planck-electron-suppression-denominator
}

tau-planck-ratio : PlanckRatio
tau-planck-ratio = record
{
  numerator = tau-electron-ratio
; denominator = planck-electron-suppression-denominator
}

running-corrected-proton-planck-ratio : PlanckRatio
running-corrected-proton-planck-ratio = record
{
  numerator = proton-electron-ratio * running-correction-numerator
; denominator = running-corrected-electron-planck-denominator
}

running-corrected-muon-planck-ratio : PlanckRatio
running-corrected-muon-planck-ratio = record

```

```

{ numerator = muon-electron-ratio * running-correction-numerator
; denominator = running-corrected-electron-planck-denominator
}

running-corrected-tau-planck-ratio : PlanckRatio
running-corrected-tau-planck-ratio = record
{ numerator = tau-electron-ratio * running-correction-numerator
; denominator = running-corrected-electron-planck-denominator
}

running-corrected-proton-numerator-is-1529388 :
PlanckRatio.numerator running-corrected-proton-planck-ratio ≡ 1529388
running-corrected-proton-numerator-is-1529388 = refl

running-corrected-muon-numerator-is-172431 :
PlanckRatio.numerator running-corrected-muon-planck-ratio ≡ 172431
running-corrected-muon-numerator-is-172431 = refl

running-corrected-tau-numerator-is-2931327 :
PlanckRatio.numerator running-corrected-tau-planck-ratio ≡ 2931327
running-corrected-tau-numerator-is-2931327 = refl

```

The same running-corrected bridge is then applied to the neutral denominators. This produces the middle and heaviest Planck-ratio denominators before the tower action.

```

-- $ Neutral Planck denominators before and after the running correction.
heaviest-neutral-planck-denominator : ℕ
heaviest-neutral-planck-denominator =
planck-electron-suppression-denominator * heaviest-neutral-denominator

heaviest-neutral-planck-denominator-is-249006868539573835040938661120 :
heaviest-neutral-planck-denominator ≡ 249006868539573835040938661120
heaviest-neutral-planck-denominator-is-249006868539573835040938661120 = refl

middle-neutral-planck-denominator : ℕ
middle-neutral-planck-denominator =
planck-electron-suppression-denominator * middle-neutral-denominator

middle-neutral-planck-denominator-is-1494041211237443010245631966720 :
middle-neutral-planck-denominator ≡ 1494041211237443010245631966720
middle-neutral-planck-denominator-is-1494041211237443010245631966720 = refl

heaviest-neutral-planck-ratio : PlanckRatio
heaviest-neutral-planck-ratio = record
{ numerator = 1
; denominator = heaviest-neutral-planck-denominator
}

middle-neutral-planck-ratio : PlanckRatio
middle-neutral-planck-ratio = record
{ numerator = 1
; denominator = middle-neutral-planck-denominator
}

normal-hierarchy-planck-sum : PlanckRatio
normal-hierarchy-planck-sum = record
{ numerator = 7
; denominator = middle-neutral-planck-denominator
}

running-corrected-heaviest-neutral-planck-denominator : ℕ
running-corrected-heaviest-neutral-planck-denominator =
running-corrected-electron-planck-denominator * heaviest-neutral-denominator

running-corrected-heaviest-neutral-planck-denominator-is-204683645939529692403651579440640 :
running-corrected-heaviest-neutral-planck-denominator ≡
204683645939529692403651579440640
running-corrected-heaviest-neutral-planck-denominator-is-204683645939529692403651579440640 = refl

```

```

running-corrected-middle-neutral-planck-denominator : ℕ
running-corrected-middle-neutral-planck-denominator =
  running-corrected-electron-planck-denominator * middle-neutral-denominator

running-corrected-middle-neutral-planck-denominator-is-1228101875637178154421909476643840 :
  running-corrected-middle-neutral-planck-denominator ≡
    1228101875637178154421909476643840
running-corrected-middle-neutral-planck-denominator-is-1228101875637178154421909476643840 = refl

running-corrected-heaviest-neutral-planck-ratio : PlanckRatio
running-corrected-heaviest-neutral-planck-ratio = record
{ numerator = running-correction-numerator
; denominator = running-corrected-heaviest-neutral-planck-denominator
}

running-corrected-middle-neutral-planck-ratio : PlanckRatio
running-corrected-middle-neutral-planck-ratio = record
{ numerator = running-correction-numerator
; denominator = running-corrected-middle-neutral-planck-denominator
}

running-corrected-normal-hierarchy-planck-sum : PlanckRatio
running-corrected-normal-hierarchy-planck-sum = record
{ numerator = 7 * running-correction-numerator
; denominator = running-corrected-middle-neutral-planck-denominator
}

running-corrected-neutral-sum-numerator-is-5831 :
  PlanckRatio.numerator running-corrected-normal-hierarchy-planck-sum ≡ 5831
running-corrected-neutral-sum-numerator-is-5831 = refl

```

The tower action now multiplies the running-corrected electron bridge. The same corrected bridge is then used for the charged particles and for the two neutral denominators.

```

-- $ tower-corrected-electron-planck-numerator.
tower-corrected-electron-planck-numerator : ℕ
tower-corrected-electron-planck-numerator =
  running-correction-numerator * tower-action-numerator

tower-corrected-electron-planck-numerator-is-1270137806425227192258771 :
  tower-corrected-electron-planck-numerator ≡ 1270137806425227192258771
tower-corrected-electron-planck-numerator-is-1270137806425227192258771 = refl

tower-corrected-electron-planck-denominator : ℕ
tower-corrected-electron-planck-denominator =
  running-corrected-electron-planck-denominator * tower-action-denominator

tower-corrected-electron-planck-denominator-is-30346400935488703177589755538218026601674178560 :
  tower-corrected-electron-planck-denominator ≡
    30346400935488703177589755538218026601674178560
tower-corrected-electron-planck-denominator-is-30346400935488703177589755538218026601674178560 = refl

tower-corrected-electron-planck-ratio : PlanckRatio
tower-corrected-electron-planck-ratio = record
{ numerator = tower-corrected-electron-planck-numerator
; denominator = tower-corrected-electron-planck-denominator
}

tower-corrected-proton-planck-ratio : PlanckRatio
tower-corrected-proton-planck-ratio = record
{ numerator = proton-electron-ratio * tower-corrected-electron-planck-numerator
; denominator = tower-corrected-electron-planck-denominator
}

tower-corrected-muon-planck-ratio : PlanckRatio
tower-corrected-muon-planck-ratio = record
{ numerator = muon-electron-ratio * tower-corrected-electron-planck-numerator
; denominator = tower-corrected-electron-planck-denominator
}

```

```

tower-corrected-tau-planck-ratio : PlanckRatio
tower-corrected-tau-planck-ratio = record
{ numerator = tau-electron-ratio * tower-corrected-electron-planck-numerator
; denominator = tower-corrected-electron-planck-denominator
}

tower-corrected-proton-numerator-is-2331973012596717124987103556 :
PlanckRatio.numerator tower-corrected-proton-planck-ratio =
2331973012596717124987103556
tower-corrected-proton-numerator-is-2331973012596717124987103556 = refl

tower-corrected-muon-numerator-is-262918525930022028797565597 :
PlanckRatio.numerator tower-corrected-muon-planck-ratio =
262918525930022028797565597
tower-corrected-muon-numerator-is-262918525930022028797565597 = refl

tower-corrected-tau-numerator-is-4469614940810374489558615149 :
PlanckRatio.numerator tower-corrected-tau-planck-ratio =
4469614940810374489558615149
tower-corrected-tau-numerator-is-4469614940810374489558615149 = refl

tower-corrected-heaviest-neutral-planck-denominator : ℕ
tower-corrected-heaviest-neutral-planck-denominator =
tower-corrected-electron-planck-denominator * heaviest-neutral-denominator

tower-corrected-heaviest-neutral-planck-denominator-is-312125236338686733527219802689854149425178816251166720 :
tower-corrected-heaviest-neutral-planck-denominator =
312125236338686733527219802689854149425178816251166720
tower-corrected-heaviest-neutral-planck-denominator-is-312125236338686733527219802689854149425178816251166720 = refl

tower-corrected-middle-neutral-planck-denominator : ℕ
tower-corrected-middle-neutral-planck-denominator =
tower-corrected-electron-planck-denominator * middle-neutral-denominator

tower-corrected-middle-neutral-planck-denominator-is-1872751418032120401163318816139124896551072897507000320 :
tower-corrected-middle-neutral-planck-denominator =
1872751418032120401163318816139124896551072897507000320
tower-corrected-middle-neutral-planck-denominator-is-1872751418032120401163318816139124896551072897507000320 = refl

```

The separator remains invariant under the running and tower layers because those corrections act uniformly on the shared Planck bridge. This matters: the neutral splitting ratio is not re-fitted after the tower is applied.

```

-- $ bare-neutral-pair-separator-identity.
bare-neutral-pair-separator-identity :
middle-neutral-denominator =
separator-value separator-edges * heaviest-neutral-denominator
bare-neutral-pair-separator-identity = refl

running-corrected-neutral-pair-separator-identity :
running-corrected-middle-neutral-planck-denominator =
separator-value separator-edges *
running-corrected-heaviest-neutral-planck-denominator
running-corrected-neutral-pair-separator-identity = refl

tower-corrected-neutral-pair-separator-identity :
tower-corrected-middle-neutral-planck-denominator =
separator-value separator-edges *
tower-corrected-heaviest-neutral-planck-denominator
tower-corrected-neutral-pair-separator-identity = refl

record NeutralPairSeparatorUniformity : Set where
field
bare-identity :
middle-neutral-denominator =
separator-value separator-edges * heaviest-neutral-denominator
running-corrected-identity :
running-corrected-middle-neutral-planck-denominator =

```

```

separator-value separator-edges *
running-corrected-heaviest-neutral-planck-denominator
tower-corrected-identity :
tower-corrected-middle-neutral-planck-denominator =
separator-value separator-edges *
tower-corrected-heaviest-neutral-planck-denominator
bare-square-locks-36 : bare-mass-ratio-square = 36
separator-locks-six :
separator-value separator-edges = 6

neutral-pair-separator-uniformity : NeutralPairSeparatorUniformity
neutral-pair-separator-uniformity = record
{ bare-identity = refl
; running-corrected-identity = refl
; tower-corrected-identity = refl
; bare-square-locks-36 = refl
; separator-locks-six = refl
}

tower-corrected-heaviest-neutral-planck-ratio : PlanckRatio
tower-corrected-heaviest-neutral-planck-ratio = record
{ numerator = tower-corrected-electron-planck-numerator
; denominator = tower-corrected-heaviest-neutral-planck-denominator
}

tower-corrected-middle-neutral-planck-ratio : PlanckRatio
tower-corrected-middle-neutral-planck-ratio = record
{ numerator = tower-corrected-electron-planck-numerator
; denominator = tower-corrected-middle-neutral-planck-denominator
}

tower-corrected-normal-hierarchy-planck-sum : PlanckRatio
tower-corrected-normal-hierarchy-planck-sum = record
{ numerator = 7 * tower-corrected-electron-planck-numerator
; denominator = tower-corrected-middle-neutral-planck-denominator
}

tower-corrected-neutral-sum-numerator-is-8890964644976590345811397 :
PlanckRatio.numerator tower-corrected-normal-hierarchy-planck-sum =
8890964644976590345811397
tower-corrected-neutral-sum-numerator-is-8890964644976590345811397 = refl

```

The charged particles then receive their sector-specific correction fractions on top of the common tower-corrected bridge. This is the final charged-mass layer used for physical comparison.

```

-- $ sector-corrected-proton-planck-ratio.
sector-corrected-proton-planck-ratio : PlanckRatio
sector-corrected-proton-planck-ratio = record
{ numerator =
tower-corrected-electron-planck-numerator *
proton-sector-corrected-numerator
; denominator =
tower-corrected-electron-planck-denominator *
proton-sector-corrected-denominator
}

sector-corrected-muon-planck-ratio : PlanckRatio
sector-corrected-muon-planck-ratio = record
{ numerator =
tower-corrected-electron-planck-numerator *
muon-sector-corrected-numerator
; denominator =
tower-corrected-electron-planck-denominator *
muon-sector-corrected-denominator
}

sector-corrected-tau-planck-ratio : PlanckRatio
sector-corrected-tau-planck-ratio = record
{ numerator =
tower-corrected-electron-planck-numerator *

```

```

    tau-electron-sector-corrected-numerator
; denominator =
    tower-corrected-electron-planck-denominator *
    tau-electron-sector-corrected-denominator
}

sector-corrected-proton-numerator-is-167916028422834310498186302513 :
PlanckRatio.numerator sector-corrected-proton-planck-ratio =
167916028422834310498186302513
sector-corrected-proton-numerator-is-167916028422834310498186302513 = refl

sector-corrected-proton-denominator-is-2184940867355186628786462398751697915320540856320 :
PlanckRatio.denominator sector-corrected-proton-planck-ratio =
2184940867355186628786462398751697915320540856320
sector-corrected-proton-denominator-is-2184940867355186628786462398751697915320540856320 = refl

sector-corrected-muon-numerator-is-18121056084268716351955885857 :
PlanckRatio.numerator sector-corrected-muon-planck-ratio =
18121056084268716351955885857
sector-corrected-muon-numerator-is-18121056084268716351955885857 = refl

sector-corrected-muon-denominator-is-2093901664548720519253693132137043835515518320640 :
PlanckRatio.denominator sector-corrected-muon-planck-ratio =
2093901664548720519253693132137043835515518320640
sector-corrected-muon-denominator-is-2093901664548720519253693132137043835515518320640 = refl

sector-corrected-tau-numerator-is-46625477304823407173582494310061 :
PlanckRatio.numerator sector-corrected-tau-planck-ratio =
46625477304823407173582494310061
sector-corrected-tau-numerator-is-46625477304823407173582494310061 = refl

sector-corrected-tau-denominator-is-320366954675954239445815049216967706833874303057920 :
PlanckRatio.denominator sector-corrected-tau-planck-ratio =
320366954675954239445815049216967706833874303057920
sector-corrected-tau-denominator-is-320366954675954239445815049216967706833874303057920 = refl

```

The neutral observables are packaged after the ratios are fixed. The squared splittings are obtained by squaring the exact Planck ratios; no decimal oscillation data enter this block.

```

-- $ square-planck-ratio.
square-planck-ratio : PlanckRatio → PlanckRatio
square-planck-ratio ratio = record
{ numerator = PlanckRatio.numerator ratio ^ 2
; denominator = PlanckRatio.denominator ratio ^ 2
}

normal-delta21-planck-ratio : PlanckRatio
normal-delta21-planck-ratio =
square-planck-ratio tower-corrected-middle-neutral-planck-ratio

normal-delta31-planck-ratio : PlanckRatio
normal-delta31-planck-ratio =
square-planck-ratio tower-corrected-heaviest-neutral-planck-ratio

record NormalHierarchyObservableRatios : Set where
field
lightest-mass-ratio      : PlanckRatio
middle-mass-ratio       : PlanckRatio
heaviest-mass-ratio      : PlanckRatio
mass-sum-ratio          : PlanckRatio
delta21-squared-ratio    : PlanckRatio
delta31-squared-ratio    : PlanckRatio
lightest-is-zero         : PlanckRatio.numerator lightest-mass-ratio ≡ 0
middle-ratio-locked      : middle-mass-ratio ≡ tower-corrected-middle-neutral-planck-ratio
heaviest-ratio-locked     : heaviest-mass-ratio ≡ tower-corrected-heaviest-neutral-planck-ratio
sum-ratio-locked         : mass-sum-ratio ≡ tower-corrected-normal-hierarchy-planck-sum
delta21-from-middle      :
    delta21-squared-ratio ≡
    square-planck-ratio tower-corrected-middle-neutral-planck-ratio

```



```

delta31-from-heaviest :
  delta31-squared-ratio ≡
    square-planck-ratio tower-corrected-heaviest-neutral-planck-ratio

normal-hierarchy-observable-ratios : NormalHierarchyObservableRatios
normal-hierarchy-observable-ratios = record
{ lightest-mass-ratio = record
  { numerator = 0
    ; denominator = 1
  }
; middle-mass-ratio = tower-corrected-middle-neutral-planck-ratio
; heaviest-mass-ratio = tower-corrected-heaviest-neutral-planck-ratio
; mass-sum-ratio = tower-corrected-normal-hierarchy-planck-sum
; delta21-squared-ratio = normal-delta21-planck-ratio
; delta31-squared-ratio = normal-delta31-planck-ratio
; lightest-is-zero = refl
; middle-ratio-locked = refl
; heaviest-ratio-locked = refl
; sum-ratio-locked = refl
; delta21-from-middle = refl
; delta31-from-heaviest = refl
}

```

The candidate record collects the bridge, corrections, charged ratios and neutral observables into one closed object. Its fields are long because it acts as the ledger sheet for the full Planck-anchored chain.

```

-- $ PlanckAnchoredParticleMassCandidate.
record PlanckAnchoredParticleMassCandidate : Set where
field
  bridge-candidate          : StructuredPlanckElectronCandidate
  running-correction        : CanonicalRunningCorrectionSchema
  correction-tower          : CanonicalCorrectionTowerInterface
  tower-action              : CanonicalTowerActionSchema
  low-energy-bridge          : CanonicalLowEnergyBridgeSchema
  charged-sector-corrections : CanonicalChargedSectorCorrections
  neutrino-observables      : NormalHierarchyObservableRatios
  electron-ratio            : PlanckRatio
  proton-ratio              : PlanckRatio
  muon-ratio                : PlanckRatio
  tau-ratio                 : PlanckRatio
  corrected-electron-ratio   : PlanckRatio
  corrected-proton-ratio     : PlanckRatio
  corrected-muon-ratio       : PlanckRatio
  corrected-tau-ratio        : PlanckRatio
  middle-neutral-ratio      : PlanckRatio
  heaviest-neutral-ratio    : PlanckRatio
  neutral-sum-ratio         : PlanckRatio
  corrected-middle-neutral-ratio : PlanckRatio
  corrected-heaviest-neutral-ratio : PlanckRatio
  corrected-neutral-sum-ratio : PlanckRatio
  tower-corrected-electron-ratio : PlanckRatio
  tower-corrected-proton-ratio : PlanckRatio
  tower-corrected-muon-ratio : PlanckRatio
  tower-corrected-tau-ratio : PlanckRatio
  sector-corrected-proton-ratio : PlanckRatio
  sector-corrected-muon-ratio : PlanckRatio
  sector-corrected-tau-ratio : PlanckRatio
  tower-corrected-middle-neutral-ratio : PlanckRatio
  tower-corrected-heaviest-neutral-ratio : PlanckRatio
  tower-corrected-neutral-sum-ratio : PlanckRatio
  electron-denominator-locked :
    PlanckRatio.denominator electron-ratio ≡ 24209712604567890429760
  corrected-electron-numerator-locked :
    PlanckRatio.numerator corrected-electron-ratio ≡ 833
  corrected-electron-denominator-locked :
    PlanckRatio.denominator corrected-electron-ratio ≡
      19900383760954805933262720
  tau-numerator-locked : PlanckRatio.numerator tau-ratio ≡ 3519
  corrected-tau-numerator-locked :

```

```

PlanckRatio.numerator corrected-tau-ratio = 2931327
middle-denominator-locked :
PlanckRatio.denominator middle-neutral-ratio =
1494041211237443010245631966720
heaviest-denominator-locked :
PlanckRatio.denominator heaviest-neutral-ratio =
249006868539573835040938661120
corrected-middle-denominator-locked :
PlanckRatio.denominator corrected-middle-neutral-ratio =
1228101875637178154421909476643840
corrected-heaviest-denominator-locked :
PlanckRatio.denominator corrected-heaviest-neutral-ratio =
204683645939529692403651579440640
tower-corrected-electron-numerator-locked :
PlanckRatio.numerator tower-corrected-electron-ratio =
1270137806425227192258771
tower-corrected-electron-denominator-locked :
PlanckRatio.denominator tower-corrected-electron-ratio =
30346400935488703177589755538218026601674178560
tower-corrected-middle-denominator-locked :
PlanckRatio.denominator tower-corrected-middle-neutral-ratio =
1872751418032120401163318816139124896551072897507000320
tower-corrected-heaviest-denominator-locked :
PlanckRatio.denominator tower-corrected-heaviest-neutral-ratio =
312125236338686733527219802689854149425178816251166720
sector-corrected-proton-numerator-locked :
PlanckRatio.numerator sector-corrected-proton-ratio =
167916028422834310498186302513
sector-corrected-proton-denominator-locked :
PlanckRatio.denominator sector-corrected-proton-ratio =
2184940867355186628786462398751697915320540856320
sector-corrected-muon-numerator-locked :
PlanckRatio.numerator sector-corrected-muon-ratio =
18121056084268716351955885857
sector-corrected-muon-denominator-locked :
PlanckRatio.denominator sector-corrected-muon-ratio =
2093901664548720519253693132137043835515518320640
sector-corrected-tau-numerator-locked :
PlanckRatio.numerator sector-corrected-tau-ratio =
46625477304823407173582494310061
sector-corrected-tau-denominator-locked :
PlanckRatio.denominator sector-corrected-tau-ratio =
320366954675954239445815049216967706833874303057920
uncorrected-bridge-eliminated :
PlanckBridgeAdmissible planck-electron-neutral → ⊥

```

The canonical inhabitant is just the ledger filled out. All locked integer claims reduce by ‘refl’, and the uncorrected bridge remains eliminated.

```

-- $ planck-anchored-particle-mass-candidate.
planck-anchored-particle-mass-candidate :
PlanckAnchoredParticleMassCandidate
planck-anchored-particle-mass-candidate = record
{ bridge-candidate      = structured-planck-electron-candidate
; running-correction    = canonical-running-correction-schema
; correction-tower      = canonical-correction-tower-interface
; tower-action          = canonical-tower-action-schema
; low-energy-bridge      = canonical-low-energy-bridge-schema
; charged-sector-corrections = canonical-charged-sector-corrections
; neutrino-observables  = normal-hierarchy-observable-ratios
; electron-ratio        = electron-planck-ratio
; proton-ratio          = proton-planck-ratio
; muon-ratio            = muon-planck-ratio
; tau-ratio             = tau-planck-ratio
; corrected-electron-ratio = running-corrected-electron-planck-ratio
; corrected-proton-ratio = running-corrected-proton-planck-ratio
; corrected-muon-ratio   = running-corrected-muon-planck-ratio
; corrected-tau-ratio    = running-corrected-tau-planck-ratio
; middle-neutral-ratio   = middle-neutral-planck-ratio
; heaviest-neutral-ratio = heaviest-neutral-planck-ratio
; neutral-sum-ratio      = normal-hierarchy-planck-sum

```

```

; corrected-middle-neutral-ratio = running-corrected-middle-neutral-planck-ratio
; corrected-heaviest-neutral-ratio = running-corrected-heaviest-neutral-planck-ratio
; corrected-neutral-sum-ratio = running-corrected-normal-hierarchy-planck-sum
; tower-corrected-electron-ratio = tower-corrected-electron-planck-ratio
; tower-corrected-proton-ratio = tower-corrected-proton-planck-ratio
; tower-corrected-muon-ratio = tower-corrected-muon-planck-ratio
; tower-corrected-tau-ratio = tower-corrected-tau-planck-ratio
; sector-corrected-proton-ratio = sector-corrected-proton-planck-ratio
; sector-corrected-muon-ratio = sector-corrected-muon-planck-ratio
; sector-corrected-tau-ratio = sector-corrected-tau-planck-ratio
; tower-corrected-middle-neutral-ratio = tower-corrected-middle-neutral-planck-ratio
; tower-corrected-heaviest-neutral-ratio = tower-corrected-heaviest-neutral-planck-ratio
; tower-corrected-neutral-sum-ratio = tower-corrected-normal-hierarchy-planck-sum
; electron-denominator-locked = refl
; corrected-electron-numerator-locked = refl
; corrected-electron-denominator-locked = refl
; tau-numerator-locked = refl
; corrected-tau-numerator-locked = refl
; middle-denominator-locked = refl
; heaviest-denominator-locked = refl
; corrected-middle-denominator-locked = refl
; corrected-heaviest-denominator-locked = refl
; tower-corrected-electron-numerator-locked = refl
; tower-corrected-electron-denominator-locked = refl
; tower-corrected-middle-denominator-locked = refl
; tower-corrected-heaviest-denominator-locked = refl
; sector-corrected-proton-numerator-locked = refl
; sector-corrected-proton-denominator-locked = refl
; sector-corrected-muon-numerator-locked = refl
; sector-corrected-muon-denominator-locked = refl
; sector-corrected-tau-numerator-locked = refl
; sector-corrected-tau-denominator-locked = refl
; uncorrected-bridge-eliminated = planck-electron-neutral-not-admissible-before-correction
}

```

9 Residual layer: forced kind and carrier boundary

The first-order corrections classified above leave a structured residual when the rational ratios are read against precision data. Two observations are taken as input here, not as choices. In the charged sector the ppm-level residual carries a sign asymmetry: the hadronic ratio p/e misses on the negative side, while both leptonic ratios μ/e and τ/e miss on the positive side. In the neutral sector both squared-mass differences miss on the positive side, but their relative shifts are not equal: a single multiplicative factor on m_ν would preserve $\Delta m_{31}^2/\Delta m_{21}^2$, and that ratio is not preserved.

The residual layer is therefore classified at the level of *kind*. Three candidate kinds are admitted formally and one is forced by the data. In the charged sector, a global Planck-scale shift and a uniform running factor would impose a single sign on all charged residuals; both are eliminated by the observed Proton/Lepton sign split. Only a *sector-asymmetric* residual kind survives. In the neutral sector, a single multiplicative shift on the neutrino mass scale is eliminated by the non-unit splitting ratio; only a *splitting-asymmetric* residual kind survives.

The kind classification is the first formal output of this layer. In the charged sector it records a physical comparison boundary: the sign split is visible, but no exact charged residual carrier is selected by decimal matching. In the neutral sector the carrier is classified below by the coupling complement. Thus the charged residual is kept as **physical-test**, while the neutral residual is carried to a formal ratio.

```

-- $ ResidualSign.
data ResidualSign : Set where
  positive : ResidualSign
  negative : ResidualSign

```

```

data ChargedResidualKind : Set where
  global-scale-shift : ChargedResidualKind
  uniform-running    : ChargedResidualKind
  sector-asymmetric  : ChargedResidualKind
  no-charged-residual : ChargedResidualKind

ChargedNonTrivial : ChargedResidualKind → Set
ChargedNonTrivial global-scale-shift = ⊤
ChargedNonTrivial uniform-running    = ⊤
ChargedNonTrivial sector-asymmetric  = ⊤
ChargedNonTrivial no-charged-residual = ⊥

CarriesChargedSignAsymmetry : ChargedResidualKind → Set
CarriesChargedSignAsymmetry global-scale-shift = ⊥
CarriesChargedSignAsymmetry uniform-running    = ⊥
CarriesChargedSignAsymmetry sector-asymmetric  = ⊤
CarriesChargedSignAsymmetry no-charged-residual = ⊥

ChargedResidualKindAdmissible : ChargedResidualKind → Set
ChargedResidualKindAdmissible k =
  ChargedNonTrivial k × CarriesChargedSignAsymmetry k

charged-residual-kind-admissible :
  ChargedResidualKindAdmissible sector-asymmetric
charged-residual-kind-admissible = tt , tt

charged-residual-kind-unique :
  (k : ChargedResidualKind) →
  ChargedResidualKindAdmissible k →
  k ≡ sector-asymmetric
charged-residual-kind-unique global-scale-shift ( _ , () )
charged-residual-kind-unique uniform-running    ( _ , () )
charged-residual-kind-unique sector-asymmetric _ = refl
charged-residual-kind-unique no-charged-residual ( () , _ )

record ChargedResidualClassification : Set where
  field
    kind : ChargedResidualKind
    kind-admissible : ChargedResidualKindAdmissible kind
    kind-is-sector-asymmetric : kind ≡ sector-asymmetric
    hadronic-sign : ResidualSign
    leptonic-mu-sign : ResidualSign
    leptonic-tau-sign : ResidualSign
    hadronic-sign-locked : hadronic-sign ≡ negative
    leptonic-mu-sign-locked : leptonic-mu-sign ≡ positive
    leptonic-tau-sign-locked : leptonic-tau-sign ≡ positive
    carrier-status : ProgrammeStatus
    carrier-status-physical : carrier-status ≡ physical-test

charged-residual-classification : ChargedResidualClassification
charged-residual-classification = record
{ kind = sector-asymmetric
; kind-admissible = charged-residual-kind-admissible
; kind-is-sector-asymmetric = refl
; hadronic-sign = negative
; leptonic-mu-sign = positive
; leptonic-tau-sign = positive
; hadronic-sign-locked = refl
; leptonic-mu-sign-locked = refl
; leptonic-tau-sign-locked = refl
; carrier-status = physical-test
; carrier-status-physical = refl
}

```

The neutrino residual is different. It is not merely a sign asymmetry in a physical comparison table; the splitting-asymmetric carrier is later closed by the coupling complement.

```

-- $ NeutrinoResidualKind.
data NeutrinoResidualKind : Set where
  uniform-neutral-shift : NeutrinoResidualKind

```

```

splitting-asymmetric : NeutrinoResidualKind
no-neutrino-residual : NeutrinoResidualKind

NeutrinoNonTrivial : NeutrinoResidualKind → Set
NeutrinoNonTrivial uniform-neutral-shift = ⊤
NeutrinoNonTrivial splitting-asymmetric = ⊤
NeutrinoNonTrivial no-neutrino-residual = ⊥

CarriesSplittingAsymmetry : NeutrinoResidualKind → Set
CarriesSplittingAsymmetry uniform-neutral-shift = ⊥
CarriesSplittingAsymmetry splitting-asymmetric = ⊤
CarriesSplittingAsymmetry no-neutrino-residual = ⊥

NeutrinoResidualKindAdmissible : NeutrinoResidualKind → Set
NeutrinoResidualKindAdmissible k =
  NeutrinoNonTrivial k × CarriesSplittingAsymmetry k

neutrino-residual-kind-admissible :
  NeutrinoResidualKindAdmissible splitting-asymmetric
neutrino-residual-kind-admissible = tt , tt

neutrino-residual-kind-unique :
  (k : NeutrinoResidualKind) →
  NeutrinoResidualKindAdmissible k →
  k ≡ splitting-asymmetric
neutrino-residual-kind-unique uniform-neutral-shift ( _ , () )
neutrino-residual-kind-unique splitting-asymmetric _ = refl
neutrino-residual-kind-unique no-neutrino-residual ( () , _ )

record NeutrinoResidualClassification : Set where
  field
    kind : NeutrinoResidualKind
    kind-admissible : NeutrinoResidualKindAdmissible kind
    kind-is-splitting-asymmetric : kind ≡ splitting-asymmetric
    delta21-sign : ResidualSign
    delta31-sign : ResidualSign
    delta21-sign-locked : delta21-sign ≡ positive
    delta31-sign-locked : delta31-sign ≡ positive
    carrier-status : ProgrammeStatus
    carrier-status-checked-here : carrier-status ≡ checked-here

neutrino-residual-classification : NeutrinoResidualClassification
neutrino-residual-classification = record
{ kind = splitting-asymmetric
; kind-admissible = neutrino-residual-kind-admissible
; kind-is-splitting-asymmetric = refl
; delta21-sign = positive
; delta31-sign = positive
; delta21-sign-locked = refl
; delta31-sign-locked = refl
; carrier-status = checked-here
; carrier-status-checked-here = refl
}

record ResidualLayerStatus : Set where
  field
    charged-residuals : ChargedResidualClassification
    neutrino-residuals : NeutrinoResidualClassification
    charged-kind-locked :
      ChargedResidualClassification.kind charged-residuals
      ≡ sector-asymmetric
    neutrino-kind-locked :
      NeutrinoResidualClassification.kind neutrino-residuals
      ≡ splitting-asymmetric
    charged-carrier-physical :
      ChargedResidualClassification.carrier-status charged-residuals
      ≡ physical-test
    neutrino-carrier-checked-here :
      NeutrinoResidualClassification.carrier-status neutrino-residuals
      ≡ checked-here

```

```

residual-layer-status : ResidualLayerStatus
residual-layer-status = record
{ charged-residuals   = charged-residual-classification
; neutrino-residuals  = neutrino-residual-classification
; charged-kind-locked = refl
; neutrino-kind-locked = refl
; charged-carrier-physical = refl
; neutrino-carrier-checked-here = refl
}

```

10 Required-input classification for the carrier

The kind classification fixes *what* the residual layer must do. It does not fix *which* ledger slot carries it. Rather than guessing a slot by decimal matching, we classify the minimal structural input that any carrier candidate must provide before it can close. The candidate space is intentionally small and exhaustive at the level of admissible inputs.

The data fix two structural axes for the charged sector: a hadronic versus leptonic sign split, and a μ versus τ magnitude split. For the neutral sector the data fix one axis: a Δm_{21}^2 versus Δm_{31}^2 magnitude split. An admissible input must provide the corresponding resolution. The empirical patch input is globally excluded by the bridge classification above.

Two clean structural facts follow. The charged carrier requires both **sign-resolution** and **generation-resolution**; a higher-order-tower input alone does not produce a sign split, and a sign-resolution input alone does not separate μ from τ . The neutral carrier requires **generation-resolution**; a uniform neutral shift cannot reproduce the non-unit splitting ratio. The **higher-order-tower** input remains admissible only as a combination partner, never as a sole carrier. The **empirical-patch** input is forbidden on principle.

The basic K_4 ledger used up to this point does not carry an internal hadronic/leptonic $\mathbf{Z}_2$ distinction. The carrier closure therefore requires a genuine ledger extension, not a recycling of occupied slots. The next section supplies that extension from the intrinsic bipartitions and perfect matchings of K_4 .

```

-- $ CarrierInputCandidate.
data CarrierInputCandidate : Set where
empirical-patch   : CarrierInputCandidate
sign-resolution   : CarrierInputCandidate
generation-resolution : CarrierInputCandidate
higher-order-tower : CarrierInputCandidate
combined-resolution : CarrierInputCandidate

NotEmpiricalPatch : CarrierInputCandidate → Set
NotEmpiricalPatch empirical-patch   = ⊥
NotEmpiricalPatch sign-resolution   = ⊤
NotEmpiricalPatch generation-resolution = ⊤
NotEmpiricalPatch higher-order-tower = ⊤
NotEmpiricalPatch combined-resolution = ⊤

ProvidesSignResolution : CarrierInputCandidate → Set
ProvidesSignResolution empirical-patch   = ⊥
ProvidesSignResolution sign-resolution   = ⊤
ProvidesSignResolution generation-resolution = ⊥
ProvidesSignResolution higher-order-tower = ⊥
ProvidesSignResolution combined-resolution = ⊤

ProvidesGenerationResolution : CarrierInputCandidate → Set
ProvidesGenerationResolution empirical-patch   = ⊥
ProvidesGenerationResolution sign-resolution   = ⊥
ProvidesGenerationResolution generation-resolution = ⊤
ProvidesGenerationResolution higher-order-tower = ⊥
ProvidesGenerationResolution combined-resolution = ⊤

```

```

ChargedCarrierAdmissibleInput : CarrierInputCandidate → Set
ChargedCarrierAdmissibleInput k =
  NotEmpiricalPatch k ×
  (ProvidesSignResolution k × ProvidesGenerationResolution k)

```

```

charged-carrier-input-admissible :
  ChargedCarrierAdmissibleInput combined-resolution
charged-carrier-input-admissible = tt , (tt , tt)

```

```

charged-carrier-input-unique :
  (k : CarrierInputCandidate) →
  ChargedCarrierAdmissibleInput k →
  k ≡ combined-resolution
charged-carrier-input-unique empirical-patch      (( ) , _ )
charged-carrier-input-unique sign-resolution      ( _ , ( _ , ( ) ) )
charged-carrier-input-unique generation-resolution ( _ , (( ) , _ ) )
charged-carrier-input-unique higher-order-tower   ( _ , (( ) , _ ) )
charged-carrier-input-unique combined-resolution  _ = refl

```

```

NeutrinoCarrierAdmissibleInput : CarrierInputCandidate → Set
NeutrinoCarrierAdmissibleInput k =
  NotEmpiricalPatch k × ProvidesGenerationResolution k

```

```

neutrino-carrier-input-generation-admissible :
  NeutrinoCarrierAdmissibleInput generation-resolution
neutrino-carrier-input-generation-admissible = tt , tt

```

```

neutrino-carrier-input-combined-admissible :
  NeutrinoCarrierAdmissibleInput combined-resolution
neutrino-carrier-input-combined-admissible = tt , tt

```

The neutral admissible input is therefore not a singleton: both the pure generation-resolution input and the combined input satisfy the neutral filter. The next predicate selects the *minimal* admissible input by adding the requirement that no extra resolution is imported. In this finite candidate space, only the pure generation-resolution input survives.

```

-- $ NoSignResolutionImported.
NoSignResolutionImported : CarrierInputCandidate → Set
NoSignResolutionImported empirical-patch      = ⊥
NoSignResolutionImported sign-resolution      = ⊥
NoSignResolutionImported generation-resolution = ⊤
NoSignResolutionImported higher-order-tower   = ⊤
NoSignResolutionImported combined-resolution  = ⊥

NeutrinoCarrierMinimalInput : CarrierInputCandidate → Set
NeutrinoCarrierMinimalInput k =
  NeutrinoCarrierAdmissibleInput k × NoSignResolutionImported k

neutrino-carrier-input-minimal :
  NeutrinoCarrierMinimalInput generation-resolution
neutrino-carrier-input-minimal =
  neutrino-carrier-input-generation-admissible , tt

neutrino-carrier-input-minimal-unique :
  (k : CarrierInputCandidate) →
  NeutrinoCarrierMinimalInput k →
  k ≡ generation-resolution
neutrino-carrier-input-minimal-unique empirical-patch      ((( ) , _ ) , _ )
neutrino-carrier-input-minimal-unique sign-resolution      ( _ , ( ) )
neutrino-carrier-input-minimal-unique generation-resolution _ = refl
neutrino-carrier-input-minimal-unique higher-order-tower   (( _ , ( ) ) , _ )
neutrino-carrier-input-minimal-unique combined-resolution  ( _ , ( ) )

```

The next predicate records that the basic ledger does not carry the hadronic/leptonic distinction internally. This is a single boolean slot rather than a derivation, because the ledger exposed so far has no such distinction; the field keeps that limitation explicit.


```

-- $ LedgerSignStructure.
data LedgerSignStructure : Set where
  ledger-has-internal-sign-split : LedgerSignStructure
  ledger-lacks-internal-sign-split : LedgerSignStructure

current-ledger-sign-structure : LedgerSignStructure
current-ledger-sign-structure = ledger-lacks-internal-sign-split

LedgerSignSplitAbsent : LedgerSignStructure → Set
LedgerSignSplitAbsent ledger-has-internal-sign-split = ⊥
LedgerSignSplitAbsent ledger-lacks-internal-sign-split = ⊤

current-ledger-lacks-sign-split :
  LedgerSignSplitAbsent current-ledger-sign-structure
current-ledger-lacks-sign-split = tt

```

The combination is the formal output of this section. The charged carrier requires the combined input; the neutral carrier requires the generation-resolution input as the minimal admissible input; and the ledger as first displayed does not provide the sign split internally. The following extension resolves that structural requirement without using a fit.

```

-- $ CarrierClosureRequirement.
record CarrierClosureRequirement : Set where
  field
    charged-required-input      : CarrierInputCandidate
    neutrino-required-input     : CarrierInputCandidate
    charged-input-admissible    :
      ChargedCarrierAdmissibleInput charged-required-input
    charged-input-locked        :
    charged-required-input ≡ combined-resolution
    neutrino-input-minimal      :
      NeutrinoCarrierMinimalInput neutrino-required-input
    neutrino-input-locked       :
    neutrino-required-input ≡ generation-resolution
    ledger-sign-structure-recorded : LedgerSignStructure
    ledger-sign-structure-locked  :
    ledger-sign-structure-recorded ≡ ledger-lacks-internal-sign-split
    ledger-extension-classified   : ProgrammeStatus
    ledger-extension-classified-here :
    ledger-extension-classified ≡ checked-here

carrier-closure-requirement : CarrierClosureRequirement
carrier-closure-requirement = record
{
  charged-required-input      = combined-resolution
  neutrino-required-input     = generation-resolution
  charged-input-admissible    = charged-carrier-input-admissible
  charged-input-locked        = refl
  neutrino-input-minimal      = neutrino-carrier-input-minimal
  neutrino-input-locked       = refl
  ledger-sign-structure-recorded = ledger-lacks-internal-sign-split
  ledger-sign-structure-locked  = refl
  ledger-extension-classified   = checked-here
  ledger-extension-classified-here = refl
}

```

11 Internal ledger extension: bipartition and matching

The previous section recorded that the carrier closure requires a ledger extension. The next finite question is: which extension is admissible without importing structure foreign to K_4 ? Two intrinsic K_4 refinements are available before any external object is considered. First, the four vertices admit a bipartition into two pairs; the six edges then split into four cross edges and two internal edges. This is a $\mathbf{Z}_2$ refinement of the vertex set, and it is intrinsic to K_4 . Second, K_4 admits three perfect matchings, each consisting of two disjoint edges; this is an intrinsic three-fold refinement of the

edge set. Together they provide a sign resolution and a generation resolution drawn entirely from K_4 itself.

The candidate space below is exhaustive at the level of where the extension is taken from. An empirical flag is forbidden by the discipline established above. A foreign group adds a structure that is not visible in K_4 and would be a free choice. The pure bipartition provides a sign resolution but no generation refinement. The pure matching set provides a generation refinement but no sign resolution. Only the joint intrinsic extension provides both, and it does so without importing anything outside K_4 .

```
-- $ LedgerExtensionCandidate.
data LedgerExtensionCandidate : Set where
  empirical-flag      : LedgerExtensionCandidate
  foreign-group       : LedgerExtensionCandidate
  internal-bipartition : LedgerExtensionCandidate
  internal-matching-set : LedgerExtensionCandidate
  internal-bipartition-and-matching : LedgerExtensionCandidate

NotEmpiricalLedgerFlag : LedgerExtensionCandidate → Set
NotEmpiricalLedgerFlag empirical-flag      = ⊥
NotEmpiricalLedgerFlag foreign-group       = ⊤
NotEmpiricalLedgerFlag internal-bipartition = ⊤
NotEmpiricalLedgerFlag internal-matching-set = ⊤
NotEmpiricalLedgerFlag internal-bipartition-and-matching = ⊤

IntrinsicToK4 : LedgerExtensionCandidate → Set
IntrinsicToK4 empirical-flag      = ⊥
IntrinsicToK4 foreign-group       = ⊥
IntrinsicToK4 internal-bipartition = ⊤
IntrinsicToK4 internal-matching-set = ⊤
IntrinsicToK4 internal-bipartition-and-matching = ⊤

ProvidesLedgerSignSplit : LedgerExtensionCandidate → Set
ProvidesLedgerSignSplit empirical-flag      = ⊥
ProvidesLedgerSignSplit foreign-group       = ⊥
ProvidesLedgerSignSplit internal-bipartition = ⊤
ProvidesLedgerSignSplit internal-matching-set = ⊥
ProvidesLedgerSignSplit internal-bipartition-and-matching = ⊤

ProvidesLedgerGenerationSplit : LedgerExtensionCandidate → Set
ProvidesLedgerGenerationSplit empirical-flag      = ⊥
ProvidesLedgerGenerationSplit foreign-group       = ⊥
ProvidesLedgerGenerationSplit internal-bipartition = ⊥
ProvidesLedgerGenerationSplit internal-matching-set = ⊤
ProvidesLedgerGenerationSplit internal-bipartition-and-matching = ⊤

LedgerExtensionAdmissible : LedgerExtensionCandidate → Set
LedgerExtensionAdmissible k =
  NotEmpiricalLedgerFlag k ×
  (IntrinsicToK4 k ×
   (ProvidesLedgerSignSplit k × ProvidesLedgerGenerationSplit k))

ledger-extension-admissible :
  LedgerExtensionAdmissible internal-bipartition-and-matching
ledger-extension-admissible = tt , (tt , (tt , tt))

ledger-extension-unique :
  (k : LedgerExtensionCandidate) →
  LedgerExtensionAdmissible k →
  k ≡ internal-bipartition-and-matching
ledger-extension-unique empirical-flag      ((), ⊥)
ledger-extension-unique foreign-group       (⊥ , ((), ⊥))
ledger-extension-unique internal-bipartition (⊥ , (⊥ , (⊥ , ())))
ledger-extension-unique internal-matching-set (⊥ , (⊥ , ((), ⊥)))
ledger-extension-unique internal-bipartition-and-matching _ = refl
```

The intrinsic extension carries two integer counts that are forced by K_4 alone: the number of vertex bipartitions and the number of perfect matchings. A bipartition of four vertices into

two unordered pairs is determined by choosing two vertices for one side and dividing out the swap, giving three. Each perfect matching of K_4 is in bijection with such a bipartition by choosing the two cross edges that share no vertex, giving three again. Both counts equal three, and this coincidence is the formal source of the three-fold refinement used by the carrier.

```
-- $ IntrinsicLedgerExtension.
record IntrinsicLedgerExtension : Set where
field
  bipartition-count      : ℕ
  matching-count         : ℕ
  cross-edge-count       : ℕ
  internal-edge-count    : ℕ
  bipartition-count-is-three : bipartition-count ≡ 3
  matching-count-is-three  : matching-count ≡ 3
  cross-edge-count-is-four : cross-edge-count ≡ 4
  internal-edge-count-is-two : internal-edge-count ≡ 2
  cross-plus-internal-is-six :
    cross-edge-count + internal-edge-count ≡ edges canonicalK4Ledger
  bipartition-and-matching-coincide :
    bipartition-count ≡ matching-count

intrinsic-ledger-extension : IntrinsicLedgerExtension
intrinsic-ledger-extension = record
{ bipartition-count      = 3
; matching-count         = 3
; cross-edge-count       = 4
; internal-edge-count    = 2
; bipartition-count-is-three = refl
; matching-count-is-three  = refl
; cross-edge-count-is-four = refl
; internal-edge-count-is-two = refl
; cross-plus-internal-is-six = refl
; bipartition-and-matching-coincide = refl
}
```

The closure record below packages the intrinsic extension with the carrier-closure requirement. After this point the residual carrier problem reduces to the physical assignment of which side of the bipartition is the hadronic side and which matching is the first generation. That assignment is a physical reading and is therefore recorded as `physical-test`, not as a formal theorem.

```
-- $ IntrinsicCarrierClosure.
record IntrinsicCarrierClosure : Set where
field
  extension-survivor      : LedgerExtensionCandidate
  extension-admissible    :
    LedgerExtensionAdmissible extension-survivor
  extension-locked        :
  extension-survivor ≡ internal-bipartition-and-matching
  intrinsic-extension     : IntrinsicLedgerExtension
  requirement             : CarrierClosureRequirement
  sign-side-assignment    : ProgrammeStatus
  generation-matching-assignment : ProgrammeStatus
  sign-side-physical      :
  sign-side-assignment ≡ physical-test
  generation-matching-physical :
  generation-matching-assignment ≡ physical-test

intrinsic-carrier-closure : IntrinsicCarrierClosure
intrinsic-carrier-closure = record
{ extension-survivor      = internal-bipartition-and-matching
; extension-admissible    = ledger-extension-admissible
; extension-locked        = refl
; intrinsic-extension     = intrinsic-ledger-extension
; requirement             = carrier-closure-requirement
; sign-side-assignment    = physical-test
; generation-matching-assignment = physical-test
; sign-side-physical      = refl
; generation-matching-physical = refl
}
```

12 Cell structure forced by the intrinsic extension

The intrinsic extension produces a finite cell table. Each cell is a pair of a bipartition side and a matching index. Two sides times three matchings gives six cells, and the six edges of K_4 provide the counting witness: each edge belongs to exactly one matching and joins exactly one ordered pair of bipartition sides. The cell count and the edge count therefore coincide as a structural identity, not as a numerical fit.

The cell table has a forced internal structure: each side carries exactly three cells, indexed by the three matchings. In the leptonic sector this matches the three observed charged leptons. In the hadronic sector the structure forces three generation slots, of which the proton observable occupies one. The remaining two hadronic slots are predictions of the cell table, not data. They are recorded here as formal slots whose physical content is a separate identification.

```
-- $ BipartitionSide.
data BipartitionSide : Set where
  side-A : BipartitionSide
  side-B : BipartitionSide

data MatchingIndex : Set where
  matching-one : MatchingIndex
  matching-two : MatchingIndex
  matching-three : MatchingIndex

data ResidualCell : Set where
  cell : BipartitionSide → MatchingIndex → ResidualCell

cell-count : ℕ
cell-count = 2 * 3

cell-count-is-six : cell-count ≡ 6
cell-count-is-six = refl

cell-count-equals-edges :
  cell-count ≡ edges canonicalK4Ledge
cell-count-equals-edges = refl

generation-slots-per-side : ℕ
generation-slots-per-side = 3

generation-slots-per-side-is-three :
  generation-slots-per-side ≡ 3
generation-slots-per-side-is-three = refl
```

The next predicate names the observed cells. By the discipline established above, the side and matching assignment is a physical reading. Naming it here is therefore a record of *which physical content is consistent with the cell structure*, not a derivation. The proton occupies one hadronic slot; the electron, muon and tau occupy the three leptonic slots. The two unfilled hadronic slots are predictions of the cell structure that the first-order ledger leaves to physical identification against measured masses.

```
-- $ CellContent.
data CellContent : Set where
  hadronic-generation-one : CellContent
  hadronic-generation-two : CellContent
  hadronic-generation-three : CellContent
  leptonic-generation-one : CellContent
  leptonic-generation-two : CellContent
  leptonic-generation-three : CellContent

record CellAssignmentReading : Set where
  field
    hadronic-side : BipartitionSide
```

```

hadronic-side-locked : hadronic-side ≡ side-A
leptonic-side       : BipartitionSide
leptonic-side-locked : leptonic-side ≡ side-B
proton-cell         : ResidualCell
electron-cell       : ResidualCell
muon-cell           : ResidualCell
tau-cell            : ResidualCell
proton-cell-locked  :
  proton-cell ≡ cell side-A matching-one
electron-cell-locked :
  electron-cell ≡ cell side-B matching-one
muon-cell-locked    :
  muon-cell ≡ cell side-B matching-two
tau-cell-locked     :
  tau-cell ≡ cell side-B matching-three
hadronic-generation-two-status : ProgrammeStatus
hadronic-generation-three-status : ProgrammeStatus
hadronic-generation-two-physical :
  hadronic-generation-two-status ≡ physical-test
hadronic-generation-three-physical :
  hadronic-generation-three-status ≡ physical-test

cell-assignment-reading : CellAssignmentReading
cell-assignment-reading = record
{ hadronic-side       = side-A
; hadronic-side-locked = refl
; leptonic-side       = side-B
; leptonic-side-locked = refl
; proton-cell         = cell side-A matching-one
; electron-cell       = cell side-B matching-one
; muon-cell           = cell side-B matching-two
; tau-cell            = cell side-B matching-three
; proton-cell-locked  = refl
; electron-cell-locked = refl
; muon-cell-locked    = refl
; tau-cell-locked     = refl
; hadronic-generation-two-status = physical-test
; hadronic-generation-three-status = physical-test
; hadronic-generation-two-physical = refl
; hadronic-generation-three-physical = refl
}

```

13 Automorphism invariance forbids matching-differential weights

The intrinsic carrier closure forces three perfect matchings on K_4 and assigns the three observed leptonic generations to those three matchings. In principle one could try to absorb the residual ratio gap between $m_3/m_2 = 6$ and the experimental value $\sqrt{33.55} \approx 5.79$ by giving each matching its own multiplicative weight. The block below classifies that move and rules it out at the formal level. $\text{Aut}(K_4) = S_4$ acts transitively on the three perfect matchings; any K_4 -intrinsic weight assignment that depends on the matching index breaks $\text{Aut}(K_4)$ invariance. Under the joint conditions K_4 -intrinsic, $\text{Aut}(K_4)$ -invariant, no external generator, the only admissible weight pattern is uniform across all three matchings. Hence no purely intrinsic mechanism can produce a matching-differential factor on the neutral pair.

```

-- $ MatchingWeightPattern.
data MatchingWeightPattern : Set where
matching-weights-uniform      : MatchingWeightPattern
matching-weights-pairwise-distinct : MatchingWeightPattern
matching-weights-fully-distinct : MatchingWeightPattern
matching-weights-from-empirical-flag : MatchingWeightPattern
matching-weights-from-foreign-group : MatchingWeightPattern

IsK4Intrinsic : MatchingWeightPattern → Set
IsK4Intrinsic matching-weights-uniform = ⊤

```

```

IsK4Intrinsic matching-weights-pairwise-distinct = T
IsK4Intrinsic matching-weights-fully-distinct    = T
IsK4Intrinsic matching-weights-from-empirical-flag = ⊥
IsK4Intrinsic matching-weights-from-foreign-group = ⊥

NoExternalGenerator : MatchingWeightPattern → Set
NoExternalGenerator matching-weights-uniform      = T
NoExternalGenerator matching-weights-pairwise-distinct = T
NoExternalGenerator matching-weights-fully-distinct  = T
NoExternalGenerator matching-weights-from-empirical-flag = ⊥
NoExternalGenerator matching-weights-from-foreign-group = ⊥

-- $ RespectsMatchingAutInvariance.
RespectsMatchingAutInvariance : MatchingWeightPattern → Set
RespectsMatchingAutInvariance matching-weights-uniform      = T
RespectsMatchingAutInvariance matching-weights-pairwise-distinct = ⊥
RespectsMatchingAutInvariance matching-weights-fully-distinct  = ⊥
RespectsMatchingAutInvariance matching-weights-from-empirical-flag = ⊥
RespectsMatchingAutInvariance matching-weights-from-foreign-group = ⊥

MatchingWeightAdmissible : MatchingWeightPattern → Set
MatchingWeightAdmissible w =
  IsK4Intrinsic w ×
  (NoExternalGenerator w × RespectsMatchingAutInvariance w)

matching-weight-admissible :
  MatchingWeightAdmissible matching-weights-uniform
matching-weight-admissible = tt , (tt , tt)

matching-weight-unique :
  (w : MatchingWeightPattern) →
  MatchingWeightAdmissible w →
  w ≡ matching-weights-uniform
matching-weight-unique matching-weights-uniform      _ = refl
matching-weight-unique matching-weights-pairwise-distinct ( , ( , ( )))
matching-weight-unique matching-weights-fully-distinct   ( , ( , ( )))
matching-weight-unique matching-weights-from-empirical-flag ((), _)
matching-weight-unique matching-weights-from-foreign-group ((), _)

-- $ IntrinsicNeutralPairLock.
record IntrinsicNeutralPairLock : Set where
  field
    separator-locked   : separator-value separator-edges ≡ 6
    bare-square-locked : bare-mass-ratio-square ≡ 36
    matching-weight-survivor :
      MatchingWeightPattern
    matching-weight-admissible-proof :
      MatchingWeightAdmissible matching-weight-survivor
    matching-weight-unique-proof :
      (w : MatchingWeightPattern) →
      MatchingWeightAdmissible w →
      w ≡ matching-weights-uniform
    matching-weight-locked :
      matching-weight-survivor ≡ matching-weights-uniform
    bare-uniformity :
      middle-neutral-denominator ≡
      separator-value separator-edges * heaviest-neutral-denominator
    running-uniformity :
      running-corrected-middle-neutral-planck-denominator ≡
      separator-value separator-edges *
      running-corrected-heaviest-neutral-planck-denominator
    tower-uniformity :
      tower-corrected-middle-neutral-planck-denominator ≡
      separator-value separator-edges *
      tower-corrected-heaviest-neutral-planck-denominator
    above-matching-carrier-classified : ProgrammeStatus
    above-matching-carrier-classified-here :
      above-matching-carrier-classified ≡ checked-here

intrinsic-neutral-pair-lock : IntrinsicNeutralPairLock
intrinsic-neutral-pair-lock = record

```

```

{ separator-locked           = refl
; bare-square-locked         = refl
; matching-weight-survivor    = matching-weights-uniform
; matching-weight-admissible-proof = matching-weight-admissible
; matching-weight-unique-proof = matching-weight-unique
; matching-weight-locked     = refl
; bare-uniformity             = refl
; running-uniformity          = refl
; tower-uniformity            = refl
; above-matching-carrier-classified = checked-here
; above-matching-carrier-classified-here = refl
}

```

14 $\text{Aut}(K_4)$ -invariance on the matching weights as a real classification theorem

The previous block stated the consequence of automorphism invariance on a finite list of named patterns by a table. The block below proves the underlying classification on the open function space of all matching weights. A weight is any function from the three matching indices into $\mathbb{N}$. The automorphism group $\text{Aut}(K_4) = S_4$ acts on the four vertices of K_4 and induces an action on the three perfect matchings via the surjection $S_4 \twoheadrightarrow S_3$; under this induced action the three matchings form a single orbit. We use the two adjacent transpositions (matching-one matching-two) and (matching-two matching-three) as generators; both descend from $\text{Aut}(K_4)$ because each corresponds to a vertex permutation that fixes one matching and swaps the other two. Their fixed points are the constant weights.

```

-- $ MatchingWeight.
MatchingWeight : Set
MatchingWeight = MatchingIndex → ℕ

data MatchingPermutation : Set where
swap-one-two : MatchingPermutation
swap-two-three : MatchingPermutation

apply-matching-perm :
  MatchingPermutation → MatchingIndex → MatchingIndex
apply-matching-perm swap-one-two matching-one = matching-two
apply-matching-perm swap-one-two matching-two = matching-one
apply-matching-perm swap-one-two matching-three = matching-three
apply-matching-perm swap-two-three matching-one = matching-one
apply-matching-perm swap-two-three matching-two = matching-three
apply-matching-perm swap-two-three matching-three = matching-two

MatchingWeightAutInvariant : MatchingWeight → Set
MatchingWeightAutInvariant w =
  (σ : MatchingPermutation) (i : MatchingIndex) →
  w (apply-matching-perm σ i) ≡ w i

-- $ aut-invariant-weight-is-uniform.
aut-invariant-weight-is-uniform :
  (w : MatchingWeight) →
  MatchingWeightAutInvariant w →
  (w matching-one ≡ w matching-two) ×
  (w matching-two ≡ w matching-three)
aut-invariant-weight-is-uniform w inv =
  sym (inv swap-one-two matching-one) ,
  sym (inv swap-two-three matching-two)

-- $ constant-weight.
constant-weight : ℕ → MatchingWeight
constant-weight k _ = k

```

```

constant-weight-is-aut-invariant :
  (k : ℕ) → MatchingWeightAutInvariant (constant-weight k)
constant-weight-is-aut-invariant k swap-one-two    matching-one = refl
constant-weight-is-aut-invariant k swap-one-two    matching-two = refl
constant-weight-is-aut-invariant k swap-one-two    matching-three = refl
constant-weight-is-aut-invariant k swap-two-three matching-one   = refl
constant-weight-is-aut-invariant k swap-two-three matching-two   = refl
constant-weight-is-aut-invariant k swap-two-three matching-three = refl

-- $ aut-invariant-weight-equals-its-first-value.
aut-invariant-weight-equals-its-first-value :
  (w : MatchingWeight) →
  MatchingWeightAutInvariant w →
  (i : MatchingIndex) → w i ≡ w matching-one
aut-invariant-weight-equals-its-first-value w inv matching-one = refl
aut-invariant-weight-equals-its-first-value w inv matching-two =
  sym (sym (inv swap-one-two matching-one))
aut-invariant-weight-equals-its-first-value w inv matching-three =
  trans
  (sym (sym (inv swap-two-three matching-two)))
  (sym (sym (inv swap-one-two matching-one)))

record AutInvariantMatchingWeightTheorem : Set where
  field
  classification :
    (w : MatchingWeight) →
    MatchingWeightAutInvariant w →
    (w matching-one ≡ w matching-two) ×
    (w matching-two ≡ w matching-three)
  constants-are-invariant :
    (k : ℕ) → MatchingWeightAutInvariant (constant-weight k)
  invariant-equals-first-value :
    (w : MatchingWeight) →
    MatchingWeightAutInvariant w →
    (i : MatchingIndex) → w i ≡ w matching-one

aut-invariant-matching-weight-theorem :
  AutInvariantMatchingWeightTheorem
aut-invariant-matching-weight-theorem = record
{ classification      = aut-invariant-weight-is-uniform
; constants-are-invariant = constant-weight-is-aut-invariant
; invariant-equals-first-value =
  aut-invariant-weight-equals-its-first-value
}

```

15 Vertex permutations from $\text{Aut}(K_4)$ as the source of the two matching generators

The previous block proved the classification on `MatchingPermutation`, an abstract two-element generator family. The block below grounds that family in $\text{Aut}(K_4)$. We introduce the four vertices of K_4 , the six edges, the membership of each edge in a perfect matching, two specific vertex transpositions $(v_1 \ v_2)$ and $(v_2 \ v_3)$ together with proofs that they are involutions and hence bijections, and the generic induced action of an arbitrary vertex permutation on matchings via its edge image. We then check by case analysis that the induced action of $(v_1 \ v_2)$ coincides with the abstract generator `swap-two-three` and that the induced action of $(v_2 \ v_3)$ coincides with `swap-one-two`. Composing with the previous classification theorem yields the master statement: every weight on the three matchings that is invariant under the two named vertex transpositions is constant. Since K_4 is the complete graph on four vertices, every vertex bijection is an automorphism of K_4 , so the two specific transpositions are genuine elements of $\text{Aut}(K_4)$.

```

-- $ Vertex.
data Vertex : Set where

```

```

v1 : Vertex
v2 : Vertex
v3 : Vertex
v4 : Vertex

data K4Edge : Set where
e12 : K4Edge
e13 : K4Edge
e14 : K4Edge
e23 : K4Edge
e24 : K4Edge
e34 : K4Edge

-- $ edge-of.
edge-of : Vertex → Vertex → K4Edge
edge-of v1 v1 = e12
edge-of v1 v2 = e12
edge-of v1 v3 = e13
edge-of v1 v4 = e14
edge-of v2 v1 = e12
edge-of v2 v2 = e12
edge-of v2 v3 = e23
edge-of v2 v4 = e24
edge-of v3 v1 = e13
edge-of v3 v2 = e23
edge-of v3 v3 = e12
edge-of v3 v4 = e34
edge-of v4 v1 = e14
edge-of v4 v2 = e24
edge-of v4 v3 = e34
edge-of v4 v4 = e12

-- $ Each edge belongs to exactly one perfect matching of K4.
matching-of-edge : K4Edge → MatchingIndex
matching-of-edge e12 = matching-one
matching-of-edge e34 = matching-one
matching-of-edge e13 = matching-two
matching-of-edge e24 = matching-two
matching-of-edge e14 = matching-three
matching-of-edge e23 = matching-three

```

Two concrete vertex transpositions generate the matching action used by the uniformity theorem. They are recorded as involutive bijections before their effect on matchings is computed.

```

-- $ swap-v1-v2.
swap-v1-v2 : Vertex → Vertex
swap-v1-v2 v1 = v2
swap-v1-v2 v2 = v1
swap-v1-v2 v3 = v3
swap-v1-v2 v4 = v4

swap-v2-v3 : Vertex → Vertex
swap-v2-v3 v1 = v1
swap-v2-v3 v2 = v3
swap-v2-v3 v3 = v2
swap-v2-v3 v4 = v4

swap-v1-v2-involutive :
(v : Vertex) → swap-v1-v2 (swap-v1-v2 v) ≡ v
swap-v1-v2-involutive v1 = refl
swap-v1-v2-involutive v2 = refl
swap-v1-v2-involutive v3 = refl
swap-v1-v2-involutive v4 = refl

swap-v2-v3-involutive :
(v : Vertex) → swap-v2-v3 (swap-v2-v3 v) ≡ v
swap-v2-v3-involutive v1 = refl
swap-v2-v3-involutive v2 = refl
swap-v2-v3-involutive v3 = refl

```



```

swap-v2-v3-involutive v4 = refl

-- $ VertexBijection.
record VertexBijection : Set where
field
  forward  : Vertex → Vertex
  backward : Vertex → Vertex
  forward-backward : (v : Vertex) → forward (backward v) ≡ v
  backward-forward : (v : Vertex) → backward (forward v) ≡ v

swap-v1-v2-bijection : VertexBijection
swap-v1-v2-bijection = record
{ forward  = swap-v1-v2
; backward = swap-v1-v2
; forward-backward = swap-v1-v2-involutive
; backward-forward = swap-v1-v2-involutive
}

swap-v2-v3-bijection : VertexBijection
swap-v2-v3-bijection = record
{ forward  = swap-v2-v3
; backward = swap-v2-v3
; forward-backward = swap-v2-v3-involutive
; backward-forward = swap-v2-v3-involutive
}

```

The induced action on matchings is then computed from the image of a representative edge. This connects the vertex automorphisms to the abstract matching permutations used in the previous theorem.

```

-- $ induced-on-matchings.
induced-on-matchings :
(Vertex → Vertex) → MatchingIndex → MatchingIndex
induced-on-matchings σ matching-one =
  matching-of-edge (edge-of (σ v1) (σ v2))
induced-on-matchings σ matching-two =
  matching-of-edge (edge-of (σ v1) (σ v3))
induced-on-matchings σ matching-three =
  matching-of-edge (edge-of (σ v1) (σ v4))

-- $ swap-v1-v2-induces-swap-two-three.
swap-v1-v2-induces-swap-two-three :
(i : MatchingIndex) →
induced-on-matchings swap-v1-v2 i ≡
  apply-matching-perm swap-two-three i
swap-v1-v2-induces-swap-two-three matching-one = refl
swap-v1-v2-induces-swap-two-three matching-two = refl
swap-v1-v2-induces-swap-two-three matching-three = refl

swap-v2-v3-induces-swap-one-two :
(i : MatchingIndex) →
induced-on-matchings swap-v2-v3 i ≡
  apply-matching-perm swap-one-two i
swap-v2-v3-induces-swap-one-two matching-one = refl
swap-v2-v3-induces-swap-one-two matching-two = refl
swap-v2-v3-induces-swap-one-two matching-three = refl

```

Now the matching-weight classification is lifted from the abstract matching generators to the two concrete vertex transpositions. This is the bridge from K_4 automorphisms to matching-uniform weights.

```

-- $ MatchingWeightVertexAutInvariant.
MatchingWeightVertexAutInvariant : MatchingWeight → Set
MatchingWeightVertexAutInvariant w =
((i : MatchingIndex) →
  w (induced-on-matchings swap-v1-v2 i) ≡ w i) ×
((i : MatchingIndex) →
  w (induced-on-matchings swap-v2-v3 i) ≡ w i)

```

vertex-aut-invariance-implies-matching-aut-invariance :

```
(w : MatchingWeight) →
MatchingWeightVertexAutInvariant w →
MatchingWeightAutInvariant w
vertex-aut-invariance-implies-matching-aut-invariance
w (inv12 , inv23) swap-one-two i =
  trans
    (cong w (sym (swap-v2-v3-induces-swap-one-two i)))
    (inv23 i)
```

vertex-aut-invariance-implies-matching-aut-invariance

```
w (inv12 , inv23) swap-two-three i =
  trans
    (cong w (sym (swap-v1-v2-induces-swap-two-three i)))
    (inv12 i)
```

vertex-aut-invariant-weight-is-uniform :

```
(w : MatchingWeight) →
MatchingWeightVertexAutInvariant w →
(w matching-one ≡ w matching-two) ×
(w matching-two ≡ w matching-three)
vertex-aut-invariant-weight-is-uniform w hyp =
aut-invariant-weight-is-uniform w
(vertex-aut-invariance-implies-matching-aut-invariance w hyp)
```

vertex-aut-invariant-weight-equals-its-first-value :

```
(w : MatchingWeight) →
MatchingWeightVertexAutInvariant w →
(i : MatchingIndex) → w i ≡ w matching-one
vertex-aut-invariant-weight-equals-its-first-value w hyp =
aut-invariant-weight-equals-its-first-value w
(vertex-aut-invariance-implies-matching-aut-invariance w hyp)
```

constant-weight-is-vertex-aut-invariant :

```
(k : ℕ) → MatchingWeightVertexAutInvariant (constant-weight k)
constant-weight-is-vertex-aut-invariant k =
(λ _ → refl) , (λ _ → refl)
```

record VertexAutMatchingWeightTheorem : Set where
field

```
swap12-bijection :
VertexBijection
swap23-bijection :
VertexBijection
swap12-induces-swap-two-three :
(i : MatchingIndex) →
induced-on-matchings swap-v1-v2 i ≡
apply-matching-perm swap-two-three i
swap23-induces-swap-one-two :
(i : MatchingIndex) →
induced-on-matchings swap-v2-v3 i ≡
apply-matching-perm swap-one-two i
classification-on-open-space :
(w : MatchingWeight) →
MatchingWeightVertexAutInvariant w →
(w matching-one ≡ w matching-two) ×
(w matching-two ≡ w matching-three)
constants-are-vertex-invariant :
(k : ℕ) →
MatchingWeightVertexAutInvariant (constant-weight k)
```

vertex-aut-matching-weight-theorem :

VertexAutMatchingWeightTheorem

vertex-aut-matching-weight-theorem = record

```
{ swap12-bijection      = swap-v1-v2-bijection
; swap23-bijection      = swap-v2-v3-bijection
; swap12-induces-swap-two-three = swap-v1-v2-induces-swap-two-three
; swap23-induces-swap-one-two  = swap-v2-v3-induces-swap-one-two
; classification-on-open-space = vertex-aut-invariant-weight-is-uniform
; constants-are-vertex-invariant = constant-weight-is-vertex-aut-invariant
}
```

The last step verifies that the two named vertex bijections are genuine graph automorphisms of K_4 . Since K_4 is complete, preserving distinctness is enough to preserve adjacency.

```
-- $ NotEqualVertex.
data NotEqualVertex : Vertex → Vertex → Set where
ne-v1-v2 : NotEqualVertex v1 v2
ne-v1-v3 : NotEqualVertex v1 v3
ne-v1-v4 : NotEqualVertex v1 v4
ne-v2-v1 : NotEqualVertex v2 v1
ne-v2-v3 : NotEqualVertex v2 v3
ne-v2-v4 : NotEqualVertex v2 v4
ne-v3-v1 : NotEqualVertex v3 v1
ne-v3-v2 : NotEqualVertex v3 v2
ne-v3-v4 : NotEqualVertex v3 v4
ne-v4-v1 : NotEqualVertex v4 v1
ne-v4-v2 : NotEqualVertex v4 v2
ne-v4-v3 : NotEqualVertex v4 v3

K4Adjacent : Vertex → Vertex → Set
K4Adjacent = NotEqualVertex

swap-v1-v2-preserves-adjacency :
  (u v : Vertex) → K4Adjacent u v →
  K4Adjacent (swap-v1-v2 u) (swap-v1-v2 v)
swap-v1-v2-preserves-adjacency v1 v2 ne-v1-v2 = ne-v2-v1
swap-v1-v2-preserves-adjacency v1 v3 ne-v1-v3 = ne-v2-v3
swap-v1-v2-preserves-adjacency v1 v4 ne-v1-v4 = ne-v2-v4
swap-v1-v2-preserves-adjacency v2 v1 ne-v2-v1 = ne-v1-v2
swap-v1-v2-preserves-adjacency v2 v3 ne-v2-v3 = ne-v1-v3
swap-v1-v2-preserves-adjacency v2 v4 ne-v2-v4 = ne-v1-v4
swap-v1-v2-preserves-adjacency v3 v1 ne-v3-v1 = ne-v3-v2
swap-v1-v2-preserves-adjacency v3 v2 ne-v3-v2 = ne-v3-v1
swap-v1-v2-preserves-adjacency v3 v4 ne-v3-v4 = ne-v3-v4
swap-v1-v2-preserves-adjacency v4 v1 ne-v4-v1 = ne-v4-v2
swap-v1-v2-preserves-adjacency v4 v2 ne-v4-v2 = ne-v4-v1
swap-v1-v2-preserves-adjacency v4 v3 ne-v4-v3 = ne-v4-v3

swap-v2-v3-preserves-adjacency :
  (u v : Vertex) → K4Adjacent u v →
  K4Adjacent (swap-v2-v3 u) (swap-v2-v3 v)
swap-v2-v3-preserves-adjacency v1 v2 ne-v1-v2 = ne-v1-v3
swap-v2-v3-preserves-adjacency v1 v3 ne-v1-v3 = ne-v1-v2
swap-v2-v3-preserves-adjacency v1 v4 ne-v1-v4 = ne-v1-v4
swap-v2-v3-preserves-adjacency v2 v1 ne-v2-v1 = ne-v3-v1
swap-v2-v3-preserves-adjacency v2 v3 ne-v2-v3 = ne-v3-v2
swap-v2-v3-preserves-adjacency v2 v4 ne-v2-v4 = ne-v3-v4
swap-v2-v3-preserves-adjacency v3 v1 ne-v3-v1 = ne-v2-v1
swap-v2-v3-preserves-adjacency v3 v2 ne-v3-v2 = ne-v2-v3
swap-v2-v3-preserves-adjacency v3 v4 ne-v3-v4 = ne-v2-v4
swap-v2-v3-preserves-adjacency v4 v1 ne-v4-v1 = ne-v4-v1
swap-v2-v3-preserves-adjacency v4 v2 ne-v4-v2 = ne-v4-v3
swap-v2-v3-preserves-adjacency v4 v3 ne-v4-v3 = ne-v4-v2

record K4GraphAutomorphism : Set where
field
  underlying-bijection : VertexBijection
  preserves-adjacency :
    (u v : Vertex) → K4Adjacent u v →
    K4Adjacent
      (VertexBijection.forward underlying-bijection u)
      (VertexBijection.forward underlying-bijection v)

swap-v1-v2-as-graph-automorphism : K4GraphAutomorphism
swap-v1-v2-as-graph-automorphism = record
{ underlying-bijection = swap-v1-v2-bijection
; preserves-adjacency = swap-v1-v2-preserves-adjacency
}

swap-v2-v3-as-graph-automorphism : K4GraphAutomorphism
swap-v2-v3-as-graph-automorphism = record
```

```

{ underlying-bijection = swap-v2-v3-bijection
; preserves-adjacency = swap-v2-v3-preserves-adjacency
}

```

The matching-level invariance used inside the previous classification is therefore not a stipulation: it is the consequence of invariance under two concrete vertex transpositions, each given as a **VertexBijection**, with their induced action on matchings computed from the vertex permutation, the edge structure and the matching membership table. The classification of matching-uniform weights is now reduced to: *any matching weight invariant under $(v_1 \ v_2)$ and $(v_2 \ v_3)$ is constant*. Combined with the separator filter, the bare/running/tower uniformity locks and the absence of any K_4 -intrinsic mechanism that breaks vertex-level invariance, the bare structural ratio middle/heaviest = $E = 6$ is forced through every K_4 -intrinsic, $\text{Aut}(K_4)$ -invariant layer of the schema.

This is a real classification theorem on an open function space, not a table over a finite list of named patterns. The earlier finite table is therefore now backed by an underlying mathematical statement: the constraint *$\text{Aut}(K_4)$ -invariant on the three perfect matchings* is preserved by the constant weights and by no other. In particular, no K_4 -intrinsic, $\text{Aut}(K_4)$ -invariant multiplicative reweighting of the matchings can produce a matching-differential factor between m_2 and m_3 . The bare structural ratio middle / heaviest = $E = 6$ is therefore forced not just on the named candidates but on every weight function in $\mathbb{N}^{\text{Matching}}$ that respects the automorphism action.

16 Neutral residual carrier classification

The matching theorem closes the tempting but wrong local move. A neutral residual that changes the splitting ratio cannot be carried by matching weights, because any K_4 -intrinsic matching weight that respects the vertex automorphisms is uniform across the three perfect matchings. The residual carrier must therefore act one layer above the matching index.

The finite candidate space has exactly three entries. There may be no residual carrier; the residual may be placed directly on the matchings; or the residual may be carried by the already available coupling complement. The four filters are the closure contract of this section: the carrier must be splitting-asymmetric, must preserve the $\text{Aut}(K_4)$ -uniformity of matching weights, must use a ledger complement already present, and must introduce no empirical patch. Under those filters only the coupling-complement carrier survives. This is the formal closure of the carrier slot; the physical reading as a neutrino splitting correction remains an empirical test.

```

-- $ NeutralResidualCarrier.
data NeutralResidualCarrier : Set where
no-residual      : NeutralResidualCarrier
matching-residual : NeutralResidualCarrier
coupling-complement-residual : NeutralResidualCarrier

ResidualCarrierSplittingAsymmetric : NeutralResidualCarrier → Set
ResidualCarrierSplittingAsymmetric no-residual      = ⊥
ResidualCarrierSplittingAsymmetric matching-residual = ⊤
ResidualCarrierSplittingAsymmetric coupling-complement-residual = ⊤

PreservesMatchingAutUniformity : NeutralResidualCarrier → Set
PreservesMatchingAutUniformity no-residual      = ⊤
PreservesMatchingAutUniformity matching-residual = ⊥
PreservesMatchingAutUniformity coupling-complement-residual = ⊤

UsesExistingLedgerComplement : NeutralResidualCarrier → Set
UsesExistingLedgerComplement no-residual      = ⊥
UsesExistingLedgerComplement matching-residual = ⊥
UsesExistingLedgerComplement coupling-complement-residual = ⊤

```

```

NoEmpiricalResidualPatch : NeutralResidualCarrier → Set
NoEmpiricalResidualPatch no-residual      = ⊤
NoEmpiricalResidualPatch matching-residual = ⊤
NoEmpiricalResidualPatch coupling-complement-residual = ⊤

NeutralResidualCarrierAdmissible : NeutralResidualCarrier → Set
NeutralResidualCarrierAdmissible carrier =
  ResidualCarrierSplittingAsymmetric carrier ×
  (PreservesMatchingAutUniformity carrier ×
   (UsesExistingLedgerComplement carrier ×
    NoEmpiricalResidualPatch carrier))

neutral-residual-carrier-admissible :
  NeutralResidualCarrierAdmissible coupling-complement-residual
neutral-residual-carrier-admissible = tt , (tt , (tt , tt))

neutral-residual-carrier-unique :
  (carrier : NeutralResidualCarrier) →
  NeutralResidualCarrierAdmissible carrier →
  carrier ≡ coupling-complement-residual
neutral-residual-carrier-unique no-residual (() , _)
neutral-residual-carrier-unique matching-residual (_, (() , _))
neutral-residual-carrier-unique coupling-complement-residual _ = refl

ActsAboveMatchingLevel : NeutralResidualCarrier → Set
ActsAboveMatchingLevel no-residual      = ⊥
ActsAboveMatchingLevel matching-residual = ⊥
ActsAboveMatchingLevel coupling-complement-residual = ⊤

neutral-residual-acts-above-matching :
  ActsAboveMatchingLevel coupling-complement-residual
neutral-residual-acts-above-matching = tt

```

The surviving carrier is quantified by the coupling complement. The ledger splits 306 into the active loop part 11 and its complement 295.

```

-- $ CouplingComplementLedger.
record CouplingComplementLedger : Set where
  field
    coupling-total      : ℕ
    active-loop-part    : ℕ
    complement-part      : ℕ
    coupling-total-is-306 : coupling-total ≡ 306
    active-loop-is-ledger-loop :
      active-loop-part ≡ loopNumerator canonicalK4Ledger
    active-loop-is-11 : active-loop-part ≡ 11
    complement-is-295 : complement-part ≡ 295
    total-splits-as-loop-plus-complement :
      coupling-total ≡ active-loop-part + complement-part
    complement-by-subtraction :
      coupling-total ÷ active-loop-part ≡ complement-part

neutral-coupling-complement-ledger : CouplingComplementLedger
neutral-coupling-complement-ledger = record
{ coupling-total      = 306
; active-loop-part    = loopNumerator canonicalK4Ledger
; complement-part      = 295
; coupling-total-is-306 = refl
; active-loop-is-ledger-loop = refl
; active-loop-is-11 = refl
; complement-is-295 = refl
; total-splits-as-loop-plus-complement = refl
; complement-by-subtraction = refl
}

neutral-residual-ratio : PlanckRatio
neutral-residual-ratio = record
{ numerator =

```

```

    CouplingComplementLedger.complement-part
    neutral-coupling-complement-ledger
; denominator =
    CouplingComplementLedger.coupling-total
    neutral-coupling-complement-ledger
}

neutral-residual-ratio-is-295-over-306 :
PlanckRatio.numerator neutral-residual-ratio ≡ 295 ×
PlanckRatio.denominator neutral-residual-ratio ≡ 306
neutral-residual-ratio-is-295-over-306 = refl , refl

neutral-effective-edge-separator-ratio : PlanckRatio
neutral-effective-edge-separator-ratio = record
{ numerator =
    separator-value separator-edges *
    CouplingComplementLedger.complement-part
    neutral-coupling-complement-ledger
; denominator =
    CouplingComplementLedger.coupling-total
    neutral-coupling-complement-ledger
}

neutral-effective-edge-numerator-is-1770 :
PlanckRatio.numerator neutral-effective-edge-separator-ratio ≡ 1770
neutral-effective-edge-numerator-is-1770 = refl

neutral-effective-edge-denominator-is-306 :
PlanckRatio.denominator neutral-effective-edge-separator-ratio ≡ 306
neutral-effective-edge-denominator-is-306 = refl

```

The carrier classification packages the survivor, the matching theorem, the complement ledger, and the effective separator ratio. The physical reading of that ratio remains explicitly marked as a test.

```

-- $ NeutralResidualCarrierClassification.
record NeutralResidualCarrierClassification : Set where
field
    carrier-survivor          : NeutralResidualCarrier
    carrier-admissible        :
        NeutralResidualCarrierAdmissible carrier-survivor
    carrier-locked            :
    carrier-survivor ≡ coupling-complement-residual
    matching-theorem          : VertexAutMatchingWeightTheorem
    carrier-acts-above-matching :
    ActsAboveMatchingLevel carrier-survivor
    complement-ledger          : CouplingComplementLedger
    residual-ratio              : PlanckRatio
    residual-ratio-numerator    :
    PlanckRatio.numerator residual-ratio ≡ 295
    residual-ratio-denominator :
    PlanckRatio.denominator residual-ratio ≡ 306
    effective-edge-ratio        : PlanckRatio
    effective-edge-numerator    :
    PlanckRatio.numerator effective-edge-ratio ≡ 1770
    effective-edge-denominator :
    PlanckRatio.denominator effective-edge-ratio ≡ 306
    carrier-classification-status : ProgrammeStatus
    carrier-classification-checked-here :
    carrier-classification-status ≡ checked-here
    residual-action-status       : ProgrammeStatus
    residual-action-checked-here :
    residual-action-status ≡ checked-here
    physical-reading-status      : ProgrammeStatus
    physical-reading-is-test     :
    physical-reading-status ≡ physical-test

neutral-residual-carrier-classification :
NeutralResidualCarrierClassification

```

```

neutral-residual-carrier-classification = record
{
  carrier-survivor           = coupling-complement-residual
  ; carrier-admissible       = neutral-residual-carrier-admissible
  ; carrier-locked           = refl
  ; matching-theorem         = vertex-aut-matching-weight-theorem
  ; carrier-acts-above-matching = tt
  ; complement-ledger         = neutral-coupling-complement-ledger
  ; residual-ratio           = neutral-residual-ratio
  ; residual-ratio-numerator = refl
  ; residual-ratio-denominator = refl
  ; effective-edge-ratio     = neutral-effective-edge-separator-ratio
  ; effective-edge-numerator = refl
  ; effective-edge-denominator = refl
  ; carrier-classification-status = checked-here
  ; carrier-classification-checked-here = refl
  ; residual-action-status     = checked-here
  ; residual-action-checked-here = refl
  ; physical-reading-status    = physical-test
  ; physical-reading-is-test   = refl
}

```

17 Final residual-corrected neutrino prediction

The tower-corrected Planck bridge fixes the heaviest neutral ratio. The neutral residual carrier does not change that scale; it changes the separator between the two nonzero neutral masses. The old edge separator $E = 6$ is replaced at the final neutral readout by

$$E_\nu = 6 \cdot \frac{295}{306} = \frac{1770}{306}.$$

Thus

$$\frac{m_3}{m_2} = \frac{1770}{306}, \quad m_2 = m_3 \cdot \frac{306}{1770}.$$

Using the same Planck input as the tower-corrected table, the final normal-hierarchy prediction is

$$m_1 = 0, \quad m_2 = 0.008589080150 \text{ eV}, \quad m_3 = 0.049681934200 \text{ eV},$$

with

$$\sum_i m_i = 0.058271014350 \text{ eV},$$

and

$$\Delta m_{21}^2 = 7.3772297820 \cdot 10^{-5} \text{ eV}^2, \quad \Delta m_{31}^2 = 2.4682945859 \cdot 10^{-3} \text{ eV}^2.$$

Equivalently, the squared-splitting ratio is approximately 33.45828527. These decimals are the physical readout of the exact Planck ratios below; they are not used as filters.

```

-- $ residual-corrected-middle-neutral-planck-ratio.
residual-corrected-middle-neutral-planck-ratio : PlanckRatio
residual-corrected-middle-neutral-planck-ratio = record
{
  numerator =
    tower-corrected-electron-planck-numerator *
    PlanckRatio.denominator neutral-effective-edge-separator-ratio
  ; denominator =
    tower-corrected-heaviest-neutral-planck-denominator *
    PlanckRatio.numerator neutral-effective-edge-separator-ratio
}

residual-corrected-normal-hierarchy-planck-sum : PlanckRatio
residual-corrected-normal-hierarchy-planck-sum = record
{
  numerator =

```

```

tower-corrected-electron-planck-numerator *
(PlanckRatio.numerator neutral-effective-edge-separator-ratio +
 PlanckRatio.denominator neutral-effective-edge-separator-ratio)
; denominator =
tower-corrected-heaviest-neutral-planck-denominator *
PlanckRatio.numerator neutral-effective-edge-separator-ratio
}

residual-corrected-delta21-planck-ratio : PlanckRatio
residual-corrected-delta21-planck-ratio =
square-planck-ratio residual-corrected-middle-neutral-planck-ratio

residual-corrected-delta31-planck-ratio : PlanckRatio
residual-corrected-delta31-planck-ratio =
square-planck-ratio tower-corrected-heaviest-neutral-planck-ratio

residual-corrected-middle-numerator-is-388662168766119520831183926 :
PlanckRatio.numerator residual-corrected-middle-neutral-planck-ratio
= 388662168766119520831183926
residual-corrected-middle-numerator-is-388662168766119520831183926 = refl

residual-corrected-middle-denominator-is-552461668319475518343179050761041844482566504764565094400 :
PlanckRatio.denominator residual-corrected-middle-neutral-planck-ratio
= 552461668319475518343179050761041844482566504764565094400
residual-corrected-middle-denominator-is-552461668319475518343179050761041844482566504764565094400 = refl

residual-corrected-sum-numerator-is-2636806086138771651129208596 :
PlanckRatio.numerator residual-corrected-normal-hierarchy-planck-sum
= 2636806086138771651129208596
residual-corrected-sum-numerator-is-2636806086138771651129208596 = refl

residual-corrected-sum-denominator-is-552461668319475518343179050761041844482566504764565094400 :
PlanckRatio.denominator residual-corrected-normal-hierarchy-planck-sum
= 552461668319475518343179050761041844482566504764565094400
residual-corrected-sum-denominator-is-552461668319475518343179050761041844482566504764565094400 = refl

record ResidualCorrectedNormalHierarchyPrediction : Set where
field
final-carrier-classification : NeutralResidualCarrierClassification
final-effective-separator : PlanckRatio
final-lightest-ratio : PlanckRatio
final-middle-ratio : PlanckRatio
final-heaviest-ratio : PlanckRatio
final-sum-ratio : PlanckRatio
final-delta21-ratio : PlanckRatio
final-delta31-ratio : PlanckRatio
final-lightest-zero :
PlanckRatio.numerator final-lightest-ratio = 0
final-separator-numerator :
PlanckRatio.numerator final-effective-separator = 1770
final-separator-denominator :
PlanckRatio.denominator final-effective-separator = 306
final-middle-numerator :
PlanckRatio.numerator final-middle-ratio
= 388662168766119520831183926
final-middle-denominator :
PlanckRatio.denominator final-middle-ratio
= 552461668319475518343179050761041844482566504764565094400
final-heaviest-locked :
final-heaviest-ratio = tower-corrected-heaviest-neutral-planck-ratio
final-sum-numerator :
PlanckRatio.numerator final-sum-ratio
= 2636806086138771651129208596
final-sum-denominator :
PlanckRatio.denominator final-sum-ratio
= 552461668319475518343179050761041844482566504764565094400
final-delta21-from-middle :
final-delta21-ratio =
square-planck-ratio residual-corrected-middle-neutral-planck-ratio
final-delta31-from-heaviest :
final-delta31-ratio =

```


square-planck-ratio tower-corrected-heaviest-neutral-planck-ratio

```

residual-corrected-normal-hierarchy-prediction :
ResidualCorrectedNormalHierarchyPrediction
residual-corrected-normal-hierarchy-prediction = record
{ final-carrier-classification = neutral-residual-carrier-classification
; final-effective-separator    = neutral-effective-edge-separator-ratio
; final-lightest-ratio         = record { numerator = 0 ; denominator = 1 }
; final-middle-ratio           = residual-corrected-middle-neutral-planck-ratio
; final-heaviest-ratio         = tower-corrected-heaviest-neutral-planck-ratio
; final-sum-ratio              = residual-corrected-normal-hierarchy-planck-sum
; final-delta21-ratio          = residual-corrected-delta21-planck-ratio
; final-delta31-ratio          = residual-corrected-delta31-planck-ratio
; final-lightest-zero          = refl
; final-separator-numerator    = refl
; final-separator-denominator = refl
; final-middle-numerator       = refl
; final-middle-denominator     = refl
; final-heaviest-locked        = refl
; final-sum-numerator          = refl
; final-sum-denominator       = refl
; final-delta21-from-middle    = refl
; final-delta31-from-heaviest  = refl
}

```

The neutral sector occupies a separate but parallel structure. The two observed splittings Δm_{21}^2 and Δm_{31}^2 are labelled by ordered pairs of generation indices on the leptonic side. The cell table predicts a third splitting Δm_{32}^2 , which is not an independent observable; it is fixed by the other two as $\Delta m_{31}^2 - \Delta m_{21}^2$. This redundancy is recorded formally so that the cell table cannot smuggle in an extra free parameter.

```

-- $ SplittingPair.
data SplittingPair : Set where
splitting-21 : SplittingPair
splitting-31 : SplittingPair
splitting-32 : SplittingPair

record NeutralSplittingClosure : Set where
field
independent-pairs      : ℕ
derived-pairs          : ℕ
independent-pairs-is-two : independent-pairs ≡ 2
derived-pairs-is-one   : derived-pairs ≡ 1
total-pairs-is-three   : independent-pairs + derived-pairs ≡ 3
redundancy-status      : ProgrammeStatus
redundancy-checked-here : redundancy-status ≡ checked-here

neutral-splitting-closure : NeutralSplittingClosure
neutral-splitting-closure = record
{ independent-pairs      = 2
; derived-pairs          = 1
; independent-pairs-is-two = refl
; derived-pairs-is-one   = refl
; total-pairs-is-three   = refl
; redundancy-status      = checked-here
; redundancy-checked-here = refl
}

```

18 Charge duality and the elementary slot count

The cell table classified above carries a side index and a matching index. Inside each cell a third refinement is forced by the material already fixed above. The neutral-depth survivor lives on the neutral side of the cell, and the charged-sector ratios live on the charged side. Without a charged-versus-neutral split inside each cell, the neutral-depth carrier and the charged-mass carrier would

collide in the same slot. The split is therefore not an addition; it is the formal consequence of keeping both surviving carriers separate.

The candidate space below names the obvious alternatives. A trivial duality has only one charge state and erases the neutral-depth carrier. A foreign $U(1)$ adds a continuous structure not visible in K_4 and is excluded by the same discipline that excluded a foreign group in the previous section. Only the intrinsic $\mathbf{Z}_2$ charge duality survives.

```
-- $ ChargeDualityCandidate.
data ChargeDualityCandidate : Set where
  trivial-charge-duality : ChargeDualityCandidate
  foreign-u1-charge      : ChargeDualityCandidate
  intrinsic-z2-charge    : ChargeDualityCandidate

NotTrivialDuality : ChargeDualityCandidate → Set
NotTrivialDuality trivial-charge-duality = ⊥
NotTrivialDuality foreign-u1-charge      = ⊤
NotTrivialDuality intrinsic-z2-charge    = ⊤

NoForeignContinuousGroup : ChargeDualityCandidate → Set
NoForeignContinuousGroup trivial-charge-duality = ⊤
NoForeignContinuousGroup foreign-u1-charge      = ⊥
NoForeignContinuousGroup intrinsic-z2-charge    = ⊤

KeepsNeutralDepthCarrier : ChargeDualityCandidate → Set
KeepsNeutralDepthCarrier trivial-charge-duality = ⊥
KeepsNeutralDepthCarrier foreign-u1-charge      = ⊤
KeepsNeutralDepthCarrier intrinsic-z2-charge    = ⊤

ChargeDualityAdmissible : ChargeDualityCandidate → Set
ChargeDualityAdmissible k =
  NotTrivialDuality k ×
  (NoForeignContinuousGroup k × KeepsNeutralDepthCarrier k)

charge-duality-admissible :
  ChargeDualityAdmissible intrinsic-z2-charge
charge-duality-admissible = tt , (tt , tt)

charge-duality-unique :
  (k : ChargeDualityCandidate) →
  ChargeDualityAdmissible k →
  k ≡ intrinsic-z2-charge
charge-duality-unique trivial-charge-duality ((), _)
charge-duality-unique foreign-u1-charge      (_, ((), _))
charge-duality-unique intrinsic-z2-charge _ = refl
```

The elementary slot count is then the product of the three closed finite refinements: side count, matching count, and charge-duality count. Each factor is fixed by its own classification above. The product is locked at twelve.

```
-- $ ChargeState.
data ChargeState : Set where
  charged : ChargeState
  neutral : ChargeState

data ElementarySlot : Set where
  slot : BipartitionSide → MatchingIndex → ChargeState → ElementarySlot

side-count : ℕ
side-count = 2

matching-count-elementary : ℕ
matching-count-elementary = 3

charge-state-count : ℕ
charge-state-count = 2
```

```

elementary-slot-count : ℕ
elementary-slot-count =
  side-count * matching-count-elementary * charge-state-count

elementary-slot-count-is-twelve : elementary-slot-count ≡ 12
elementary-slot-count-is-twelve = refl

elementary-slot-count-factorisation :
  elementary-slot-count
  ≡ side-count * matching-count-elementary * charge-state-count
elementary-slot-count-factorisation = refl

```

The closure record packages the three factor counts, the locked product, and the surviving charge-duality candidate. The split into charged and neutral content per cell is named explicitly so that any any assignment must respect it. The hadronic and leptonic identifications of individual elementary slots remain physical readings.

```

-- $ ElementarySlotClosure.
record ElementarySlotClosure : Set where
  field
    duality-survivor          : ChargeDualityCandidate
    duality-admissible        :
      ChargeDualityAdmissible duality-survivor
    duality-locked            :
      duality-survivor ≡ intrinsic-z2-charge
    side-factor                : ℕ
    matching-factor            : ℕ
    charge-factor              : ℕ
    side-factor-is-two        : side-factor ≡ 2
    matching-factor-is-three   : matching-factor ≡ 3
    charge-factor-is-two       : charge-factor ≡ 2
    elementary-product         : ℕ
    elementary-product-is-twelve : elementary-product ≡ 12
    elementary-product-locked :
      elementary-product ≡ side-factor * matching-factor * charge-factor
    slot-assignment-status     : ProgrammeStatus
    slot-assignment-physical   :
      slot-assignment-status ≡ physical-test

elementary-slot-closure : ElementarySlotClosure
elementary-slot-closure = record
{
  duality-survivor          = intrinsic-z2-charge
; duality-admissible        = charge-duality-admissible
; duality-locked            = refl
; side-factor                = 2
; matching-factor            = 3
; charge-factor              = 2
; side-factor-is-two        = refl
; matching-factor-is-three   = refl
; charge-factor-is-two       = refl
; elementary-product         = 12
; elementary-product-is-twelve = refl
; elementary-product-locked = refl
; slot-assignment-status     = physical-test
; slot-assignment-physical   = refl
}

```

19 Slot-fill completeness

The elementary slot structure is finite by construction. A separate question is whether the structure is allowed to leave slots empty. An empty slot would mean that a side, matching and charge state combination is admitted by the classification but assigned no physical content. In the present finite scheme this is forbidden: every admitted combination arises from a forced refinement, and a forced refinement that produces an unfilled cell would re-introduce a free choice through the back door.

The completeness predicate below records this as a single classification with two candidates and a unique survivor.

```
-- $ Slot-fill policy: admitted structural slots must not remain empty.
data SlotFillPolicy : Set where
  allows-empty-slots : SlotFillPolicy
  fills-all-slots : SlotFillPolicy

NoEmptySlots : SlotFillPolicy → Set
NoEmptySlots allows-empty-slots = ⊥
NoEmptySlots fills-all-slots = ⊤

slot-fill-policy-survivor : SlotFillPolicy
slot-fill-policy-survivor = fills-all-slots

slot-fill-policy-admissible : NoEmptySlots slot-fill-policy-survivor
slot-fill-policy-admissible = tt

slot-fill-policy-unique :
  (p : SlotFillPolicy) →
  NoEmptySlots p →
  p ≡ fills-all-slots
slot-fill-policy-unique allows-empty-slots ()
slot-fill-policy-unique fills-all-slots _ = refl

record SlotFillCompleteness : Set where
  field
    policy          : SlotFillPolicy
    policy-admissible : NoEmptySlots policy
    policy-locked    : policy ≡ fills-all-slots
    expected-content-count : ℕ
    expected-content-count-is-twelve : expected-content-count ≡ 12

slot-fill-completeness : SlotFillCompleteness
slot-fill-completeness = record
{ policy          = fills-all-slots
; policy-admissible = slot-fill-policy-admissible
; policy-locked    = refl
; expected-content-count = 12
; expected-content-count-is-twelve = refl
}
```

20 Spectral signature of K_4 : a structural source for $1 + 3$

The adjacency operator of K_4 has the form $A = J - I$, where J is the all-ones matrix on four vertices. Its spectrum is fixed by graph data alone: the trace vanishes (no self-loops), the trace of A^2 equals $2|E| = 12$ (twice the edge count), and the Perron eigenvalue equals the regular degree $d = 3$. These three integer constraints force the multiset $\{+3, -1, -1, -1\}$ as the unique spectrum. The combinatorial Laplacian $L = dI - A$ then has the spectrum $\{0, 4, 4, 4\}$: one zero mode (the constant function on the four vertices) and three degenerate positive modes at the value $V = 4$.

Both spectra split the four modes as $1 + 3$ with opposite signs. In the adjacency picture: one positive Perron mode against three degenerate negative modes. In the Laplacian picture: one zero (non-propagating) mode against three degenerate positive modes. This $1 + 3$ split is a theorem of K_4 , not a choice.

The split is the structural source of every $1 + 3$ asymmetry that appears in the physical reading: the time-versus-space split of a $(-, +, +, +)$ Lorentz signature, the distinguished-versus-degenerate character of a Perron mode, and the asymmetric branch of the charged residual classification. The identification of the distinguished spectral mode with the physical name “time” or “hadronic sector”, and of the three degenerate modes with “space” or “leptonic generations”, is a physical

reading and is recorded as `physical-test`. What is closed at the formal level is only the existence and the cardinality of the split.

```
-- $ SpectralEntry.
data SpectralEntry : Set where
  positive-mode : ℕ → SpectralEntry
  negative-mode : ℕ → SpectralEntry
  zero-mode : SpectralEntry

data K4ModelIndex : Set where
  m0 : K4ModelIndex
  m1 : K4ModelIndex
  m2 : K4ModelIndex
  m3 : K4ModelIndex

k4-adjacency-spectrum : K4ModelIndex → SpectralEntry
k4-adjacency-spectrum m0 = positive-mode 3
k4-adjacency-spectrum m1 = negative-mode 1
k4-adjacency-spectrum m2 = negative-mode 1
k4-adjacency-spectrum m3 = negative-mode 1

k4-laplacian-spectrum : K4ModelIndex → SpectralEntry
k4-laplacian-spectrum m0 = zero-mode
k4-laplacian-spectrum m1 = positive-mode 4
k4-laplacian-spectrum m2 = positive-mode 4
k4-laplacian-spectrum m3 = positive-mode 4

magnitude : SpectralEntry → ℕ
magnitude (positive-mode n) = n
magnitude (negative-mode n) = n
magnitude zero-mode = 0

is-positive-entry : SpectralEntry → Set
is-positive-entry (positive-mode _) = ⊤
is-positive-entry (negative-mode _) = ⊥
is-positive-entry zero-mode = ⊥

is-negative-entry : SpectralEntry → Set
is-negative-entry (positive-mode _) = ⊥
is-negative-entry (negative-mode _) = ⊤
is-negative-entry zero-mode = ⊥

is-zero-entry : SpectralEntry → Set
is-zero-entry (positive-mode _) = ⊥
is-zero-entry (negative-mode _) = ⊥
is-zero-entry zero-mode = ⊤

adjacency-positive-magnitude-sum : ℕ
adjacency-positive-magnitude-sum =
  magnitude (k4-adjacency-spectrum m0)

adjacency-negative-magnitude-sum : ℕ
adjacency-negative-magnitude-sum =
  magnitude (k4-adjacency-spectrum m1) +
  magnitude (k4-adjacency-spectrum m2) +
  magnitude (k4-adjacency-spectrum m3)

adjacency-trace-balance :
  adjacency-positive-magnitude-sum ≡ adjacency-negative-magnitude-sum
adjacency-trace-balance = refl

adjacency-sum-of-squares : ℕ
adjacency-sum-of-squares =
  magnitude (k4-adjacency-spectrum m0) *
  magnitude (k4-adjacency-spectrum m0) +
  magnitude (k4-adjacency-spectrum m1) *
  magnitude (k4-adjacency-spectrum m1) +
  magnitude (k4-adjacency-spectrum m2) *
  magnitude (k4-adjacency-spectrum m2) +
  magnitude (k4-adjacency-spectrum m3) *
```

```

magnitude (k4-adjacency-spectrum m3)

adjacency-sum-of-squares-is-twelve :
  adjacency-sum-of-squares  $\equiv$  12
adjacency-sum-of-squares-is-twelve = refl

adjacency-sum-of-squares-equals-twice-edges :
  adjacency-sum-of-squares  $\equiv$  2 * edges canonicalK4Ledge
adjacency-sum-of-squares-equals-twice-edges = refl

adjacency-perron-equals-degree :
  magnitude (k4-adjacency-spectrum m0)  $\equiv$  degree canonicalK4Ledge
adjacency-perron-equals-degree = refl

laplacian-zero-mode-at-m0 :
  is-zero-entry (k4-laplacian-spectrum m0)
laplacian-zero-mode-at-m0 = tt

laplacian-degenerate-magnitude :  $\mathbb{N}$ 
laplacian-degenerate-magnitude = magnitude (k4-laplacian-spectrum m1)

laplacian-degenerate-equals-vertices :
  laplacian-degenerate-magnitude  $\equiv$  vertices canonicalK4Ledge
laplacian-degenerate-equals-vertices = refl

```

The record below collects the spectral facts into the form used by the later physical reading. Only the existence of the 1 + 3 split is formal; names such as time, space, hadronic or leptonic remain outside the theorem.

```

-- $ K4SpectralSignature.
record K4SpectralSignature : Set where
  field
    distinguished-count      :  $\mathbb{N}$ 
    degenerate-count         :  $\mathbb{N}$ 
    distinguished-is-one     : distinguished-count  $\equiv$  1
    degenerate-is-three      : degenerate-count  $\equiv$  3
    total-equals-vertex-count :
      distinguished-count + degenerate-count
       $\equiv$  vertices canonicalK4Ledge
    adjacency-distinguished-positive :
      is-positive-entry (k4-adjacency-spectrum m0)
    adjacency-degenerate-negative-1 :
      is-negative-entry (k4-adjacency-spectrum m1)
    adjacency-degenerate-negative-2 :
      is-negative-entry (k4-adjacency-spectrum m2)
    adjacency-degenerate-negative-3 :
      is-negative-entry (k4-adjacency-spectrum m3)
    laplacian-distinguished-zero :
      is-zero-entry (k4-laplacian-spectrum m0)
    laplacian-degenerate-positive-1 :
      is-positive-entry (k4-laplacian-spectrum m1)
    laplacian-degenerate-positive-2 :
      is-positive-entry (k4-laplacian-spectrum m2)
    laplacian-degenerate-positive-3 :
      is-positive-entry (k4-laplacian-spectrum m3)
    trace-balance-locked :
      adjacency-positive-magnitude-sum
       $\equiv$  adjacency-negative-magnitude-sum
    sum-of-squares-locked :
      adjacency-sum-of-squares  $\equiv$  2 * edges canonicalK4Ledge
    perron-equals-degree-locked :
      magnitude (k4-adjacency-spectrum m0)
       $\equiv$  degree canonicalK4Ledge
    laplacian-degenerate-equals-vertex-count :
      laplacian-degenerate-magnitude  $\equiv$  vertices canonicalK4Ledge

k4-spectral-signature : K4SpectralSignature
k4-spectral-signature = record
{ distinguished-count      = 1

```

```

; degenerate-count          = 3
; distinguished-is-one      = refl
; degenerate-is-three       = refl
; total-equals-vertex-count = refl
; adjacency-distinguished-positive = tt
; adjacency-degenerate-negative-1 = tt
; adjacency-degenerate-negative-2 = tt
; adjacency-degenerate-negative-3 = tt
; laplacian-distinguished-zero = tt
; laplacian-degenerate-positive-1 = tt
; laplacian-degenerate-positive-2 = tt
; laplacian-degenerate-positive-3 = tt
; trace-balance-locked      = refl
; sum-of-squares-locked     = refl
; perron-equals-degree-locked = refl
; laplacian-degenerate-equals-vertex-count = refl
}

record OneAndThreeReading : Set where
field
spectral-source      : K4SpectralSignature
distinguished-count  : ℕ
degenerate-count     : ℕ
distinguished-is-one : distinguished-count ≡ 1
degenerate-is-three  : degenerate-count ≡ 3
physical-name-status : ProgrammeStatus
physical-name-is-physical-test :
physical-name-status ≡ physical-test

one-and-three-reading : OneAndThreeReading
one-and-three-reading = record
{ spectral-source      = k4-spectral-signature
; distinguished-count  = 1
; degenerate-count     = 3
; distinguished-is-one  = refl
; degenerate-is-three  = refl
; physical-name-status = physical-test
; physical-name-is-physical-test = refl
}

```

21 Spectral anchoring of the matching index and the charge duality

The cell structure forces three perfect matchings on K_4 . The adjacency spectrum of K_4 also forces three degenerate negative modes. Both counts are theorems, and both equal three. The record `MatchingDegenerateAnchor` below locks the equality of the two counts as a structural identity, not a coincidence. Which matching corresponds to which degenerate eigenvector is a permutation freedom inside the degenerate eigenspace, and is recorded as `physical-test`.

The charge duality acts as a $\mathbf{Z}_2$ on the elementary slot table. The Laplacian spectrum of K_4 has exactly two distinct eigenvalues, 0 and $V = 4$, and the gap between them is itself V . The record `ChargeDualitySpectralAnchor` locks this gap as the structural source of the binary charge axis. The identification of which spectral side is named “charged” and which is named “neutral” is a physical reading and is recorded as `physical-test`.

```

-- $ MatchingDegenerateAnchor.
record MatchingDegenerateAnchor : Set where
field
spectral-source      : K4SpectralSignature
matching-count-source : ℕ
spectral-degenerate-count : ℕ
matching-count-is-three : matching-count-source ≡ 3
spectral-degenerate-is-three :

```

```

spectral-degenerate-count = 3
counts-coincide           :
matching-count-source     = spectral-degenerate-count
permutation-status        : ProgrammeStatus
permutation-physical       :
permutation-status = physical-test

matching-degenerate-anchor : MatchingDegenerateAnchor
matching-degenerate-anchor = record
{ spectral-source      = k4-spectral-signature
; matching-count-source = 3
; spectral-degenerate-count = 3
; matching-count-is-three = refl
; spectral-degenerate-is-three = refl
; counts-coincide         = refl
; permutation-status      = physical-test
; permutation-physical    = refl
}

laplacian-distinct-eigenvalue-count : ℕ
laplacian-distinct-eigenvalue-count = 2

laplacian-spectral-gap : ℕ
laplacian-spectral-gap = vertices canonicalK4Ledge - 0

laplacian-spectral-gap-is-vertex-count :
laplacian-spectral-gap = vertices canonicalK4Ledge
laplacian-spectral-gap-is-vertex-count = refl

record ChargeDualitySpectralAnchor : Set where
field
spectral-source      : K4SpectralSignature
elementary-source    : ElementarySlotClosure
distinct-eigenvalue-count : ℕ
distinct-eigenvalue-is-two : distinct-eigenvalue-count = 2
charge-factor-source  : ℕ
charge-factor-is-two  : charge-factor-source = 2
counts-coincide       :
distinct-eigenvalue-count = charge-factor-source
spectral-gap           : ℕ
spectral-gap-equals-vertices :
spectral-gap = vertices canonicalK4Ledge
naming-status          : ProgrammeStatus
naming-physical         :
naming-status = physical-test

charge-duality-spectral-anchor : ChargeDualitySpectralAnchor
charge-duality-spectral-anchor = record
{ spectral-source      = k4-spectral-signature
; elementary-source    = elementary-slot-closure
; distinct-eigenvalue-count = 2
; distinct-eigenvalue-is-two = refl
; charge-factor-source  = 2
; charge-factor-is-two  = refl
; counts-coincide       = refl
; spectral-gap          = laplacian-spectral-gap
; spectral-gap-equals-vertices = refl
; naming-status         = physical-test
; naming-physical       = refl
}

record SpectralFactorisationOfTwelve : Set where
field
spectral-source      : K4SpectralSignature
matching-anchor       : MatchingDegenerateAnchor
duality-anchor        : ChargeDualitySpectralAnchor
distinguished-factor  : ℕ
degenerate-factor     : ℕ
distinct-eigenvalue-factor : ℕ
distinguished-is-one  : distinguished-factor = 1
degenerate-is-three   : degenerate-factor = 3

```



```

distinct-eigenvalue-is-two : distinct-eigenvalue-factor = 2
spectral-product           :  $\mathbb{N}$ 
spectral-product-locked    :
spectral-product
  = (1 + distinguished-factor) * degenerate-factor
  * distinct-eigenvalue-factor
spectral-product-is-twelve :
spectral-product = 12
coincides-with-elementary-slot-count :
spectral-product
  = ElementarySlotClosure.elementary-product
  elementary-slot-closure

spectral-factorisation-of-twelve : SpectralFactorisationOfTwelve
spectral-factorisation-of-twelve = record
{ spectral-source      = k4-spectral-signature
; matching-anchor      = matching-degenerate-anchor
; duality-anchor       = charge-duality-spectral-anchor
; distinguished-factor  = 1
; degenerate-factor    = 3
; distinct-eigenvalue-factor = 2
; distinguished-is-one  = refl
; degenerate-is-three  = refl
; distinct-eigenvalue-is-two = refl
; spectral-product     = 12
; spectral-product-locked = refl
; spectral-product-is-twelve = refl
; coincides-with-elementary-slot-count = refl
}

```

The integer twelve, already locked as the elementary slot count by the side, matching, and charge factors, now reappears as a product of purely spectral data: $(1+1)$ for the distinguished/degenerate split of the adjacency spectrum, three for the degenerate eigenspace dimension, and two for the distinct-eigenvalue count of the Laplacian. The two factorisations of twelve coincide as a structural identity, locked by `coincides-with-elementary-slot-count`.

22 Closure of the formal layer

The capstone record below packages the closures of this paper into a single object. It collects the depth survivor, the Planck bridge, the charged-sector and neutrino-sector first-order schemas, the two residual classifications, the neutral residual carrier, the final residual-corrected normal-hierarchy prediction, the carrier requirement, the intrinsic ledger extension, the cell table, the charge duality, the elementary slot count, and the slot-fill completeness. Every component is locked by an equation; nothing is left as an unproved field.

After this point the formal layer of the paper is closed. The remaining content is a physical reading: the side and matching assignment, the slot content identification, and the comparison of the classified ratios with measured low-energy masses. These items are recorded with the `physical-test` status throughout the paper; they are not formal theorems and do not weaken the closure of the formal layer.

```

-- $ FormalSchemaClosure.
record FormalSchemaClosure : Set where
field
  programme-status      : DistinctionToParticleMassProgramme
  neutral-schema        : NeutralCompleteSchema
  neutral-mass-readout   : NeutralMassInterface
  planck-anchored-candidate : PlanckAnchoredParticleMassCandidate
  residual-status       : ResidualLayerStatus
  carrier-requirement    : CarrierClosureRequirement
  intrinsic-closure      : IntrinsicCarrierClosure
  neutral-residual-carrier : NeutralResidualCarrierClassification

```

```

final-neutrino-prediction : ResidualCorrectedNormalHierarchyPrediction
cell-reading              : CellAssignmentReading
splitting-closure         : NeutralSplittingClosure
elementary-closure        : ElementarySlotClosure
fill-completeness         : SlotFillCompleteness
spectral-signature        : K4SpectralSignature
one-and-three             : OneAndThreeReading
matching-anchor           : MatchingDegenerateAnchor
duality-spectral-anchor   : ChargeDualitySpectralAnchor
twelve-spectral-factorisation : SpectralFactorisationOfTwelve
formal-layer-closed       : ProgrammeStatus
formal-layer-closed-here  :
formal-layer-closed = checked-here
physical-reading-status   : ProgrammeStatus
physical-reading-is-test  :
physical-reading-status = physical-test

formal-schema-closure : FormalSchemaClosure
formal-schema-closure = record
{
  programme-status = distinction-to-particle-mass-programme
  ; neutral-schema = neutral-complete-schema
  ; neutral-mass-readout = neutral-mass-interface
  ; planck-anchored-candidate = planck-anchored-particle-mass-candidate
  ; residual-status = residual-layer-status
  ; carrier-requirement = carrier-closure-requirement
  ; intrinsic-closure = intrinsic-carrier-closure
  ; neutral-residual-carrier = neutral-residual-carrier-classification
  ; final-neutrino-prediction = residual-corrected-normal-hierarchy-prediction
  ; cell-reading = cell-assignment-reading
  ; splitting-closure = neutral-splitting-closure
  ; elementary-closure = elementary-slot-closure
  ; fill-completeness = slot-fill-completeness
  ; spectral-signature = k4-spectral-signature
  ; one-and-three = one-and-three-reading
  ; matching-anchor = matching-degenerate-anchor
  ; duality-spectral-anchor = charge-duality-spectral-anchor
  ; twelve-spectral-factorisation = spectral-factorisation-of-twelve
  ; formal-layer-closed = checked-here
  ; formal-layer-closed-here = refl
  ; physical-reading-status = physical-test
  ; physical-reading-is-test = refl
}

```

Formal closure and empirical boundary

The formal layer of this paper is closed at the level displayed here. Inside the finite schema the neutral depth is uniquely 5 and its square is 25. The Planck route, the suppression exponents, the running correction, the tower action, and the charged-sector corrections are selected by finite filters. The two residual layers are classified as sector-asymmetric and splitting-asymmetric. The neutral residual carrier is further classified: matching weights are blocked by $\text{Aut}(K_4)$ -uniformity, and the surviving above-matching carrier is the coupling complement $295/306$, acting on the edge separator as $6 \cdot 295/306$. This supplies the final residual-corrected neutrino prediction with $m_3/m_2 = 1770/306$. The intrinsic extension of K_4 by its bipartition and perfect matching still supplies the charged sign/generation cell structure, producing six cells that coincide with the six edges, three generation slots per side, an intrinsic $\mathbf{Z}_2$ charge duality and twelve elementary slots. The slot-fill completeness predicate locks that no admitted slot is structurally empty. The capstone record `FormalSchemaClosure` packages all of these closures into a single object; every component is locked by an equation and nothing in the formal layer is selected by decimal agreement with experiment.

The physical reading begins after that point. Three items are listed throughout the paper as `physical-test`: the side and matching side assignment, the slot content identification, and the comparison of the classified ratios with measured low-energy masses, oscillation data, cosmological

bounds, beta-decay limits and neutrinoless double-beta constraints. A failed comparison would refute the physical reading. It would not refute the formal layer that closes this paper.

Empirical boundary and falsification

The formal result closes with exact rational data. Under the Planck-mass and electron-mass conventions displayed in this paper, the proposed physical readout is

$$m_1 = 0, \quad m_2 \approx 0.008589080150 \text{ eV}, \quad m_3 \approx 0.049681934200 \text{ eV},$$

$$\sum_i m_i \approx 0.058271014350 \text{ eV}, \quad \Delta m_{21}^2 \approx 7.377228 \cdot 10^{-5} \text{ eV}^2, \quad \Delta m_{31}^2 \approx 2.4682945859 \cdot 10^{-3} \text{ eV}^2.$$

These numbers are not fitted inputs. They are the electronvolt rendering of the exact Planck-ratio chain checked above. The comparison with oscillation fits, cosmological bounds, beta-decay searches and neutrinoless-double-beta searches is therefore the empirical boundary of the paper, not another Agda claim.

The comparison uses named external conventions: the Planck-mass convention, the electron-mass value, the oscillation-data fit and confidence interval, the cosmological mass-sum bound, and the nuclear-matrix-element convention used when a neutrinoless-double-beta effective mass is quoted. Changing one of these external data states changes the comparison line, not the formal chain. The exact Agda ratios carry no statistical error bars; the empirical comparison does.

The physical identification is falsified if the neutrino hierarchy is inverted, if the lightest neutrino is forced away from zero beyond the stated boundary, if the mass sum is excluded under the declared cosmological convention, if the two squared splittings cannot be kept inside the declared oscillation intervals, or if beta-decay or neutrinoless-double-beta data exclude the effective masses implied by the normal-hierarchy, $m_1 = 0$ reading. Such an outcome would reject the proposed physical interpretation of the finite ledger. It would not alter the checked arithmetic itself.

That is the intended exposure of the result: a closed finite schema, an exact rational mass readout, and a physical identification that remains answerable to measurement.